



Assignment

เว็บแอปพลิเคชันจำหน่ายหนังสือออนไลน์ C-BOOK

Online Bookstore Web Application C-BOOK

จัดทำโดย

นางสาวภาสินี เก่งกลาง

รหัสนักศึกษา 6706022510158

นางสาวพรญาณี สมดี

รหัสนักศึกษา 6706022510140

นางสาวพิตดา ช้างน้อย

รหัสนักศึกษา 6706022610361

นางสาวมณฑาทากาญจน์ โรจนะ

รหัสนักศึกษา 6706022510042

โครงการนี้เป็นส่วนหนึ่งของการศึกษาหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย ภาควิชาเทคโนโลยีสารสนเทศ
คณะเทคโนโลยีและการจัดการอุตสาหกรรม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อสรุปผลการออกแบบและพัฒนาเว็บแอปพลิเคชัน "C-BOOK" (ระบบจำหน่ายหนังสือออนไลน์) ซึ่งเป็นส่วนหนึ่งของโครงการปลายภาคในวิชา Full Stack Web Development โดยมีวัตถุประสงค์เพื่อประยุกต์ใช้ความรู้ด้านการพัฒนาเว็บแอปพลิเคชันที่ขับเคลื่อนด้วยฐานข้อมูล ตั้งแต่การออกแบบโครงสร้างฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ไปจนถึงการสร้างส่วนติดต่อผู้ใช้ที่สามารถใช้งานได้จริง

เนื้อหาภายในรายงานฉบับนี้ครอบคลุมขั้นตอนการพัฒนาเว็บแอปพลิเคชัน ตามรูปแบบโครงสร้าง MVC (Model-View-Controller) โดยใช้เทคโนโลยี Node.js และ Express.js เป็นระบบหลังบ้าน จัดการข้อมูลผ่าน Sequelize ORM ร่วมกับฐานข้อมูล SQLite และนำเสนอส่วนหน้าบ้านด้วย EJS Template Engine และ CSS โดยระบบประกอบด้วยฟังก์ชันพื้นฐานที่สำคัญ ได้แก่ ระบบจัดการข้อมูลแบบ CRUD (Create, Read, Update, Delete) ครอบคลุมทุกตาราง และการจัดทำหน้ารายงานวิเคราะห์ข้อมูล เช่น รายงานยอดขายสินค้า และสถานะรายการสต็อกสินค้า

ผู้จัดทำหวังเป็นอย่างยิ่งว่าเว็บแอปพลิเคชัน C-BOOK และรายงานฉบับนี้ จะเป็นแนวทางที่เป็นประโยชน์ในการศึกษาการพัฒนา Full Stack System ที่มีความซับซ้อนของข้อมูลในโลกจริง และสามารถนำไปต่อยอดในการพัฒนาซอฟต์แวร์ที่มีประสิทธิภาพต่อไปในอนาคต

คณะผู้จัดทำ

สารบัญ

	หน้า
คำนำ	ก
สารบัญ	ข
สารบัญภาพ	ค
สารบัญภาพ (ต่อ)	ง
สารบัญภาพ (ต่อ)	จ
สารบัญภาพ (ต่อ)	ฉ
สารบัญตาราง	ช
บทที่ 1 บทนำ	8
1.1 วัตถุประสงค์ของโครงการ	8
1.2 ขอบเขตของโครงการ	8
1.3 ประโยชน์ที่คาดว่าจะได้รับ	8
บทที่ 2 การออกแบบเว็บแอปพลิเคชัน เครื่องมือ และเทคโนโลยีที่ใช้	9
2.1 การออกแบบฐานข้อมูล	9
2.2 การกำหนดความสัมพันธ์ระหว่างตาราง	10
2.3 แผนภาพความสัมพันธ์ของเอนทิตี (ER Diagram)	11
2.4 การออกแบบหน้ารายงานข้อมูล (Reporting)	11
2.5 เครื่องมือและเทคโนโลยีที่ใช้	12
บทที่ 3 คู่มือการใช้งานเว็บแอปพลิเคชันจำหน่ายหนังสือออนไลน์ C-BOOK	13
3.1 คู่มือสำหรับผู้ใช้งาน (Member / users)	13
3.2 คู่มือสำหรับผู้ดูแลระบบ (Admin)	17
3.3 คู่มือการติดตั้งเว็บแอปพลิเคชันจำหน่ายหนังสือออนไลน์ C-BOOK	25
บทที่ 4 การสร้างระบบและการเขียนโปรแกรม (Code)	27
4.1 โครงสร้างโปรเจกต์ MVC Pattern	27
4.2 โครงสร้างตารางฐานข้อมูลโดยใช้ไลบรารี Sequelize (ORM)	28
4.3 ส่วนฝั่งหลังบ้าน (Backend - API Server)	32
4.4 ส่วนฝั่งหน้าบ้าน (Frontend - UI Web Server)	55
บทที่ 5 ปัญหา อุปสรรค และแนวทางการแก้ไข	74
5.1 สรุปผลการดำเนินงาน	74
5.2 ปัญหาและอุปสรรค	74
5.3 แนวทางการแก้ไข	74

สารบัญภาพ

ภาพที่	หน้า
ภาพที่ 2-1 แผนภาพความสัมพันธ์ของเอนทิตี (ER Diagram)	11
ภาพที่ 3-1 หน้าสมัครสมาชิก	13
ภาพที่ 3-2 หน้าการเข้าสู่ระบบ	13
ภาพที่ 3-3 หน้าหลักและแสดงรายการหนังสือ	14
ภาพที่ 3-4 หน้าการค้นหาและกรองข้อมูลหนังสือ	14
ภาพที่ 3-5 หน้าตะกร้าสินค้า	15
ภาพที่ 3-6 หน้ายืนยันการสั่งซื้อ	15
ภาพที่ 3-7 การแจ้งเตือนคำสั่งซื้อสำเร็จ	16
ภาพที่ 3-8 หน้าช่องทางการติดต่อ	16
ภาพที่ 3-9 เมนูการออกจากระบบ	16
ภาพที่ 3-10 หน้าการเข้าสู่ระบบสำหรับแอดมิน	17
ภาพที่ 3-11 หน้าแรกสำหรับผู้ดูแลระบบ	17
ภาพที่ 3-12 หน้าคำสั่งซื้อ (Order Management)	18
ภาพที่ 3-13 หน้ารายละเอียดคำสั่งซื้อ (Order Details)	18
ภาพที่ 3-14 หน้าจัดการข้อมูลหนังสือ (Book Management)	19
ภาพที่ 3-15 หน้าเพิ่มข้อมูลหนังสือใหม่ (Add New Book)	19
ภาพที่ 3-16 หน้าแก้ไขข้อมูลหนังสือ (Edit Book)	20
ภาพที่ 3-17 การลบข้อมูลหนังสือ (Delete Book)	20
ภาพที่ 3-18 หน้าจัดการหมวดหมู่หนังสือ (Category Management)	21
ภาพที่ 3-19 หน้าเพิ่มข้อมูลหมวดหมู่หนังสือ (Create New Category)	21
ภาพที่ 3-20 หน้าแก้ไขข้อมูลหมวดหมู่หนังสือ (Edit Category)	21
ภาพที่ 3-21 การลบข้อมูลหมวดหมู่หนังสือ (Delete Category)	22
ภาพที่ 3-22 แจ้งเตือนไม่สามารถลบหมวดหมู่หนังสือ	22
ภาพที่ 3-23 หน้าจัดการข้อมูลสมาชิก (User Management)	22
ภาพที่ 3-24 หน้ารายงานสรุปยอดขายแยกตามหมวดหมู่ (Sales by Category Report)	23
ภาพที่ 3-25 หน้ารายละเอียดยอดขายรายหมวดหมู่ (Category Sales Detail)	23
ภาพที่ 3-26 หน้ารายงานวิเคราะห์สถานะสินค้าคงคลัง (Inventory Status Report)	24
ภาพที่ 3-27 หน้ารายงานวิเคราะห์สถานะสินค้าคงคลังตามการเลือกหมวดหมู่	24
ภาพที่ 3-28 ภาพแสดงเอกสารรายงานสถานะสินค้าคงคลัง	24
ภาพที่ 3-29 เมนูการออกจากระบบของผู้ดูแลระบบ	25

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
ภาพที่ 3-30 ภาพการตั้งค่าฝั่ง Backend (API Server)	25
ภาพที่ 3-31 ภาพการตั้งค่าฝั่ง Frontend (UI Server)	25
ภาพที่ 3-32 ภาพแสดงคำสั่งรันระบบหลังบ้าน	26
ภาพที่ 3-33 ภาพแสดงคำสั่งรันระบบหน้าบ้าน	26
ภาพที่ 3-34 ภาพการเข้าใช้งานเว็บแอปพลิเคชัน	26
ภาพที่ 4-1 โครงสร้างโปรเจกต์	27
ภาพที่ 4-2 คำสั่งสร้างโมเดลตารางผู้ใช้งาน	28
ภาพที่ 4-3 คำสั่งสร้างโมเดลตารางหมวดหมู่หนังสือ	29
ภาพที่ 4-4 คำสั่งสร้างโมเดลตารางหนังสือ	29
ภาพที่ 4-5 คำสั่งสร้างโมเดลตารางคำสั่งซื้อ	30
ภาพที่ 4-6 คำสั่งสร้างโมเดลตารางรายละเอียดคำสั่งซื้อ	30
ภาพที่ 4-7 การตั้งค่าการเชื่อมต่อและนำเข้าโครงสร้างฐานข้อมูล	31
ภาพที่ 4-8 ความสัมพันธ์ระหว่าง Category และ Books	31
ภาพที่ 4-9 ความสัมพันธ์ระหว่าง Users และ Order	31
ภาพที่ 4-10 ความสัมพันธ์ระหว่าง Order และ OrderDetail	31
ภาพที่ 4-11 ความสัมพันธ์ระหว่าง Books และ OrderDetail	32
ภาพที่ 4-12 การส่งออกโมเดลเพื่อนำไปใช้งาน	32
ภาพที่ 4-13 ภาพไฟล์ Middleware	32
ภาพที่ 4-14 คำสั่งไฟล์ server.js สำหรับรันเซิร์ฟเวอร์ระบบหลังบ้าน	33
ภาพที่ 4-15 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลหนังสือ	34
ภาพที่ 4-16 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลหนังสือ	35
ภาพที่ 4-17 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลสมาชิก	35
ภาพที่ 4-18 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลคำสั่งซื้อ	36
ภาพที่ 4-19 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลตะกร้าสินค้า	37
ภาพที่ 4-20 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลรายงาน	37
ภาพที่ 4-21 ภาพส่วนตั้งค่าและการจัดการไฟล์ภาพ	38
ภาพที่ 4-22 ภาพไฟล์ฟังก์ชัน getBookById	38
ภาพที่ 4-23 ภาพไฟล์ฟังก์ชัน listBooks	39
ภาพที่ 4-24 ภาพไฟล์ฟังก์ชัน createBook	40
ภาพที่ 4-25 ภาพไฟล์ฟังก์ชัน updateBook	40

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
ภาพที่ 4-26 ภาพไฟล์ฟังก์ชัน deleteBook	41
ภาพที่ 4-27 ฟังก์ชัน createCategory	41
ภาพที่ 4-28 ฟังก์ชัน listCategories	42
ภาพที่ 4-29 ฟังก์ชัน getCategoryById	43
ภาพที่ 4-30 ฟังก์ชัน updateCategory	43
ภาพที่ 4-31 ฟังก์ชัน deleteCategory	44
ภาพที่ 4-32 ฟังก์ชัน register	44
ภาพที่ 4-33 ฟังก์ชัน login	45
ภาพที่ 4-34 ฟังก์ชัน changeUserRole	46
ภาพที่ 4-35 ฟังก์ชัน deleteUser	46
ภาพที่ 4-36 ฟังก์ชัน listUsers	47
ภาพที่ 4-37 ฟังก์ชัน listOrders	48
ภาพที่ 4-38 ฟังก์ชัน getOrderById	49
ภาพที่ 4-39 ฟังก์ชัน updateOrderStatus	49
ภาพที่ 4-40 ฟังก์ชัน checkout	50
ภาพที่ 4-41 ฟังก์ชัน summarize(cart)	50
ภาพที่ 4-42 ฟังก์ชัน getSummary	51
ภาพที่ 4-43 ฟังก์ชัน addToCart	51
ภาพที่ 4-44 ฟังก์ชัน updateCart	52
ภาพที่ 4-45 ฟังก์ชัน dashboard	52
ภาพที่ 4-46 ฟังก์ชัน salesByCategory	53
ภาพที่ 4-47 ฟังก์ชัน salesByCategoryDetail	53
ภาพที่ 4-48 ฟังก์ชัน inventoryStatus	54
ภาพที่ 4-49 โครงสร้างไฟล์แพ็คเกจฝั่ง Backend	54
ภาพที่ 4-50 ส่วนการตั้งค่าเริ่มต้นฝั่ง frontend	55
ภาพที่ 4-51 ฟังก์ชันคัดกรองสิทธิ์ผู้ใช้งาน	55
ภาพที่ 4-52 ฟังก์ชันพื้นฐานฝั่งหน้าบ้าน	56
ภาพที่ 4-53 ฟังก์ชันหน้าแรกของเว็บไซต์ (Home Page Route)	58
ภาพที่ 4-54 ฟังก์ชัน Contact Page Route	59
ภาพที่ 4-55 ฟังก์ชันการจัดการหน้าจอลงเข้าสู่ระบบและสมัครสมาชิก	59

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
ภาพที่ 4-56 ฟังก์ชันการเข้าสู่ระบบ	60
ภาพที่ 4-57 ฟังก์ชันการสมัครสมาชิก	60
ภาพที่ 4-58 ฟังก์ชันการออกจากระบบ	60
ภาพที่ 4-59 ฟังก์ชันการเพิ่มสินค้าลงตะกร้า	61
ภาพที่ 4-60 ฟังก์ชันดึงข้อมูลสรุปตะกร้าสินค้า	61
ภาพที่ 4-61 ฟังก์ชันการปรับปรุงตะกร้าสินค้า	62
ภาพที่ 4-62 ฟังก์ชันเตรียมข้อมูลสำหรับหน้ายืนยันคำสั่งซื้อ	62
ภาพที่ 4-63 ฟังก์ชันหน้ายืนยันการสั่งซื้อ	63
ภาพที่ 4-64 ฟังก์ชันหน้าแรกสำหรับผู้ดูแลระบบ	63
ภาพที่ 4-65 ฟังก์ชันจัดการรายการหนังสือ	64
ภาพที่ 4-66 ฟังก์ชันการเพิ่มข้อมูลหนังสือใหม่	65
ภาพที่ 4-67 ฟังก์ชันเตรียมหน้าฟอร์มแก้ไขหนังสือ	66
ภาพที่ 4-68 ฟังก์ชันการบันทึกข้อมูลแก้ไขหนังสือ	66
ภาพที่ 4-69 ฟังก์ชันการลบข้อมูลหนังสือ	67
ภาพที่ 4-70 ฟังก์ชันแสดงผลหน้ารายการหมวดหมู่สินค้า	67
ภาพที่ 4-71 ฟังก์ชันสร้างหมวดหมู่หนังสือใหม่	68
ภาพที่ 4-72 ฟังก์ชันการแก้ไขข้อมูลหมวดหมู่หนังสือ	68
ภาพที่ 4-73 ฟังก์ชันการลบหมวดหมู่หนังสือ	69
ภาพที่ 4-74 ฟังก์ชันการข้อมูลผู้ใช้งาน	69
ภาพที่ 4-75 ฟังก์ชันการแก้ไขสิทธิ์ผู้ใช้งาน	70
ภาพที่ 4-76 ฟังก์ชันการลบข้อมูลผู้ใช้งาน	70
ภาพที่ 4-77 ฟังก์ชันจัดการรายการสั่งซื้อ	71
ภาพที่ 4-78 ฟังก์ชันแสดงรายละเอียดคำสั่งซื้อ	71
ภาพที่ 4-79 ฟังก์ชันรายงานสรุปยอดขายแยกตามหมวดหมู่	72
ภาพที่ 4-80 ฟังก์ชันรายงานยอดขายตามหมวดหมู่ที่เลือก	72
ภาพที่ 4-81 ฟังก์ชันรายงานสถานะสินค้าคงคลัง	73

สารบัญตาราง

ตารางที่	หน้า
ตารางที่ 2-1 ตารางข้อมูลผู้ใช้งาน (users)	9
ตารางที่ 2-2 ตารางข้อมูลหนังสือ (books)	9
ตารางที่ 2-3 ตารางหมวดหมู่หนังสือ (category)	10
ตารางที่ 2-4 ตารางคำสั่งซื้อหลัก (order)	10
ตารางที่ 2-5 ตารางรายละเอียดคำสั่งซื้อ (orderDetail)	10
ตารางที่ 2-6 เทคโนโลยีที่ใช้พัฒนา	12
ตารางที่ 2-7 เครื่องมือสำหรับการพัฒนา	12
ตารางที่ 4-1 เส้นทาง API สำหรับจัดการข้อมูลหนังสือ	34
ตารางที่ 4-2 เส้นทาง API สำหรับจัดการข้อมูลหมวดหมู่หนังสือ	34
ตารางที่ 4-3 เส้นทาง API สำหรับจัดการข้อมูลสมาชิก	35
ตารางที่ 4-4 เส้นทาง API สำหรับจัดการข้อมูลคำสั่งซื้อ	36
ตารางที่ 4-5 เส้นทาง API สำหรับจัดการข้อมูลตะกร้าสินค้า	36
ตารางที่ 4-6 เส้นทาง API สำหรับจัดการข้อมูลรายงาน	37
ตารางที่ 4-7 ตารางเส้นทางส่วนของผู้ใช้	56
ตารางที่ 4-8 ตารางเส้นทางส่วนจัดการตะกร้าสินค้า	56
ตารางที่ 4-9 ตารางเส้นทางส่วนการชำระเงิน	57
ตารางที่ 4-10 ตารางเส้นทางส่วนผู้ดูแลระบบ	57
ตารางที่ 4-11 ตารางเส้นทางส่วนของหน้ารายงาน	57

บทที่ 1

บทนำ

1.1 วัตถุประสงค์ของโครงการ

- 1.1.1 เพื่อพัฒนาแอปพลิเคชันจำหน่ายหนังสือออนไลน์
- 1.1.2 เพื่อฝึกทักษะการพัฒนาเว็บแอปพลิเคชันแบบ Full Stack
- 1.1.3 เพื่อออกแบบและพัฒนาระบบฐานข้อมูลเชิงสัมพันธ์ (Relational Database)
- 1.1.4 เพื่อฝึกกระบวนการคิดอย่างเป็นระบบ
- 1.1.5 เพื่อเรียนรู้การจัดการข้อมูลและไฟล์

1.2 ขอบเขตของโครงการ

1.2.1 ส่วนการจัดการข้อมูล (CRUD Operations)

รองรับการเพิ่ม, เรียกดู, แก้ไข และลบข้อมูล ในทุกตารางหลักของระบบ ประกอบด้วย

- 1.2.1.1 ระบบจัดการผู้ใช้งาน จัดการข้อมูลสมาชิก ชื่อ อีเมล และสถานะการใช้งาน
- 1.2.1.2 ระบบจัดการหมวดหมู่หนังสือ ชื่อประเภทหนังสือเพื่อใช้ในการจัดหมวดหมู่สินค้า
- 1.2.1.3 ระบบจัดการข้อมูลหนังสือ ชื่อหนังสือ ผู้แต่ง ราคา และจำนวนสินค้าคงเหลือ
- 1.2.1.4 ระบบจัดการคำสั่งซื้อ วันที่ พร้อมรายละเอียดรายการสินค้าผ่านตาราง Order Details

1.2.2 ส่วนการแสดงผลและส่วนติดต่อผู้ใช้ (UI/Frontend)

- 1.2.2.1 Navigation Bar เมนูหลักที่แสดงผลทุกหน้าเพื่อเชื่อมโยงไปหน้าที่อื่นๆ ของระบบ
- 1.2.2.2 มีระบบแจ้งเตือน Flash Messages เมื่อทำการสำเร็จหรือเกิดข้อผิดพลาด
- 1.2.2.3 JavaScript confirm ระบบยืนยันก่อนการลบข้อมูลทุกครั้งเพื่อป้องกันความผิดพลาด

1.2.3 ส่วนการรายงาน (Reports) พัฒนารายงานสรุปผล 2 รูปแบบที่ดึงข้อมูลข้ามตาราง

- 1.2.3.1 รายงานสรุปยอดขายแยกตามหมวดหมู่ แสดงรายได้รวมและจำนวนเล่มที่ขายได้จริง โดยมีตัวกรองเลือกตามประเภทหนังสือ
- 1.2.3.2 รายงานสถานะสต็อกสินค้าคงเหลือ แสดงรายการหนังสือที่จำนวนคงเหลือต่ำกว่าเกณฑ์ พร้อมส่วนสรุปจำนวนรายการสินค้าทั้งหมดที่มีปัญหา

1.3 ประโยชน์ที่คาดว่าจะได้รับ

- 1.3.1 พัฒนาทักษะการเขียนโปรแกรมแบบ Full Stack
- 1.3.2 ฝึกการคิดวิเคราะห์และแก้ไขปัญหา
- 1.3.3 ความเข้าใจเชิงลึกในการทำงานร่วมกับฐานข้อมูล

บทที่ 2

การออกแบบเว็บแอปพลิเคชัน เครื่องมือ และเทคโนโลยีที่ใช้

2.1 การออกแบบฐานข้อมูล

การพัฒนาเว็บแอปพลิเคชันจำหน่ายหนังสือออนไลน์ C-BOOK ได้มีการออกแบบฐานข้อมูลเชิงสัมพันธ์ (Relational Database) เพื่อรองรับฟังก์ชันการทำงานของระบบ โดยมีรายละเอียดโครงสร้างตารางและความสัมพันธ์ดังต่อไปนี้

2.1.1 โครงสร้างตารางข้อมูล (Data Dictionary & Schema)

ระบบประกอบไปด้วยตารางหลักจำนวน 5 ตาราง ดังนี้

ตารางที่ 2-1 ตารางข้อมูลผู้ใช้งาน (users)

ชื่อฟิลด์	ชนิดข้อมูล	ข้อกำหนด	ค่าเริ่มต้น	คำอธิบาย
user_id	INTEGER	PK, Auto Increment	-	รหัสผู้ใช้งาน
user_name	STRING	Not Null	-	ชื่อผู้ใช้งาน
user_email	STRING	Unique, Not Null	-	อีเมลสำหรับเข้าสู่ระบบ
password	STRING	Not Null	-	รหัสผ่าน
user_role	ENUM	-	member	สิทธิ์การใช้งาน (member/admin)
user_status	STRING	Not Null	active	สถานะผู้ใช้งาน

ตารางที่ 2-2 ตารางข้อมูลหนังสือ (books)

ชื่อฟิลด์	ชนิดข้อมูล	ข้อกำหนด	ค่าเริ่มต้น	คำอธิบาย
book_id	INTEGER	PK, Auto Increment	-	รหัสหนังสือ (รหัสหลัก)
title	STRING	Not Null	-	ชื่อหนังสือ
author	STRING	Not Null	-	ชื่อผู้แต่ง
price	DOUBLE	Not Null	-	ราคาหนังสือ
stock_quantity	INTEGER	-	0	จำนวนหนังสือคงเหลือในสต็อก
book_status	STRING	-	active'	สถานะการขายของหนังสือ
book_img	STRING	Allow Null	default_book.png'	ชื่อไฟล์รูปภาพปกหนังสือ
category_id	INTEGER	FK, Not Null	-	รหัสหมวดหมู่

ตารางที่ 2-3 ตารางหมวดหมู่หนังสือ (category)

ชื่อฟิลด์	ชนิดข้อมูล	ข้อกำหนด	ค่าเริ่มต้น	คำอธิบาย
category_id	INTEGER	PK, Auto Increment	-	รหัสหมวดหมู่
category_name	STRING	Unique, Not Null	-	ชื่อหมวดหมู่หนังสือ

ตารางที่ 2-4 ตารางคำสั่งซื้อหลัก (order)

ชื่อฟิลด์	ชนิดข้อมูล	ข้อกำหนด	ค่าเริ่มต้น	คำอธิบาย
order_id	INTEGER	PK, Auto Increment	-	รหัสคำสั่งซื้อ
order_date	DATE	-	-	วันและเวลาที่ทำการสั่งซื้อ
status	STRING	-	delivered	สถานะคำสั่งซื้อ
user_id	INTEGER	FK, Not Null	-	รหัสผู้ซื้อ

ตารางที่ 2-5 ตารางรายละเอียดคำสั่งซื้อ (orderDetail)

ชื่อฟิลด์	ชนิดข้อมูล	ข้อกำหนด	ค่าเริ่มต้น	คำอธิบาย
detail_id	INTEGER	PK, Auto Increment	-	รหัสรายละเอียดคำสั่งซื้อ
quantity	INTEGER	Not Null	-	จำนวนเล่มที่สั่งซื้อในรายการนี้
unit_price	DOUBLE	Not Null	-	ราคาต่อเล่ม
book_id	INTEGER	FK, Not Null	-	รหัสหนังสือที่สั่ง
order_id	INTEGER	FK, Not Null	-	รหัสคำสั่งซื้อหลัก

2.2 การกำหนดความสัมพันธ์ระหว่างตาราง

2.2.1 ความสัมพันธ์ระหว่าง category และ books (1 to Many)

หมวดหมู่ (category) 1 หมวดหมู่ สามารถมีหนังสือ (books) สังกัดอยู่ได้หลายเล่ม แต่หนังสือแต่ละเล่มจะจัดอยู่ในหมวดหมู่หลักเพียง 1 หมวดหมู่

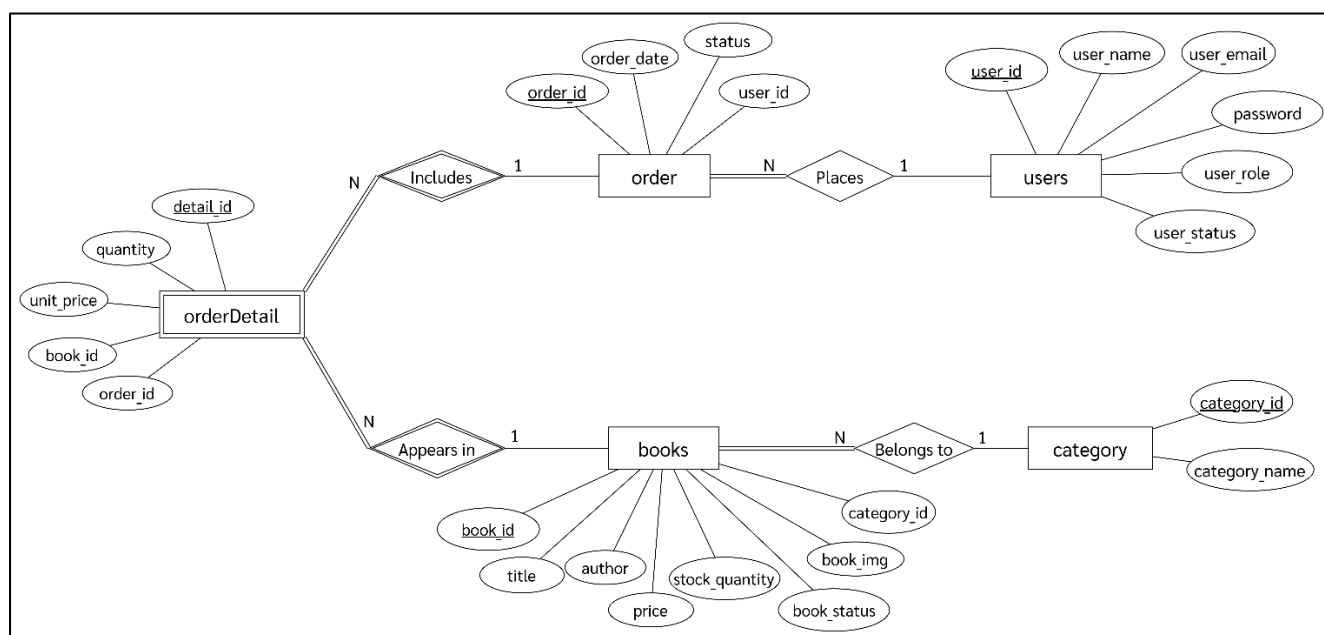
2.2.2 ความสัมพันธ์ระหว่าง users และ order (1 to Many)

ผู้ใช้งาน (user) 1 คน สามารถสร้างคำสั่งซื้อ (order) ได้หลายรายการ

2.2.3 ความสัมพันธ์ระหว่าง order และ books (Many to Many)

คำสั่งซื้อ 1 รายการประกอบด้วยหนังสือหลายเล่ม และหนังสือ 1 เล่มถูกสั่งซื้อซ้ำในหลายๆ คำสั่งซื้อ โดยใช้ตาราง orderDetail เป็น Junction Table ในการแปลงความสัมพันธ์แบบ Many-to-Many ให้อยู่ในรูป 1 to Many

2.3 แผนภาพความสัมพันธ์ของเอนทิตี (ER Diagram)



ภาพที่ 2-1 แผนภาพความสัมพันธ์ของเอนทิตี (ER Diagram)

2.4 การออกแบบหน้ารายงานข้อมูล (Reporting)

โครงสร้างฐานข้อมูลของระบบเว็บแอปพลิเคชันจำหน่ายหนังสือออนไลน์ C-BOOK ถูกออกแบบมาให้สามารถดึงข้อมูลเพื่อสรุปผลของระบบได้อย่างมีประสิทธิภาพ โดยเฉพาะรายงานหลัก 2 ส่วน ได้แก่

2.4.1 รายงานสรุปยอดขายแยกตามหมวดหมู่ (Sales by Category Report)

สามารถใช้การ Join ตาราง category, books, orderDetail และ order เพื่อคำนวณรายได้รวมและจำนวนเล่มที่ขายได้ แบ่งตามประเภทหนังสือ

2.4.2 รายงานวิเคราะห์สต็อกสินค้าคงเหลือ (Inventory Status Report)

สามารถดึงข้อมูลจากตาราง category, books โดยกรองจากฟิลด์ stock_quantity เพื่อแสดงรายการหนังสือที่มีจำนวนสต็อกน้อยกว่าที่กำหนด เพื่อช่วยในการวางแผนการสั่งซื้อหนังสือในแต่ละหมวดหมู่เพิ่มเติม

2.5 เครื่องมือและเทคโนโลยีที่ใช้

2.5.1 เทคโนโลยีที่ใช้พัฒนา

ตารางที่ 2-6 เทคโนโลยีที่ใช้พัฒนา

ชั้น (Layer)	เทคโนโลยี (Technology)	รายละเอียด (Description)
Frontend	HTML5 + CSS	โครงสร้างหน้าเว็บ จัดรูปแบบ Layout และ Responsive Design
	Axios	ไลบรารีสำหรับส่ง HTTP Request ไปดึงข้อมูล (JSON) จากฝั่ง Backend API มาแสดงผล
	Express-Session	จัดการเซสชัน (Session) ของผู้ใช้ เพื่อจดจำสถานะการเข้าสู่ระบบ
Backend	Express.js	Node.js Web Framework สำหรับจัดการ Routing และสร้าง API Endpoints
	Multer	มิดเดิลแวร์ (Middleware) สำหรับจัดการการอัปโหลดไฟล์รูปภาพ เข้าสู่เซิร์ฟเวอร์
	Bcryptjs	ไลบรารีสำหรับเข้ารหัสผ่านผู้ใช้งาน (Password Hashing) เพื่อความปลอดภัย
ORM	Sequelize	Object-Relational Mapping สำหรับเชื่อมต่อและจัดการคำสั่งฐานข้อมูล
Database	SQLite	ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ขนาดเล็ก จัดเก็บข้อมูลแบบไฟล์พกพาสะดวก
Runtime	Node.js	สภาพแวดล้อม (Runtime Environment) สำหรับรันภาษา JavaScript ฝั่งเซิร์ฟเวอร์

2.5.2 เครื่องมือสำหรับการพัฒนา

ตารางที่ 2-7 เครื่องมือสำหรับการพัฒนา

เครื่องมือ (Tool)	รายละเอียด (Description)
Visual Studio Code	โปรแกรมหลักที่ใช้ในการเขียน แก้ไข และจัดการ Source Code ของโปรเจกต์ทั้งหมด
Git	ระบบสำหรับจัดการและติดตามประวัติการเปลี่ยนแปลงของ Source Code ในแต่ละขั้นตอนการพัฒนา
GitHub	แพลตฟอร์มออนไลน์สำหรับจัดเก็บ Source Code เพื่อสำรองข้อมูลและช่วยให้สามารถทำงานร่วมกันได้

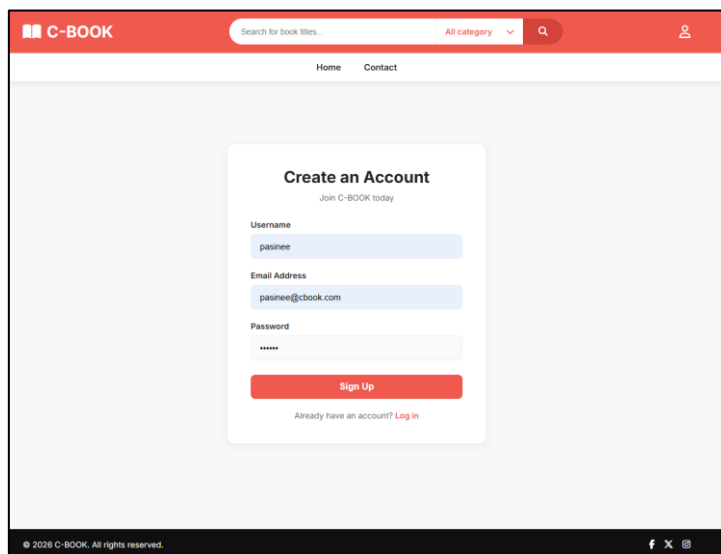
บทที่ 3

คู่มือการใช้งานเว็บแอปพลิเคชันจำหน่ายหนังสือออนไลน์ C-BOOK

3.1 คู่มือสำหรับผู้ใช้งาน (Member / users)

3.1.1 หน้าสมัครสมาชิก

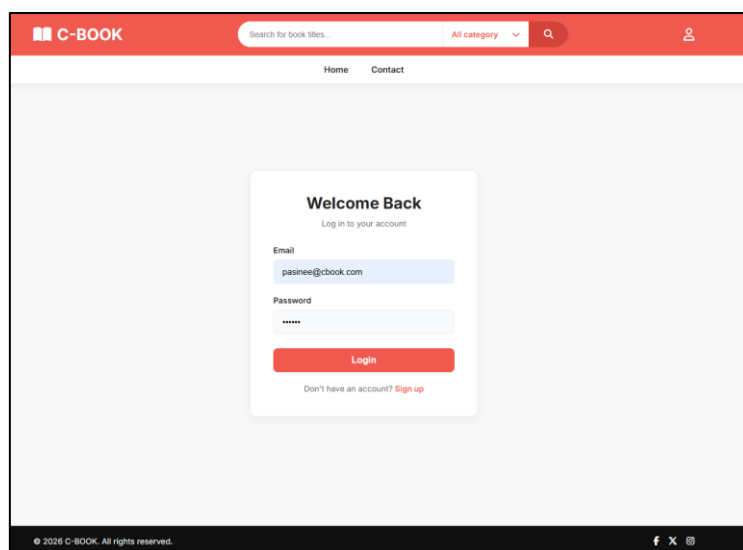
ผู้ใช้งานต้องเริ่มต้นการใช้งานด้วยการสมัครสมาชิก เพื่อเข้าใช้งานระบบ โดยกรอกข้อมูลชื่อผู้ใช้ อีเมลของผู้ใช้ และรหัสผ่าน 6 ตัวขึ้นไป จากนั้นกด sign up เพื่อสมัครสมาชิก



ภาพที่ 3-1 หน้าสมัครสมาชิก

3.1.2 หน้าการเข้าสู่ระบบ

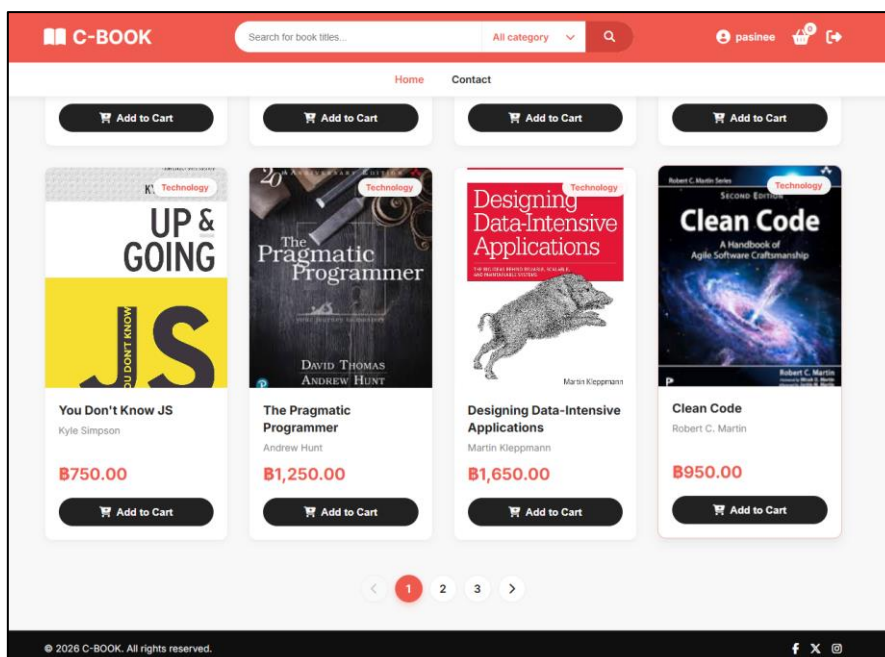
เมื่อสมัครสมาชิกสำเร็จ ผู้ใช้จะสามารถทำการกรอกข้อมูลอีเมล และรหัสผ่านที่สร้าง เพื่อเข้าสู่เว็บแอปพลิเคชัน



ภาพที่ 3-2 หน้าการเข้าสู่ระบบ

3.1.3 หน้าหลักและแสดงรายการหนังสือ

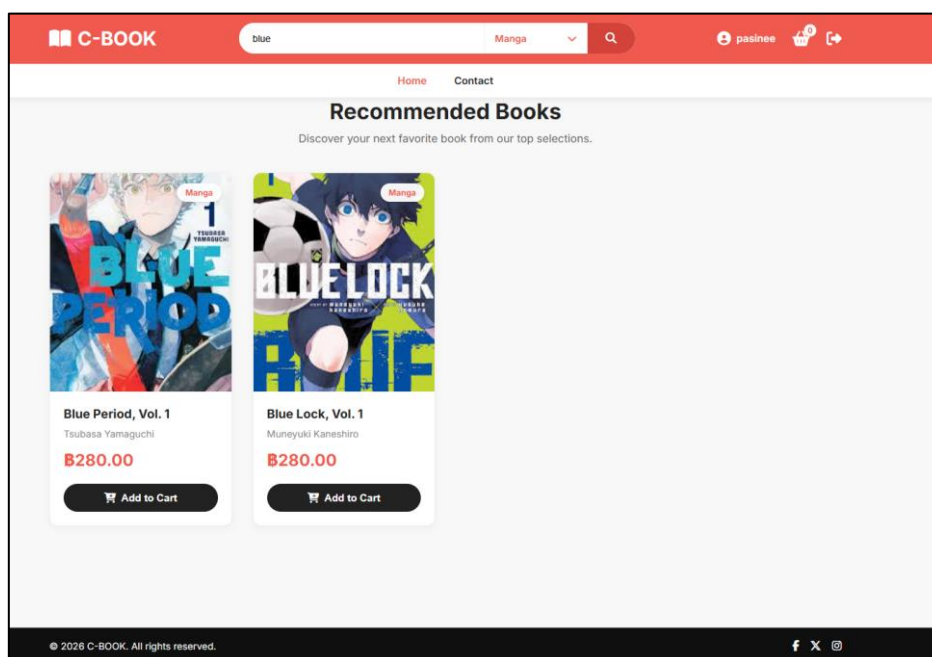
เมื่อเข้าสู่ระบบสำเร็จ จะพบกับหน้าหลักของระบบซึ่งแสดงรายการหนังสือที่วางจำหน่ายทั้งหมด ผู้ใช้สามารถกดไปที่หน้าถัดไป เพื่อดูรายการหนังสืออื่นๆ เพื่อเติม



ภาพที่ 3-3 หน้าหลักและแสดงรายการหนังสือ

3.1.4 หน้าการค้นหาและกรองข้อมูลหนังสือ

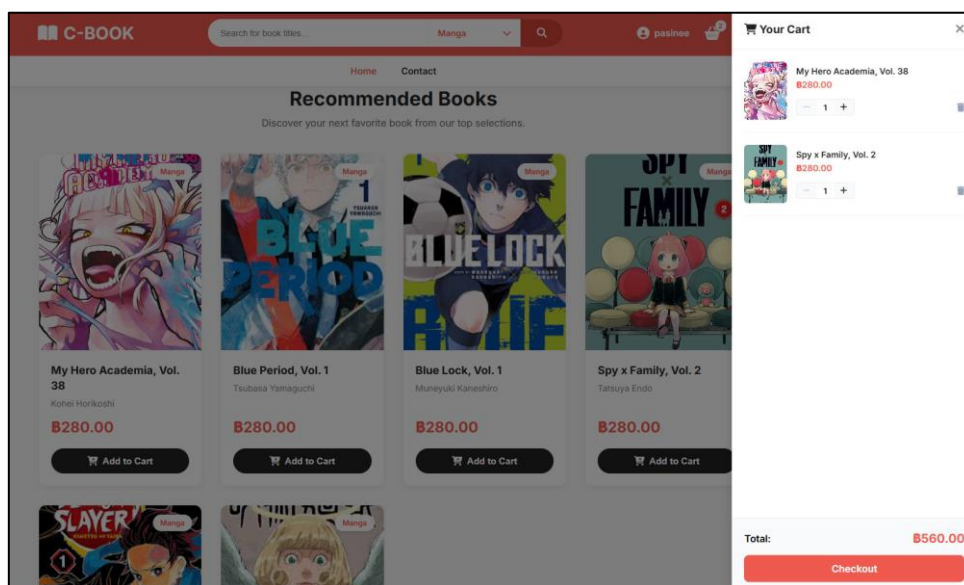
ผู้ใช้งานสามารถค้นหาซื้อหนังสือ และเลือกกรองตามหมวดหมู่ที่ต้องการได้ จากนั้นระบบจะแสดงผลรายการหนังสือที่ตรงกับเงื่อนไขขึ้นมา



ภาพที่ 3-4 หน้าการค้นหาและกรองข้อมูลหนังสือ

3.1.5 หน้าตะกร้าสินค้า

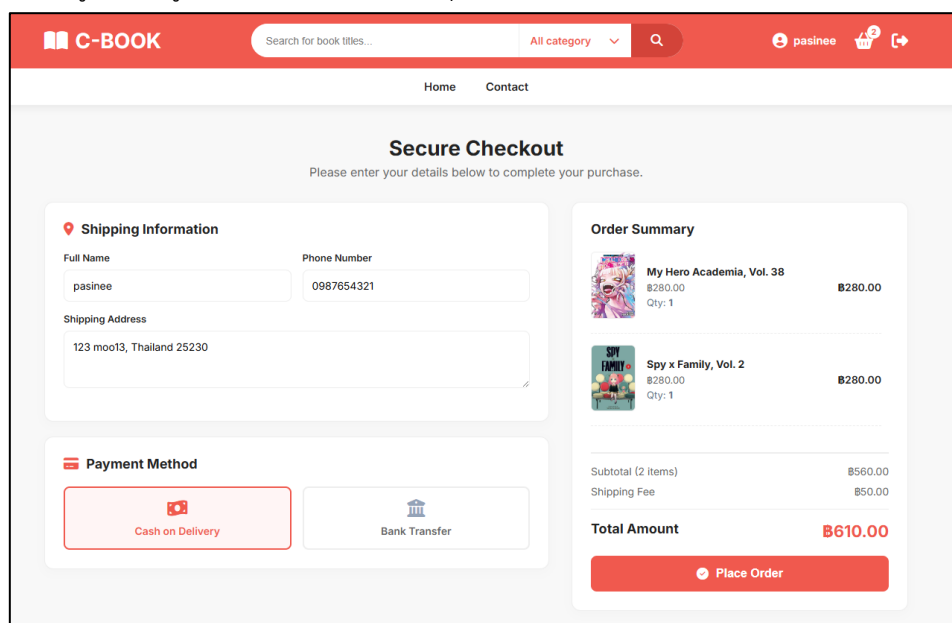
เมื่อผู้ใช้งานสามารถเพิ่มหนังสือลงในตะกร้าสินค้า จะปรากฏส่วนของตะกร้าสินค้าเพื่อแสดงรายการหนังสือที่ได้เลือกไว้ สามารถเพิ่ม-ลดจำนวนเล่ม หรือลบรายการหนังสือที่ไม่ต้องการออกได้ ระบบจะทำการแสดงยอดเงินรวม หากตรวจสอบความถูกต้องเรียบร้อยแล้ว ผู้ใช้สามารถกดปุ่ม Checkout เพื่อยืนยันและเข้าสู่ขั้นตอนการสั่งซื้อต่อไป



ภาพที่ 3-5 หน้าตะกร้าสินค้า

3.1.6 หน้ายืนยันการสั่งซื้อ

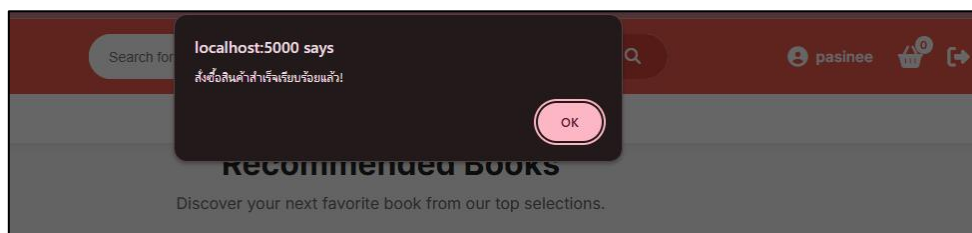
ส่วนนี้ระบบจะแสดงหน้าต่างสรุปรายการสินค้า พร้อมคำนวณค่าจัดส่งและยอดชำระเงินสุทธิไว้ เมื่อตรวจสอบข้อมูลความถูกต้องเรียบร้อยแล้ว กดปุ่ม Place Order เพื่อยืนยันคำสั่งซื้อ



ภาพที่ 3-6 หน้ายืนยันการสั่งซื้อ

3.1.7 การแจ้งเตือนคำสั่งซื้อสำเร็จ

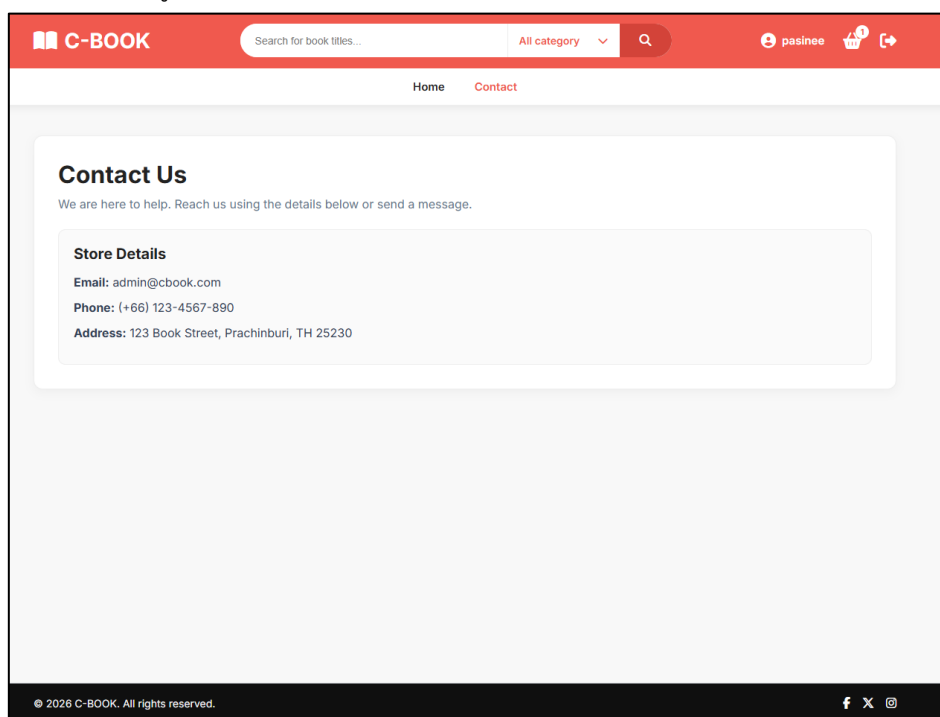
เมื่อผู้ใช้งานกดยืนยันคำสั่งซื้อระบบจะมีกล่องแจ้งเตือนว่าคำสั่งซื้อได้ถูกบันทึกแล้ว



ภาพที่ 3-7 การแจ้งเตือนคำสั่งซื้อสำเร็จ

3.1.8 หน้าช่องทางการติดต่อ

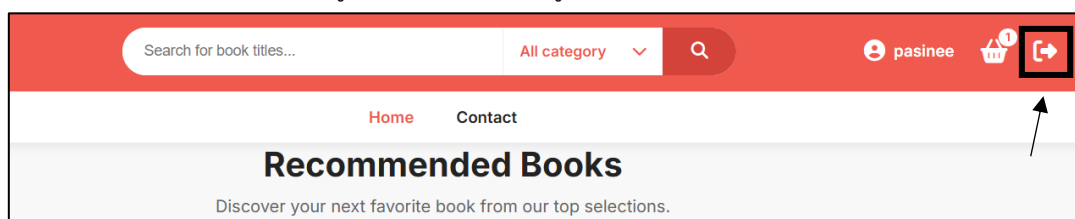
เมนู Contact บริเวณแถบเมนูด้านบน ระบบจะแสดงหน้าช่องทางการติดต่อของเว็บไซต์ โดยหน้านี้จะแสดงรายละเอียดข้อมูลของร้านค้า ในกรณีที่มียกข้อสงสัยหรือต้องการความช่วยเหลือเพิ่มเติม



ภาพที่ 3-8 หน้าช่องทางการติดต่อ

3.1.9 เมนูการออกจากระบบ

เมื่อทำการรายการเสร็จสิ้นผู้ใช้สามารถเลือกเมนูเพื่อทำการออกจากระบบได้



ภาพที่ 3-9 เมนูการออกจากระบบ

3.2 คู่มือสำหรับผู้ดูแลระบบ (Admin)

3.2.1 หน้าการเข้าสู่ระบบสำหรับผู้ดูแลระบบ

แอดมินจะต้องทำการเข้าสู่ระบบโดยการกรอกอีเมล รหัสผ่าน เพื่อเข้าสู่หน้าระบบจัดการข้อมูล เช่น หนังสือ หมวดหมู่ และรายงาน

ภาพที่ 3-10 หน้าการเข้าสู่ระบบสำหรับผู้ดูแลระบบ

3.2.2 หน้าแรกสำหรับผู้ดูแลระบบ

หน้านี้จะแสดงภาพรวมการดำเนินงานของร้านค้า โดยมีแถบเมนูหลักของการจัดการต่างๆ สำหรับพื้นที่แสดงผลหลัก จะประกอบด้วยป้ายสรุปข้อมูลสถิติที่สำคัญ ได้แก่ จำนวนคำสั่งซื้อทั้งหมด จำนวนหนังสือรวมที่ขายได้ และจำนวนลูกค้าสมาชิกในระบบ มีการแสดงตารางรายการคำสั่งซื้อล่าสุด 5 รายการ

Order ID	Date	Customer	Status	Items	Amount
#ORD-0011	07/03/2026	pasinee	Delivered	2	560.00
#ORD-0010	05/03/2026	pornyanee	Delivered	3	840.00
#ORD-0009	02/03/2026	Montakan	Delivered	2	840.00
#ORD-0008	02/03/2026	Phiyada	Delivered	3	1,520.00
#ORD-0007	02/03/2026	pornyanee	Delivered	3	840.00

ภาพที่ 3-11 หน้าแรกสำหรับผู้ดูแลระบบ

3.2.3 หน้าจัดการคำสั่งซื้อ (Order Management)

เมนู List Orders จะแสดงตารางรายการคำสั่งซื้อทั้งหมดที่เข้ามาในระบบ โดยระบุรหัสคำสั่งซื้อ (Order ID) วันที่ทำการขาย ชื่อลูกค้า จำนวนรายการ ยอดเงินรวม และสถานะปัจจุบัน สามารถค้นหาคำสั่งซื้อเฉพาะรายการได้ผ่านช่องค้นหา หรือใช้ตัวกรองสถานะเพื่อความสะดวกรวดเร็ว หากต้องการดูรายละเอียดของคำสั่งซื้อนั้น ๆ สามารถคลิกที่ปุ่มสัญลักษณ์รูปดวงตา (Actions)

Order ID	Date	Customer	Items	Total Amount	Status	Actions
#ORD-0011	07/03/2026	pasinee	2 Items	560.00	delivered	👁
#ORD-0010	05/03/2026	pornyane	3 Items	840.00	delivered	👁
#ORD-0009	02/03/2026	Montakan	2 Items	840.00	delivered	👁
#ORD-0008	02/03/2026	Phiyada	3 Items	1,520.00	delivered	👁
#ORD-0007	02/03/2026	pornyane	3 Items	840.00	delivered	👁
#ORD-0006	02/03/2026	pornyane	2 Items	1,930.00	delivered	👁
#ORD-0005	02/03/2026	pornyane	2 Items	770.00	delivered	👁
#ORD-0004	01/03/2026	Unknown User	3 Items	2,210.00	delivered	👁

ภาพที่ 3-12 หน้าคำสั่งซื้อ (Order Management)

3.2.4 หน้ารายละเอียดคำสั่งซื้อ (Order Details)

ระบบจะแสดงข้อมูลรายละเอียดของหมายเลขคำสั่งซื้อ โดยประกอบด้วยข้อมูลหลักๆ คือข้อมูลลูกค้า และรายการหนังสือที่สั่งซื้อ วันที่ หมายเลขคำสั่งซื้อ พร้อมสรุปยอดชำระเงินทั้งหมด

ITEMS NAME	QUANTITY	PRICE	TOTAL
My Hero Academia, Vol. 38	1	280.00	280.00
Spy x Family, Vol. 2	1	280.00	280.00
Subtotal			560.00
TOTAL AMOUNT			560.00

ภาพที่ 3-13 หน้ารายละเอียดคำสั่งซื้อ (Order Details)

3.2.5 หน้าจัดการข้อมูลหนังสือ (Book Management)

หน้านี้ใช้สำหรับจัดการข้อมูลหนังสือทั้งหมดภายในระบบ โดยตารางจะแสดงรายละเอียดสำคัญ ได้แก่ ภาพปกหนังสือ ชื่อเรื่อง ชื่อผู้แต่ง หมวดหมู่ ราคา จำนวนสต็อกคงเหลือ และสถานะการวางจำหน่าย สามารถใช้แถบค้นหา หรือตัวกรองข้อมูลตามหมวดหมู่และสถานะ เพื่อค้นหาหนังสือที่ต้องการได้ นอกจากนี้ รวมถึงสามารถคลิกที่ปุ่ม เพื่อแก้ไขรายละเอียด หรือลบ ข้อมูลหนังสือที่มีอยู่

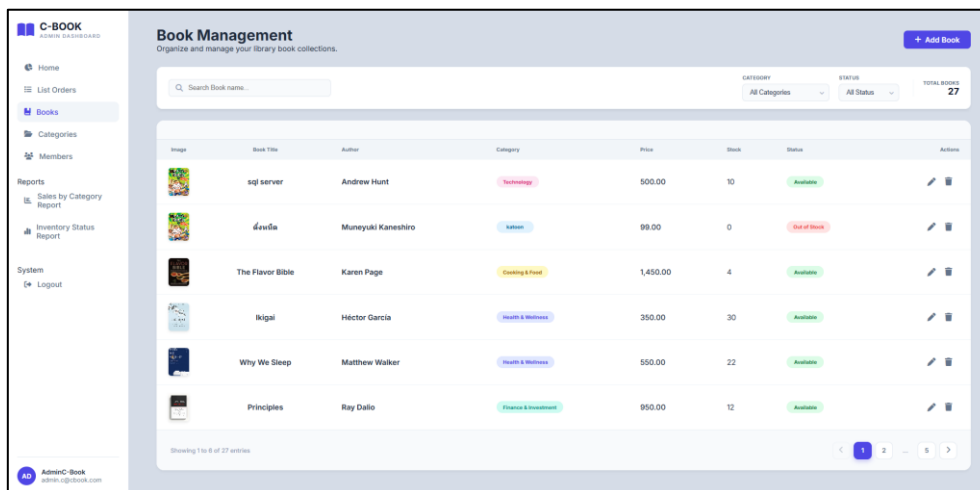
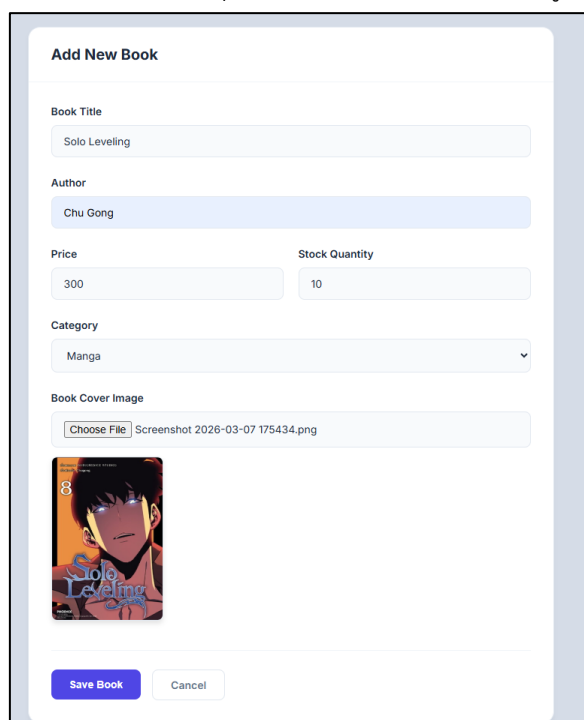


Image	Book Title	Author	Category	Price	Stock	Status	Actions
	sqj server	Andrew Hunt	Technology	500.00	10	Available	
	ฝันดี	Muneyuki Kaneshiro	Music	99.00	0	Out of Stock	
	The Flavor Bible	Karen Page	Cooking & Food	1,450.00	4	Available	
	Ikigai	Hector Garcia	Health & Wellness	350.00	30	Available	
	Why We Sleep	Matthew Walker	Health & Wellness	550.00	22	Available	
	Principles	Ray Dalio	Finance & Investment	950.00	12	Available	

ภาพที่ 3-14 หน้าจัดการข้อมูลหนังสือ (Book Management)

3.2.6 หน้าเพิ่มข้อมูลหนังสือใหม่ (Add New Book)

ผู้ดูแลระบบจะต้องทำการกรอกรายละเอียดของหนังสือให้ครบถ้วน เมื่อตรวจสอบความถูกต้องของข้อมูลทั้งหมดเรียบร้อยแล้ว สามารถคลิกที่ปุ่ม Save Book เพื่อบันทึกข้อมูลเข้าสู่ระบบ



Add New Book

Book Title: Solo Leveling

Author: Chu Gong

Price: 300 Stock Quantity: 10

Category: Manga

Book Cover Image:

Save Book **Cancel**

ภาพที่ 3-15 หน้าเพิ่มข้อมูลหนังสือใหม่ (Add New Book)

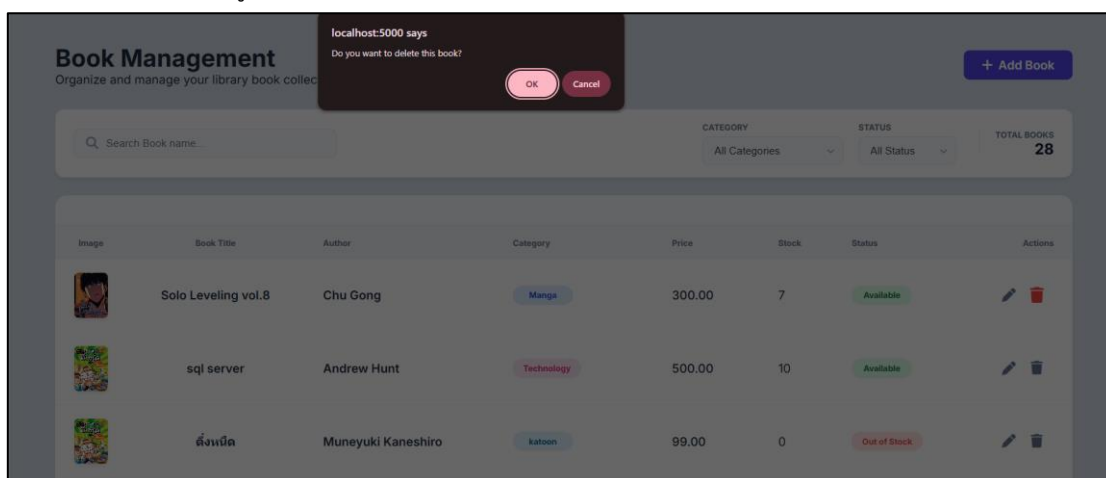
3.2.7 หน้าแก้ไขข้อมูลหนังสือ (Edit Book)

ผู้ดูแลระบบต้องการแก้ไขข้อมูล โดยระบบจะทำการดึงข้อมูลเดิมของหนังสือเล่มนั้น ๆ มาแสดงในแบบฟอร์มให้อัตโนมัติ เมื่อทำการปรับปรุงข้อมูลตามต้องการเสร็จสิ้นแล้ว ให้คลิกที่ปุ่ม Save Changes เพื่อบันทึกการเปลี่ยนแปลง

ภาพที่ 3-16 หน้าแก้ไขข้อมูลหนังสือ (Edit Book)

3.2.8 การลบข้อมูลหนังสือ (Delete Book)

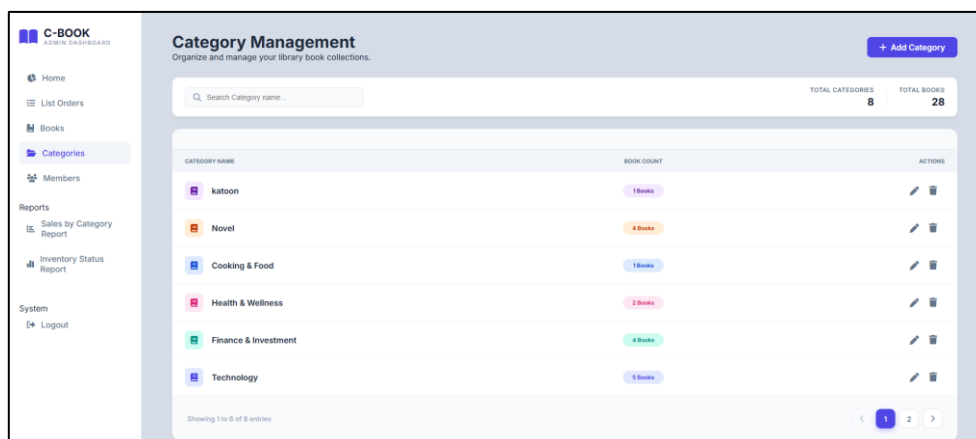
กรณีที่ผู้ดูแลระบบต้องการลบข้อมูลหนังสือ ระบบจะแสดงหน้าต่างแจ้งเตือน (Confirmation Pop-up) ขึ้นมาถามเพื่อยืนยันคำสั่ง เพื่อป้องกันความผิดพลาดจากการลบข้อมูลโดยไม่ตั้งใจ สามารถกดปุ่ม OK เพื่อยืนยันการลบข้อมูลออกจากระบบ



ภาพที่ 3-17 การลบข้อมูลหนังสือ (Delete Book)

3.2.9 หน้าจัดการหมวดหมู่หนังสือ (Category Management)

หน้านี้ใช้สำหรับจัดการกลุ่มของหนังสือภายในระบบ โดยระบบจะแสดงตารางรายการหมวดหมู่ ประกอบด้วย ชื่อหมวดหมู่ และป้ายแสดงจำนวนหนังสือที่มีอยู่ในหมวดหมู่ ระบบสามารถค้นหาหมวดหมู่ได้ผ่านช่องค้นหา ยังมีเครื่องมือสำหรับแก้ไข และลบ ข้อมูลหมวดหมู่ที่ต้องการได้อีกด้วย



ภาพที่ 3-18 หน้าจัดการหมวดหมู่หนังสือ (Category Management)

3.2.10 หน้าเพิ่มข้อมูลหมวดหมู่หนังสือ (Create New Category)

สำหรับการสร้างหมวดหมู่ใหม่ ผู้ดูแลระบบจะต้องทำการกรอกชื่อหมวดหมู่ที่ต้องการ ลงในช่องกรอกข้อมูลให้เรียบร้อย จากนั้นคลิกที่ปุ่ม Create Category เพื่อบันทึกข้อมูลหมวดหมู่ใหม่เข้าสู่ระบบ

ภาพที่ 3-19 หน้าเพิ่มข้อมูลหมวดหมู่หนังสือ (Create New Category)

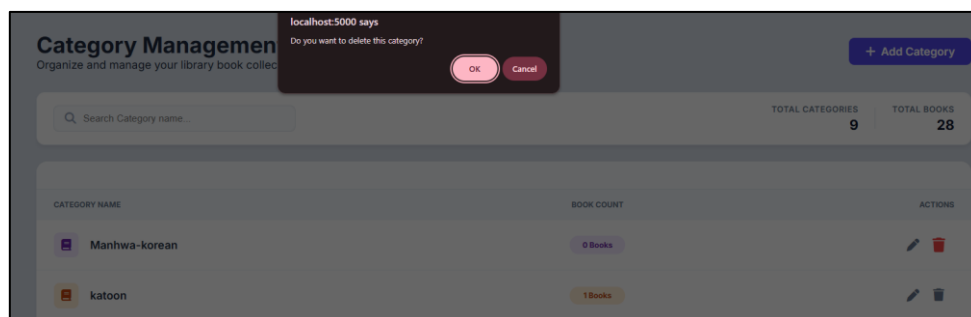
3.2.11 หน้าแก้ไขข้อมูลหมวดหมู่หนังสือ (Edit Category)

ผู้ดูแลระบบสามารถทำการแก้ไขชื่อหมวดหมู่หนังสือ เมื่อเสร็จสิ้นแล้วสามารถคลิกที่ปุ่ม Update Category เพื่อบันทึกข้อมูล

ภาพที่ 3-20 หน้าแก้ไขข้อมูลหมวดหมู่หนังสือ (Edit Category)

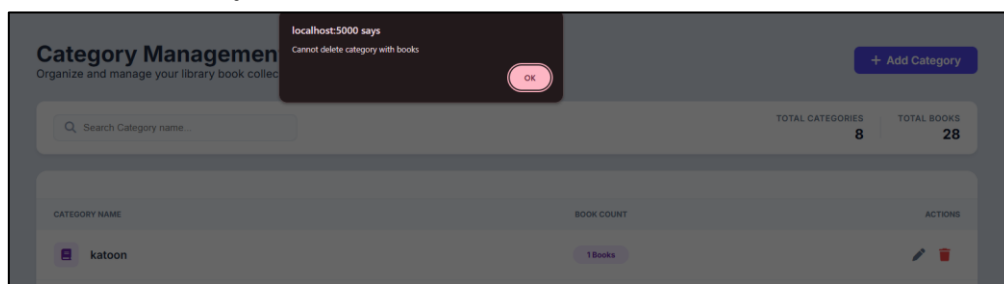
3.2.12 การลบข้อมูลหมวดหมู่หนังสือ (Delete Category)

เมื่อต้องการลบข้อมูลหมวดหมู่หนังสือออกจากระบบ ระบบจะแสดงหน้าต่างแจ้งเตือน (Confirmation Pop-up) ขึ้นมาถามเพื่อยืนยันว่าต้องการลบ หากต้องการลบข้อมูลจริง สามารถกดปุ่ม OK เพื่อยืนยันการลบข้อมูล



ภาพที่ 3-21 การลบข้อมูลหมวดหมู่หนังสือ (Delete Category)

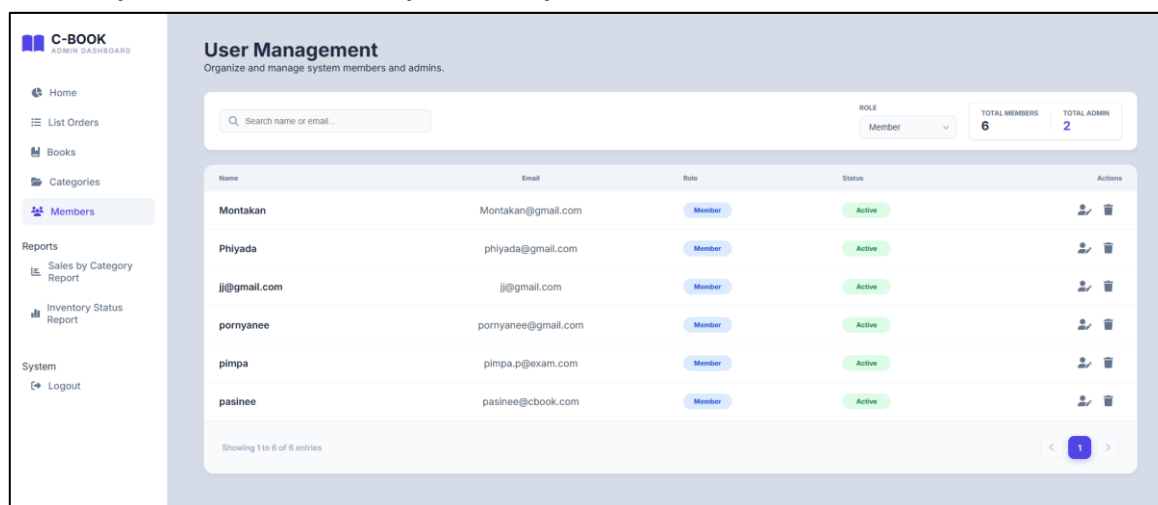
แต่หากหมวดหมู่นั้นมีรายการหนังสือระบบจะแจ้งว่า จะไม่สามารถลบหมวดหมู่นั้นได้ เพื่อป้องกันรายการหนังสือที่ไม่มีหมวดหมู่



ภาพที่ 3-22 แจ้งเตือนไม่สามารถลบหมวดหมู่หนังสือ

3.2.13 หน้าจัดการข้อมูลสมาชิก (User Management)

ระบบจะแสดงตารางรายชื่อผู้ใช้งาน สิทธิการใช้งาน และสถานะของบัญชี ผู้ดูแลระบบสามารถค้นหาบัญชีผู้ใช้ได้ หรือใช้ตัวกรองเพื่อดูเฉพาะกลุ่มผู้ใช้งานตามสิทธิ์ที่กำหนดคือ member, admin



ภาพที่ 3-23 หน้าจัดการข้อมูลสมาชิก (User Management)

3.2.14 หน้ารายงานสรุปยอดขายแยกตามหมวดหมู่ (Sales by Category Report)

หน้านี้จะแสดงภาพรวมของรายได้ และจำนวนหนังสือที่ขายได้ ตามหมวดหมู่หนังสือ ตารางจัดอันดับยอดขาย แสดงข้อมูลประกอบด้วย ชื่อหมวดหมู่ จำนวนเล่มที่ขายได้ และรายได้รวมของหมวดหมู่นั้น ๆ พร้อมทั้งมีการสรุปยอดขายรวมทั้งหมด ผู้ดูแลระบบยังสามารถค้นหาข้อมูลเฉพาะหมวดหมู่ หรือคลิกที่ปุ่ม Print Report เพื่อสั่งพิมพ์รายงานออกมาเป็นเอกสาร และสามารถคลิกที่ปุ่ม View เพื่อดูรายละเอียดยอดขายของหมวดหมู่ที่เลือก

Sales by Category				
Report showing total revenue and books sold grouped by category.				
Search Category name...			TOTAL CATEGORIES	5
			TOTAL BOOKS	26
RANK	CATEGORY NAME	BOOKS SOLD (QTY)	TOTAL REVENUE	ACTION
1	Manga	13 pcs	฿3,640.00	View
2	Novel	6 pcs	฿2,230.00	View
3	Finance & Investment	4 pcs	฿1,680.00	View
4	Technology	2 pcs	฿2,400.00	View
5	Cooking & Food	1 pcs	฿1,450.00	View
GRAND TOTAL		26 pcs	฿11,400.00	

ภาพที่ 3-24 หน้ารายงานสรุปยอดขายแยกตามหมวดหมู่ (Sales by Category Report)

3.2.15 หน้ารายละเอียดยอดขายรายหมวดหมู่ (Category Sales Detail)

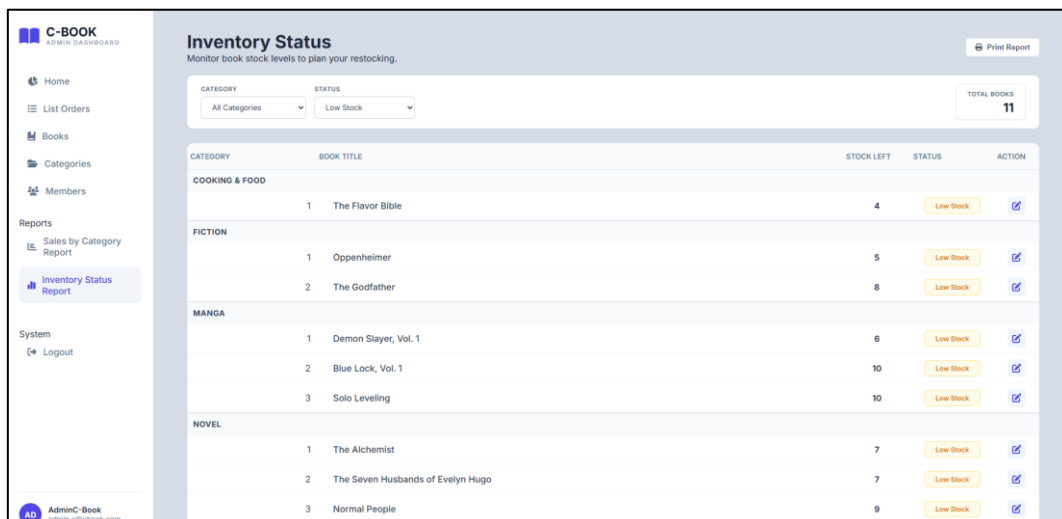
หน้านี้จะแสดงรายละเอียดเจาะลึกของการขายในหมวดหมู่นั้น ๆ โดยตารางแจกแจงรายการหนังสือแต่ละเล่มที่ขายได้ ประกอบด้วย ชื่อหนังสือ จำนวนเล่มที่ขายได้ และรายได้ที่เกิดจากหนังสือเล่มนั้น พร้อมทั้งสรุปยอดรวมสุทธิ ไว้ที่บรรทัดสุดท้ายของตาราง นอกจากนี้ ผู้ดูแลระบบยังสามารถคลิกที่ปุ่ม Print Report เพื่อสั่งพิมพ์เอกสารรายงานหน้านี้ หรือคลิกปุ่มย้อนกลับไปดูภาพรวมยอดขายของทุกหมวดหมู่ได้

Category Manga			
Detailed breakdown of books sold in this category			
Total Books Sold		Total Revenue	
13 pcs		฿3,640.00	
RANK	BOOK TITLE	QTY SOLD	REVENUE GENERATED
1	Spy x Family, Vol. 2	6 pcs	฿1,680.00
2	My Hero Academia, Vol. 38	5 pcs	฿1,400.00
3	Demon Slayer, Vol. 1	1 pcs	฿280.00
4	Blue Period, Vol. 1	1 pcs	฿280.00
GRAND TOTAL		13 pcs	฿3,640.00

ภาพที่ 3-25 หน้ารายละเอียดยอดขายรายหมวดหมู่ (Category Sales Detail)

3.2.16 หน้ารายงานวิเคราะห์สถานะสินค้าคงคลัง (Inventory Status Report)

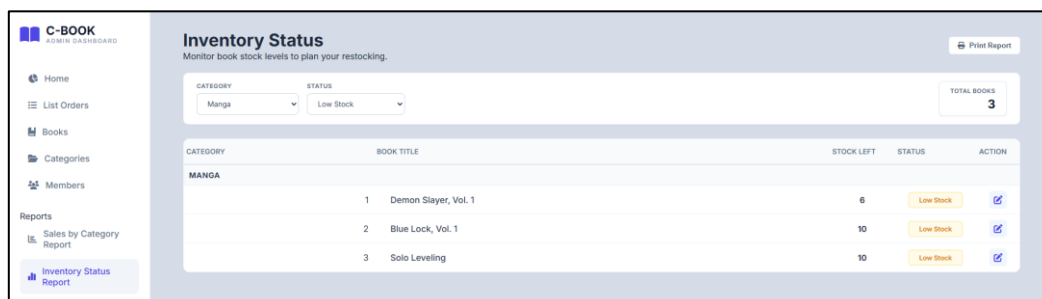
ผู้ดูแลระบบสามารถตรวจสอบปริมาณหนังสือคงเหลือในระบบและช่วยในการวางแผนการสั่งซื้อสินค้ามาเติมสต็อก โดยด้านบนจะมีตัวกรองข้อมูล ให้ผู้ดูแลระบบสามารถเลือกดูข้อมูลตามหมวดหมู่ หรือเลือกกรองเฉพาะสถานะของสินค้าที่ต้องการได้ มีปุ่ม Print Report สำหรับสั่งพิมพ์หน้ารายงานออกมาเป็นเอกสารได้



CATEGORY	BOOK TITLE	STOCK LEFT	STATUS	ACTION
COOKING & FOOD	1 The Flavor Bible	4	Low Stock	
FICTION	1 Oppenheimer	5	Low Stock	
	2 The Godfather	8	Low Stock	
MANGA	1 Demon Slayer, Vol. 1	6	Low Stock	
	2 Blue Lock, Vol. 1	10	Low Stock	
	3 Solo Leveling	10	Low Stock	
NOVEL	1 The Alchemist	7	Low Stock	
	2 The Seven Husbands of Evelyn Hugo	7	Low Stock	
	3 Normal People	9	Low Stock	

ภาพที่ 3-26 หน้ารายงานวิเคราะห์สถานะสินค้าคงคลัง (Inventory Status Report)

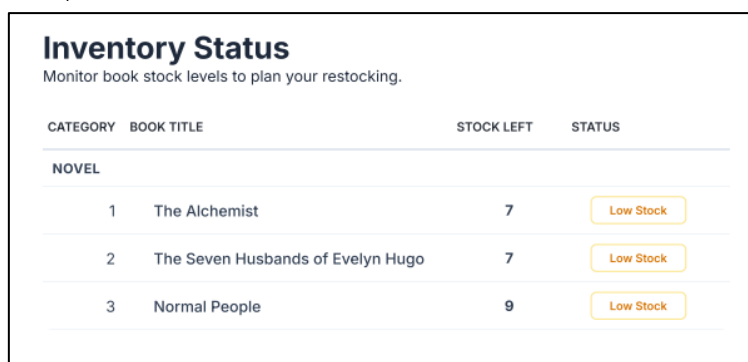
เมื่อมีการเลือกกรองรายการสินค้าคงคลังตามหมวดหมู่ และเลือกสถานะจนวนหนังสือที่มีจำนวนน้อยกว่า 10 เล่มเป็นต้นไป ระบบก็จะแสดงผลลัพธ์ออกมา



CATEGORY	BOOK TITLE	STOCK LEFT	STATUS	ACTION
MANGA	1 Demon Slayer, Vol. 1	6	Low Stock	
	2 Blue Lock, Vol. 1	10	Low Stock	
	3 Solo Leveling	10	Low Stock	

ภาพที่ 3-27 หน้ารายงานวิเคราะห์สถานะสินค้าคงคลังตามการเลือกหมวดหมู่

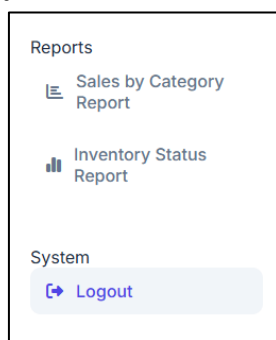
เมื่อมีการกดปุ่ม Print Report สำหรับสั่งพิมพ์หน้ารายงานออกมาเป็นเอกสาร



CATEGORY	BOOK TITLE	STOCK LEFT	STATUS
NOVEL			
1	The Alchemist	7	Low Stock
2	The Seven Husbands of Evelyn Hugo	7	Low Stock
3	Normal People	9	Low Stock

ภาพที่ 3-28 ภาพแสดงเอกสารรายงานสถานะสินค้าคงคลัง

3.2.17 เมนูการออกจากระบบของผู้ดูแลระบบ



ภาพที่ 3-29 เมนูการออกจากระบบของผู้ดูแลระบบ

3.3 คู่มือการติดตั้งเว็บแอปพลิเคชันจำหน่ายหนังสือออนไลน์ C-BOOK

3.3.1 สิ่งที่ต้องมีในเครื่องก่อนติดตั้ง

- Node.js (เวอร์ชัน 16.x หรือใหม่กว่า)
- Git (สำหรับ Clone โปรเจกต์)

3.3.2 โคลนโปรเจกต์ลงมาที่เครื่อง

Repositories: <https://github.com/iampasinee/C-BOOK-WEB-APP.git>

3.3.3 ตั้งค่าฝั่ง Backend (API Server)

- เปิด Terminal สำหรับจัดการหลังบ้าน
- เข้าไปที่โฟลเดอร์ backend
- ติดตั้งไลบรารีที่จำเป็น

```
pasinee@LAPTOP-QVEJHE1I MINGW64 /d/inet1/full stack/C-BOOK-WEB-APP (main)
● $ cd backend

pasinee@LAPTOP-QVEJHE1I MINGW64 /d/inet1/full stack/C-BOOK-WEB-APP/backend (main)
○ $ npm install
```

ภาพที่ 3-30 ภาพการตั้งค่าฝั่ง Backend (API Server)

3.3.4 ตั้งค่าฝั่ง Frontend (UI Server)

- เปิด Terminal สำหรับจัดการหน้าบ้าน
- เข้าไปที่โฟลเดอร์ frontend
- ติดตั้งไลบรารีที่จำเป็น

```
pasinee@LAPTOP-QVEJHE1I MINGW64 /d/inet1/full stack/C-BOOK-WEB-APP (main)
● $ cd frontend

pasinee@LAPTOP-QVEJHE1I MINGW64 /d/inet1/full stack/C-BOOK-WEB-APP/frontend (main)
○ $ npm install
```

ภาพที่ 3-31 ภาพการตั้งค่าฝั่ง Frontend (UI Server)

3.3.5 สั้รันระบบ (Start the Application)

คุณต้องรัน Server ทั้ง 2 ตัวพร้อมกัน

3.3.5.1 ที่ Terminal 1 (Backend)

```
pasinee@LAPTOP-QVEJHE1I MINGW64 /d/inet1/full stack/C-BOOK-WEB-APP/backend (main)
$ npm run dev

> c-book-backend@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (1) from .env -- tip: 🛑 prevent committing .env to code: https://dotenvx.com/precommit
Executing (default): SELECT name FROM sqlite_master WHERE type='table' AND name='users';
Executing (default): PRAGMA INDEX_LIST('users')
Executing (default): PRAGMA INDEX_INFO('sqlite_autoindex_users_1')
Executing (default): SELECT name FROM sqlite_master WHERE type='table' AND name='categories';
Executing (default): PRAGMA INDEX_LIST('categories')
Executing (default): PRAGMA INDEX_INFO('sqlite_autoindex_categories_1')
Executing (default): SELECT name FROM sqlite_master WHERE type='table' AND name='books';
Executing (default): PRAGMA INDEX_LIST('books')
Executing (default): SELECT name FROM sqlite_master WHERE type='table' AND name='orders';
Executing (default): PRAGMA INDEX_LIST('orders')
Executing (default): SELECT name FROM sqlite_master WHERE type='table' AND name='orderDetails';
Executing (default): PRAGMA INDEX_LIST('orderDetails')
C-BOOK backend API running at http://localhost:3000
```

ภาพที่ 3-32 ภาพแสดงคำสั่งรันระบบหลังบ้าน

3.3.5.2 ที่ Terminal 2 (Frontend)

```
pasinee@LAPTOP-QVEJHE1I MINGW64 /d/inet1/full stack/C-BOOK-WEB-APP/frontend (main)
$ npm run dev

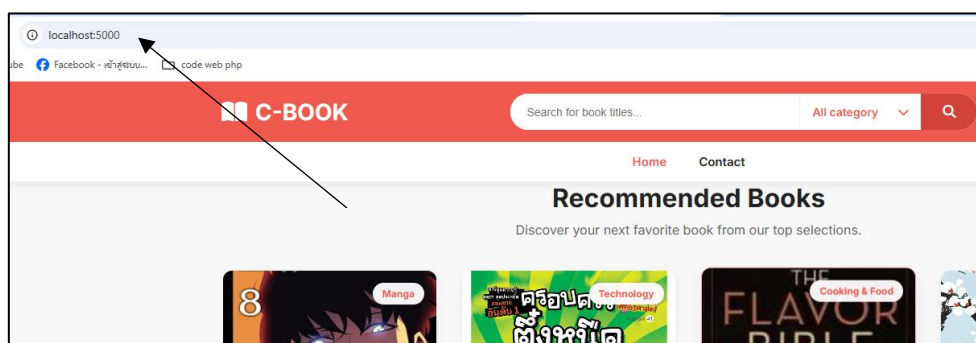
> c-book-frontend@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
[dotenv@17.3.1] injecting env (0) from .env -- tip: 🛑 prevent committing .env to code: https://dotenvx.com/precommit
C-BOOK FRONTEND running at http://localhost:5000
```

ภาพที่ 3-33 ภาพแสดงคำสั่งรันระบบหน้าบ้าน

3.3.6 การเข้าใช้งาน

เปิดเว็บเบราว์เซอร์และเข้าไปที่: <http://localhost:5000> (PORT ที่ Frontend ทำงานอยู่)

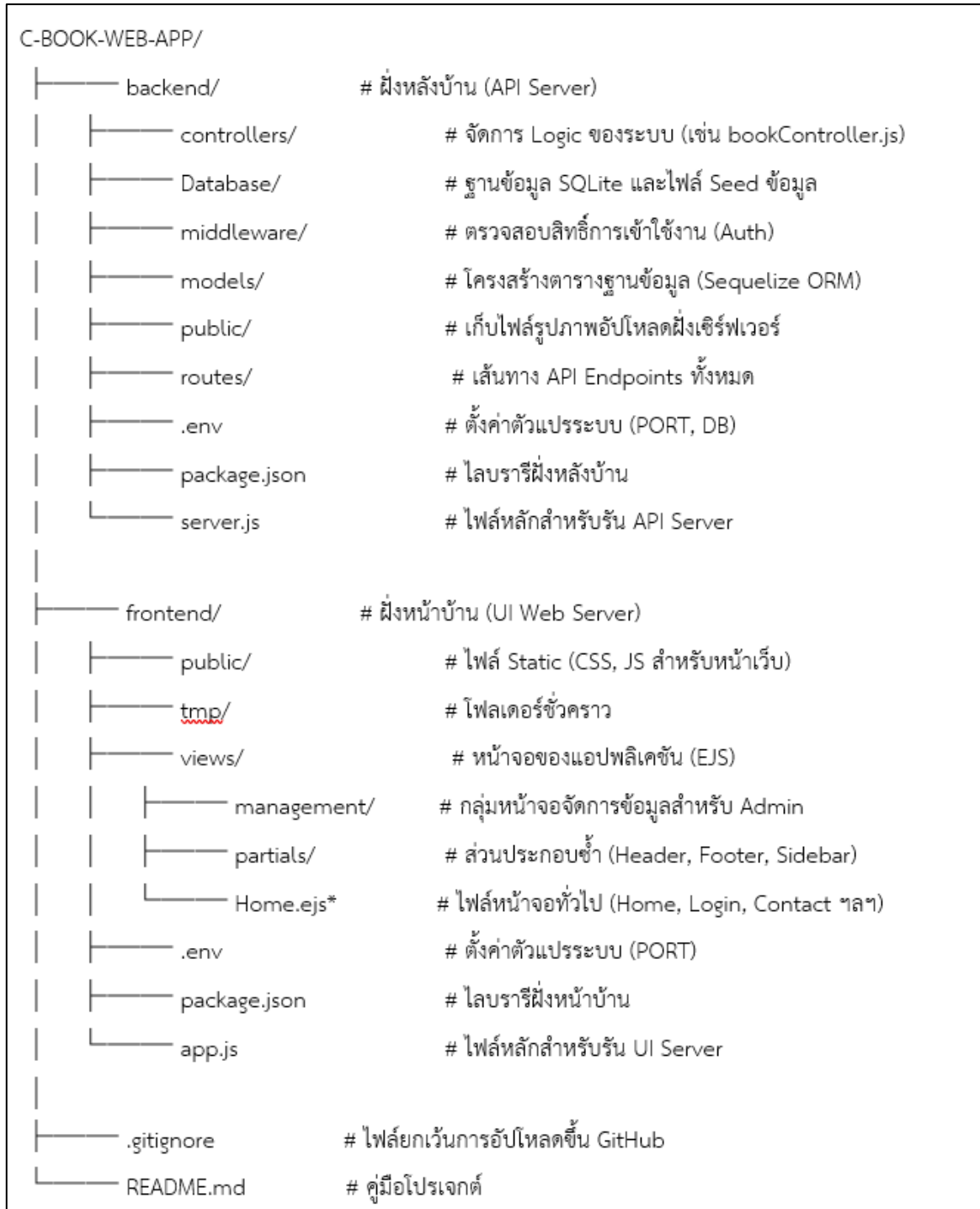


ภาพที่ 3-34 ภาพการเข้าใช้งานเว็บแอปพลิเคชัน

บทที่ 4

การสร้างระบบและการเขียนโปรแกรม (Code)

4.1 โครงสร้างโปรเจกต์ MVC Pattern



ภาพที่ 4-1 โครงสร้างโปรเจกต์

4.2 โครงสร้างตารางฐานข้อมูลโดยใช้ไลบรารี Sequelize (ORM)

4.2.1 โครงสร้างตารางข้อมูล (Models Definition)

มีการสร้างโมเดล (Models) เพื่อใช้แทนตารางข้อมูลหลักในระบบจำนวน 5 ตาราง ได้แก่

4.2.1.1 ตารางผู้ใช้งาน (Users)

ใช้สำหรับเก็บข้อมูลบัญชีผู้ใช้งานและผู้ดูแลระบบ ประกอบด้วยฟิลด์ที่สำคัญ ได้แก่ รหัสผู้ใช้ (PK), ชื่อ, อีเมล, รหัสผ่าน, สิทธิการใช้งาน (Admin/Member) และสถานะบัญชี ในโมเดลนี้มีการใช้ Hooks ของ Sequelize ร่วมกับไลบรารี bcryptjs เพื่อทำการเข้ารหัสผ่าน

```
backend > models > .js users.js > <unknown> > exports > users > constructor > password
1  const bcrypt = require('bcryptjs');
2
3  module.exports = (sequelize, DataTypes) => {
4    const users = sequelize.define('users', {
5      user_id: {
6        type: DataTypes.INTEGER,
7        primaryKey: true,
8        autoIncrement: true
9      },
10     user_name: {
11       type: DataTypes.STRING,
12       allowNull: false
13     },
14     user_email: {
15       type: DataTypes.STRING,
16       allowNull: false,
17       unique: true,
18       validate: {
19         isEmail: true
20       }
21     },
22     password: {
23       type: DataTypes.STRING,
24       allowNull: false
25     },
26     user_role: {
27       type: DataTypes.ENUM('admin', 'member'),
28       defaultValue: 'member',
29       allowNull: false
30     },
31     user_status: {
32       type: DataTypes.STRING,
33       defaultValue: 'active',
34       allowNull: false
35     }
36   },
37   {
38     // ระบบเข้ารหัสผ่านอัตโนมัติ (Hooks)
39     hooks: {
40       beforeCreate: async (user) => {
41         if (user.password) {
42           const salt = await bcrypt.genSalt(10);
43           user.password = await bcrypt.hash(user.password, salt);
44         }
45       },
46       beforeUpdate: async (user) => {
47         if (user.changed('password')) {
48           const salt = await bcrypt.genSalt(10);
49           user.password = await bcrypt.hash(user.password, salt);
50         }
51       }
52     }
53   });
54
55   // ฟังก์ชันสำหรับเช็กรหัสผ่านตอน Login
56   users.prototype.validatePassword = async function (password) {
57     return await bcrypt.compare(password, this.password);
58   };
59
60   return users;
61 };
```

ภาพที่ 4-2 คำสั่งสร้างโมเดลตารางผู้ใช้งาน

4.2.1.2 ตารางหมวดหมู่หนังสือ (Category)

ใช้สำหรับเก็บข้อมูลหมวดหมู่เพื่อใช้จัดกลุ่มหนังสือ ประกอบด้วยข้อมูล รหัสหมวดหมู่ (PK) และชื่อหมวดหมู่ที่ห้ามซ้ำกัน (Unique)

```
backend > models > JS category.js > ...
1
2 module.exports = (sequelize, Datatypes) => {
3   const category = sequelize.define('category', {
4     category_id: {
5       type: Datatypes.INTEGER,
6       primaryKey: true,
7       autoIncrement: true
8     },
9     category_name: {
10      type: Datatypes.STRING,
11      allowNull: false,
12      unique: true
13    }
14  });
15  return category;
16 }
```

ภาพที่ 4-3 คำสั่งสร้างโมเดลตารางหมวดหมู่หนังสือ

4.2.1.3 ตารางข้อมูลหนังสือ (Books)

ใช้สำหรับเก็บรายละเอียดของหนังสือที่วางจำหน่าย ประกอบด้วย รหัสหนังสือ (PK), ชื่อหนังสือ, ชื่อผู้แต่ง, ราคา, จำนวนสต็อกคงเหลือ, สถานะ, ภาพปกหนังสือ และรหัสหมวดหมู่ (FK)

```
backend > models > JS books.js > ...
1
2 module.exports = (sequelize, Datatypes) => {
3   const books = sequelize.define('books', {
4     book_id: {
5       type: Datatypes.INTEGER,
6       primaryKey: true,
7       autoIncrement: true
8     },
9     title: {
10      type: Datatypes.STRING,
11      allowNull: false
12    },
13     author: {
14      type: Datatypes.STRING,
15      allowNull: false
16    },
17     price: {
18      type: Datatypes.DOUBLE,
19      allowNull: false
20    },
21     stock_quantity: {
22      type: Datatypes.INTEGER,
23      defaultValue: 0
24    },
25     book_status: {
26      type: Datatypes.STRING,
27      defaultValue: 'active'
28    },
29     book_img: {
30      type: Datatypes.STRING,
31      allowNull: true,
32      defaultValue: 'default_book.png'
33    },
34     category_id: {
35      type: Datatypes.INTEGER,
36      allowNull: false
37    }
38  });
39  return books;
40 }
```

ภาพที่ 4-4 คำสั่งสร้างโมเดลตารางหนังสือ

4.2.1.4 ตารางคำสั่งซื้อ (Order)

ใช้สำหรับเก็บข้อมูลส่วนหัวของคำสั่งซื้อแต่ละรายการ ประกอบด้วย รหัสคำสั่งซื้อ (PK), วันที่สั่งซื้อ, สถานะคำสั่งซื้อ (เช่น delived) และรหัสผู้สั่งซื้อ (FK)

```
backend > models > JS order.js > ...
1
2 module.exports = (sequelize, Datatypes) => {
3   const order = sequelize.define('order', {
4     order_id: {
5       type: Datatypes.INTEGER,
6       primaryKey: true,
7       autoIncrement: true
8     },
9     order_date: {
10      type: Datatypes.DATE,
11      defaultValue: Datatypes.NOW
12    },
13    status: {
14      type: Datatypes.STRING,
15      defaultValue: 'pending'
16    },
17    user_id: {
18      type: Datatypes.INTEGER,
19      allowNull: false
20    }
21  });
22  return order;
23 };
24
```

ภาพที่ 4-5 คำสั่งสร้างโมเดลตารางคำสั่งซื้อ

4.2.1.5 ตารางรายละเอียดคำสั่งซื้อ (OrderDetail)

ใช้สำหรับเก็บรายการหนังสือย่อยที่อยู่ในแต่ละคำสั่งซื้อ ประกอบด้วย รหัสรายละเอียด (PK), จำนวนเล่มที่สั่ง, ราคาต่อหน่วย, รหัสหนังสือ (FK) และรหัสคำสั่งซื้อ (FK)

```
backend > models > JS order_detail.js > ...
1
2 module.exports = (sequelize, Datatypes) => {
3   const orderDetail = sequelize.define('orderDetail', {
4     detail_id: {
5       type: Datatypes.INTEGER,
6       primaryKey: true,
7       autoIncrement: true
8     },
9     quantity: {
10      type: Datatypes.INTEGER,
11      allowNull: false
12    },
13    unit_price: {
14      type: Datatypes.DOUBLE,
15      allowNull: false
16    },
17    book_id: {
18      type: Datatypes.INTEGER,
19      allowNull: false
20    },
21    order_id: {
22      type: Datatypes.INTEGER,
23      allowNull: false
24    }
25  });
26  return orderDetail;
27 };
28
```

ภาพที่ 4-6 คำสั่งสร้างโมเดลตารางรายละเอียดคำสั่งซื้อ

4.2.2 การกำหนดความสัมพันธ์ระหว่างตาราง (Associations)

เพื่อให้ฐานข้อมูลสามารถเชื่อมโยงข้อมูลถึงกันได้ตามหลักการฐานข้อมูลเชิงสัมพันธ์ ระบบได้กำหนดความสัมพันธ์ผ่านคำสั่ง `hasMany` (มีได้หลายรายการ) และ `belongsTo` (เป็นของ) ดังนี้

4.2.2.1 การตั้งค่าการเชื่อมต่อและนำเข้าโครงสร้างฐานข้อมูล

ส่วนสำคัญในการเริ่มต้นระบบฐานข้อมูลฝั่งหลังบ้าน (Backend) โดยใช้ไลบรารี Sequelize ซึ่งสามารถแบ่งการทำงานออกเป็น 3 ส่วนหลัก คือ การเรียกใช้งานไลบรารี การตั้งค่าการเชื่อมต่อฐานข้อมูล การนำเข้าโมเดลตาราง

```
backend > models > JS index.js > ...
1  const { Sequelize, DataTypes } = require('sequelize');
2
3  // Setup Connection
4  const sequelize = new Sequelize('database', 'username', 'password', {
5    host: 'localhost',
6    dialect: 'sqlite',
7    storage: './Database/DB_Books.sqlite'
8  });
9
10 // Import Models
11 const users = require('./users')(sequelize, DataTypes);
12 const category = require('./category')(sequelize, DataTypes);
13 const books = require('./books')(sequelize, DataTypes);
14 const order = require('./order')(sequelize, DataTypes);
15 const orderDetail = require('./order_detail')(sequelize, DataTypes);
```

ภาพที่ 4-7 การตั้งค่าการเชื่อมต่อและนำเข้าโครงสร้างฐานข้อมูล

4.2.2.2 ความสัมพันธ์ระหว่าง Category และ Books (One-to-Many)

หมวดหมู่หนังสือ 1 หมวดหมู่ สามารถมีหนังสือได้หลายเล่ม และหนังสือแต่ละเล่มจะถูกจัดอยู่ในหมวดหมู่ใดหมวดหมู่หนึ่งเท่านั้น

```
19 // Books <-> Category (One-to-Many)
20 category.hasMany(books, { foreignKey: 'category_id' });
21 books.belongsTo(category, { foreignKey: 'category_id' });
```

ภาพที่ 4-8 ความสัมพันธ์ระหว่าง Category และ Books

4.2.2.3 ความสัมพันธ์ระหว่าง Users และ Order (One-to-Many)

ผู้ใช้งาน 1 คน สามารถทำรายการสั่งซื้อได้หลายรายการ และคำสั่งซื้อแต่ละรายการจะเป็นของลูกค้านั้นๆ รายใดรายหนึ่งเท่านั้น

```
23 // Users <-> Order (One-to-Many)
24 users.hasMany(order, { foreignKey: 'user_id' });
25 order.belongsTo(users, { foreignKey: 'user_id' });
```

ภาพที่ 4-9 ความสัมพันธ์ระหว่าง Users และ Order

4.2.2.4 ความสัมพันธ์ระหว่าง Order และ OrderDetail (One-to-Many)

คำสั่งซื้อ 1 รายการ สามารถมีรายละเอียดหนังสือย่อยได้หลายรายการ

```
27 // Order <-> OrderDetail (One-to-Many)
28 order.hasMany(orderDetail, { foreignKey: 'order_id' });
29 orderDetail.belongsTo(order, { foreignKey: 'order_id' });
30
```

ภาพที่ 4-10 ความสัมพันธ์ระหว่าง Order และ OrderDetail

4.2.2.5 ความสัมพันธ์ระหว่าง Books และ OrderDetail (One-to-Many)

หนังสือ 1 เล่ม สามารถไปปรากฏอยู่ในรายละเอียดคำสั่งซื้อได้หลายรายการ

```
31 // Books <-> OrderDetail (One-to-Many)
32 books.hasMany(orderDetail, { foreignKey: 'book_id' });
33 orderDetail.belongsTo(books, { foreignKey: 'book_id' });
34
```

ภาพที่ 4-11 ความสัมพันธ์ระหว่าง Books และ OrderDetail

4.2.2.6 การส่งออกโมเดลเพื่อนำไปใช้งาน

ใช้คำสั่งสำหรับการส่งออกโมเดล เพื่อให้การทำงานในโปรเจกต์สามารถเรียกใช้งานได้

```
36 // Export Models
37 module.exports = {
38   sequelize, users, category, books, order, orderDetail
39 }
```

ภาพที่ 4-12 การส่งออกโมเดลเพื่อนำไปใช้งาน

4.3 ส่วนฝั่งหลังบ้าน (Backend - API Server)

4.3.1 Middleware สำหรับระบบรักษาความปลอดภัยของ API

ทำหน้าที่คัดกรองและตรวจสอบสิทธิ์ผู้ใช้งานก่อนอนุญาตให้เข้าถึงฐานข้อมูล โดยแบ่งการทำงานเป็น 3 ส่วนหลัก คือ การตรวจสอบแหล่งที่มา การตรวจสอบสถานะเข้าสู่ระบบ การตรวจสอบสิทธิ์ระดับผู้ดูแลระบบเท่านั้น

```
backend > middleware > JS authMiddleware.js > ...
1  const INTERNAL_API_KEY = process.env.INTERNAL_API_KEY || 'cbook_internal_key';
2
3  function hasInternalKey(req) {
4    return req.headers['x-internal-api-key'] === INTERNAL_API_KEY;
5  }
6
7  function getUserContext(req) {
8    return {
9      userId: req.headers['x-user-id'] || null,
10     role: req.headers['x-user-role'] || null
11   };
12 }
13
14 exports.isLoggedInIn = (req, res, next) => {
15   if (!hasInternalKey(req)) {
16     return res.status(401).json({ success: false, message: 'Unauthorized client' });
17   }
18
19   const { userId } = getUserContext(req);
20   if (!userId) {
21     return res.status(401).json({ success: false, message: 'Authentication required' });
22   }
23
24   req.authUser = getUserContext(req);
25   return next();
26 };
27
28 exports.isAdmin = (req, res, next) => {
29   if (!hasInternalKey(req)) {
30     return res.status(401).json({ success: false, message: 'Unauthorized client' });
31   }
32
33   const { userId, role } = getUserContext(req);
34   if (!userId || role !== 'admin') {
35     return res.status(403).json({ success: false, message: 'Admin access required' });
36   }
37
38   req.authUser = getUserContext(req);
39   return next();
40 };
```

ภาพที่ 4-13 ภาพไฟล์ Middleware

4.3.2 ไฟล์หลักที่ใช้เริ่มต้นการทำงานของ Backend API

ไฟล์ server.js ทำหน้าที่ตั้งค่า Middleware จัดการสิทธิ์ CORS รวบรวมเส้นทาง API ทั้งหมดของระบบ และเชื่อมต่อกับข้อมูลด้วย Sequelize ก่อนเปิดรับเซิร์ฟเวอร์การทำงานหลักได้ 3 ส่วนดังนี้

4.3.2.1 โหลดตัวแปรระบบจากไฟล์ .env ตั้งค่าไลบรารี cors เพื่ออนุญาตให้ฝั่งหน้าบ้าน (Frontend ที่รันบนพอร์ต 5000) สามารถเชื่อมต่อเข้ามาดึงข้อมูลได้อย่างปลอดภัย เซิร์ฟเวอร์สามารถอ่านข้อมูลรูปแบบ JSON และเปิดสิทธิ์ให้โฟลเดอร์ public/images สามารถถูกเข้าถึงผ่าน URL ได้โดยตรง

4.3.2.2 การจัดการเส้นทาง นำเข้าเส้นทาง (Routes) ทั้งหมดมาใช้งาน เพื่อให้โครงสร้าง URL เป็นระเบียบ มีการสร้างเส้นทาง /api/health ไว้สำหรับตรวจสอบสถานะการทำงาน

4.3.2.3 ระบบจะเรียกใช้คำสั่ง sequelize.sync() เพื่อเชื่อมต่อและเตรียมความพร้อมของตารางในฐานข้อมูล จากนั้นเซิร์ฟเวอร์จึงจะเริ่มต้นทำงาน (Listen) เพื่อรอรับคำขอจากผู้ใช้งานที่พอร์ต 3000

```
backend > JS server.js > ...
1  require('dotenv').config();
2
3  const express = require('express');
4  const path = require('path');
5  const cors = require('cors');
6  const { sequelize } = require('./models');
7
8  const bookRoutes = require('./routes/bookRoutes');
9  const categoryRoutes = require('./routes/categoryRoutes');
10 const userRoutes = require('./routes/userRoutes');
11 const cartRoutes = require('./routes/cartRoutes');
12 const orderRoutes = require('./routes/orderRoutes');
13 const reportRoutes = require('./routes/reportRoutes');
14
15 const app = express();
16
17 app.use(cors({ origin: process.env.FRONTEND_ORIGIN || 'http://localhost:5000' }));
18 app.use(express.json());
19 app.use(express.urlencoded({ extended: false }));
20 app.use('/images', express.static(path.join(__dirname, 'public/images')));
21
22 app.use('/api/books', bookRoutes);
23 app.use('/api/categories', categoryRoutes);
24 app.use('/api/users', userRoutes);
25 app.use('/api/cart', cartRoutes);
26 app.use('/api/orders', orderRoutes);
27 app.use('/api/reports', reportRoutes);
28
29 app.get('/api/health', (req, res) => {
30   res.json({ success: true, service: 'backend', date: new Date().toISOString() });
31 });
32
33 const port = Number(process.env.PORT || 3000);
34
35 sequelize.sync().then(() => {
36   app.listen(port, () => {
37     console.log(`C-BOOK backend API running at http://localhost:${port}`);
38   });
39 });
40
```

ภาพที่ 4-14 คำสั่งไฟล์ server.js สำหรับรันเซิร์ฟเวอร์ระบบหลังบ้าน

4.3.3 กำหนดเส้นทาง (Router)

4.3.3.1 เส้นทาง API สำหรับจัดการข้อมูลหนังสือ

ตารางที่ 4-1 เส้นทาง API สำหรับจัดการข้อมูลหนังสือ

การทำงาน	HTTP Method	Route Pattern	การประมวลผล (Controller)
แสดงหนังสือทั้งหมด	GET	/api/books	เรียกใช้ฟังก์ชัน listBooks
แสดงรายละเอียด 1 เล่ม	GET	/api/books/:id	เรียกใช้ฟังก์ชัน getBookById ตาม ID
เพิ่มข้อมูลหนังสือใหม่	POST	/api/books	เรียกใช้ฟังก์ชัน createBook
อัปเดตข้อมูลหนังสือ	PUT	/api/books/:id	เรียกใช้ฟังก์ชัน updateBook
ลบข้อมูลหนังสือ	DELETE	/api/books/:id	เรียกใช้ฟังก์ชัน deleteBook

ระบบจะมีการบังคับใช้ Middleware auth.isAdmin กับบางเส้นทางเพื่อความปลอดภัย โดยผู้ใช้งานจะต้องมีสิทธิ์ระดับผู้ดูแลระบบเท่านั้นจึงจะสามารถเรียกใช้งานได้ มีการเรียกใช้ไลบรารี multer เพื่อทำหน้าที่รับไฟล์รูปภาพหน้าปกหนังสือที่ฝั่งหน้าบ้าน

```

backend > routes > JS bookRoutes.js > ...
1  const express = require('express');
2  const path = require('path');
3  const multer = require('multer');
4  const controller = require('../controllers/bookController');
5  const auth = require('../middleware/authMiddleware');
6
7  const router = express.Router();
8
9  const storage = multer.diskStorage({
10   destination: (req, file, cb) => cb(null, path.join(__dirname, '../public/images')),
11   filename: (req, file, cb) => cb(null, `${Date.now()}${path.extname(file.originalname)})
12 });
13
14 const upload = multer({ storage });
15
16 router.get('/', controller.listBooks);
17 router.get('/:id', controller.getBookById);
18 router.post('/', auth.isAdmin, upload.single('book_img'), controller.createBook);
19 router.put('/:id', auth.isAdmin, upload.single('book_img'), controller.updateBook);
20 router.delete('/:id', auth.isAdmin, controller.deleteBook);
21
22 module.exports = router;

```

ภาพที่ 4-15 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลหนังสือ

4.3.3.2 เส้นทาง API สำหรับจัดการข้อมูลหมวดหมู่หนังสือ

ตารางที่ 4-2 เส้นทาง API สำหรับจัดการข้อมูลหมวดหมู่หนังสือ

การทำงาน	HTTP Method	Route Pattern	การประมวลผล (Controller)
แสดงหมวดหมู่ทั้งหมด	GET	/api/categories	เรียกใช้ฟังก์ชัน listCategories
แสดงรายละเอียด 1 หมวดหมู่	GET	/api/categories/:id	เรียกใช้ฟังก์ชัน getCategoryById ตาม ID
เพิ่มหมวดหมู่ใหม่	POST	/api/categories	เรียกใช้ฟังก์ชัน createCategory
อัปเดตข้อมูลหมวดหมู่	PUT	/api/categories/:id	เรียกใช้ฟังก์ชัน updateCategory
ลบข้อมูลหมวดหมู่	DELETE	/api/categories/:id	เรียกใช้ฟังก์ชัน deleteCategory

ในการดึงข้อมูลหมวดหมู่มีการให้เข้าถึงได้แบบสาธารณะเพื่อให้ฝั่ง Frontend นำไปแสดงผลบนหน้าเว็บได้ทันที แต่สำหรับการจัดการข้อมูล (POST, PUT, DELETE) จะมีการล็อกความปลอดภัยด้วย Middleware `auth.isAdmin` เพื่อป้องกันไม่ให้ผู้ใช้ทั่วไปเข้ามาแก้ไขข้อมูลหมวดหมู่ของระบบได้

```
backend > routes > JS categoryRoutes.js > ...
1  const express = require('express');
2  const controller = require('../controllers/categoryController');
3  const auth = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  router.get('/', controller.listCategories);
8  router.get('/:id', controller.getCategoryById);
9  router.post('/', auth.isAdmin, controller.createCategory);
10 router.put('/:id', auth.isAdmin, controller.updateCategory);
11 router.delete('/:id', auth.isAdmin, controller.deleteCategory);
12
13 module.exports = router;
14
```

ภาพที่ 4-16 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลหนังสือ

4.3.3.3 เส้นทาง API สำหรับจัดการข้อมูลผู้ใช้งาน

ตารางที่ 4-3 เส้นทาง API สำหรับจัดการข้อมูลสมาชิก

การทำงาน	HTTP Method	Route Pattern	การประมวลผล (Controller)
เข้าสู่ระบบ (Login)	POST	/api/users/login	เรียกใช้ฟังก์ชัน login
สมัครสมาชิก (Register)	POST	/api/users/register	เรียกใช้ฟังก์ชัน register
แสดงผู้ใช้งานทั้งหมด	GET	/api/users	เรียกใช้ฟังก์ชัน listUsers
เปลี่ยนสิทธิ์ผู้ใช้ (Role)	PUT	/api/users/:id/role	เรียกใช้ฟังก์ชัน changeUserRole
ลบข้อมูลผู้ใช้งาน	DELETE	/api/users/:id	เรียกใช้ฟังก์ชัน deleteUser

สำหรับการเข้าสู่ระบบและสมัครสมาชิก จะเปิดให้บุคคลทั่วไปสามารถเข้าถึงได้เพื่อใช้งานเว็บไซต์ แต่สำหรับเส้นทางที่ใช้จัดการข้อมูลสมาชิกในระบบ (GET, PUT, DELETE) ระบบจะมีการบังคับใช้ Middleware `auth.isAdmin` เพื่อความปลอดภัย

```
backend > routes > JS userRoutes.js > ...
1  const express = require('express');
2  const controller = require('../controllers/userController');
3  const auth = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  router.post('/login', controller.login);
8  router.post('/register', controller.register);
9  router.get('/', auth.isAdmin, controller.listUsers);
10 router.put('/:id/role', auth.isAdmin, controller.changeUserRole);
11 router.delete('/:id', auth.isAdmin, controller.deleteUser);
12
13 module.exports = router;
14
```

ภาพที่ 4-17 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลสมาชิก

4.3.3.4 เส้นทาง API สำหรับจัดการข้อมูลคำสั่งซื้อ

ตารางที่ 4-4 เส้นทาง API สำหรับจัดการข้อมูลคำสั่งซื้อ

การทำงาน	HTTP Method	Route Pattern	การประมวลผล (Controller)
แสดงคำสั่งซื้อทั้งหมด	GET	/api/orders	เรียกใช้ฟังก์ชัน listOrders
แสดงรายละเอียดคำสั่งซื้อ	GET	/api/orders/:id	เรียกใช้ฟังก์ชัน getOrderById
อัปเดตสถานะคำสั่งซื้อ	PUT	/api/orders/:id/status	เรียกใช้ฟังก์ชัน updateOrderStatus
สร้างคำสั่งซื้อ (Checkout)	POST	/api/orders/checkout/process	เรียกใช้ฟังก์ชัน checkout

ระบบมีการบังคับใช้ด้านตรวจสอบความปลอดภัยดังนี้ เส้นทางสำหรับผู้ดูแลระบบ การดูรายการคำสั่งซื้อทั้งหมด, ดูรายละเอียด, หรือเปลี่ยนสถานะการจัดส่ง และเส้นทางสำหรับลูกค้า ในส่วนของ การยืนยันคำสั่งซื้อ/ชำระเงิน เพื่อตรวจสอบได้ว่าผู้ใช้ได้เข้าสู่ระบบเป็นสมาชิกแล้วก็เพียงพอสำหรับการสั่งซื้อหนังสือ

```

backend > routes > JS orderRoutes.js > ...
1  const express = require('express');
2  const controller = require('../controllers/orderController');
3  const auth = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  router.get('/', auth.isAdmin, controller.listOrders);
8  router.get('/:id', auth.isAdmin, controller.getOrderById);
9  router.put('/:id/status', auth.isAdmin, controller.updateOrderStatus);
10 router.post('/checkout/process', auth.isLoggedIn, controller.checkout);
11
12 module.exports = router;
13

```

ภาพที่ 4-18 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลคำสั่งซื้อ

4.3.3.5 เส้นทาง API สำหรับจัดการข้อมูลตะกร้าสินค้า

ตารางที่ 4-5 เส้นทาง API สำหรับจัดการข้อมูลตะกร้าสินค้า

การทำงาน	HTTP Method	Route Pattern	การประมวลผล (Controller)
ดูสรุปข้อมูลตะกร้าสินค้า	POST	/api/cart/summary	เรียกใช้ฟังก์ชัน getSummary
เพิ่มสินค้าลงตะกร้า	POST	/api/cart/add	เรียกใช้ฟังก์ชัน addToCart
อัปเดตจำนวนสินค้า	POST	/api/cart/update	เรียกใช้ฟังก์ชัน updateCart

เส้นทางจัดการตะกร้าสินค้า ระบบจะมีการบังคับใช้ Middleware auth.isLoggedIn ทั้งหมด เพื่อตรวจสอบและยืนยันตัวตนว่า ผู้ใช้จะต้องทำการเข้าสู่ระบบเป็นสมาชิกแล้วเท่านั้น จึงจะสามารถจัดการกับตะกร้าสินค้าของตนเองได้

```

backend > routes > JS cartRoutes.js > ...
1  const express = require('express');
2  const controller = require('../controllers/cartController');
3  const auth = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  router.post('/summary', auth.isLoggedIn, controller.getSummary);
8  router.post('/add', auth.isLoggedIn, controller.addToCart);
9  router.post('/update', auth.isLoggedIn, controller.updateCart);
10
11  module.exports = router;
12

```

ภาพที่ 4-19 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลตะกร้าสินค้า

4.3.3.6 เส้นทาง API สำหรับจัดการข้อมูลรายงาน

ตารางที่ 4-6 เส้นทาง API สำหรับจัดการข้อมูลรายงาน

การทำงาน	HTTP Method	Route Pattern	การประมวลผล (Controller)
แสดงข้อมูลแดชบอร์ดหลัก	GET	/api/reports/dashboard	เรียกใช้ฟังก์ชัน dashboard
แสดงยอดขายตามหมวดหมู่	GET	/api/reports/sales_category	เรียกใช้ฟังก์ชัน salesByCategory
แสดงรายละเอียดแต่ละ หมวดหมู่	GET	/api/reports/sales-by- category/:id	เรียกใช้ฟังก์ชัน salesByCategoryDetail
แสดงสถานะสินค้าคงคลัง	GET	/api/reports/inventory- status	เรียกใช้ฟังก์ชัน inventoryStatus

ข้อมูลเส้นทางหน้ารายงานของระบบระบบมีการบังคับใช้ Middleware auth.isAdmin ในทุกๆ เส้นทางอย่าง โดยอนุญาตให้เฉพาะผู้ใช้ที่มีสิทธิ์ระดับผู้ดูแลระบบเท่านั้นที่สามารถเรียกดูข้อมูลเหล่านี้ได้ ลูกค้านั้นจะไม่สามารถเข้าถึงข้อมูลส่วนนี้ได้อย่างเด็ดขาด

```

backend > routes > JS reportRoutes.js > ...
1  const express = require('express');
2  const controller = require('../controllers/reportController');
3  const auth = require('../middleware/authMiddleware');
4
5  const router = express.Router();
6
7  router.get('/dashboard', auth.isAdmin, controller.dashboard);
8  router.get('/sales_category', auth.isAdmin, controller.salesByCategory);
9  router.get('/sales-by-category/:id', auth.isAdmin, controller.salesByCategoryDetail);
10  router.get('/inventory-status', auth.isAdmin, controller.inventoryStatus);
11
12  module.exports = router;
13

```

ภาพที่ 4-20 ภาพไฟล์เส้นทาง API สำหรับจัดการข้อมูลรายงาน

4.3.4 การประมวลผลฝั่งเซิร์ฟเวอร์ (Controller)

4.3.4.1 ส่วนจัดการข้อมูลหนังสือ

ไฟล์ bookController.js ทำหน้าที่เป็น Controller โดยทำงานเชื่อมต่อกับฐานข้อมูลผ่าน Sequelize ORM และจัดการไฟล์ภาพบนเซิร์ฟเวอร์ มีรายละเอียดการทำงานแต่ละส่วนดังนี้

- ส่วนตั้งค่าและการจัดการไฟล์ภาพ

มีการเรียกใช้ File System และ path เพื่อจัดการกับไฟล์ในเครื่องเซิร์ฟเวอร์ รวมถึงเรียกใช้ Op จาก Sequelize สำหรับสร้างเงื่อนไขการค้นหาแบบยืดหยุ่น มีฟังก์ชันลบรูปภาพหน้าปกหนังสือเก่าออกจากโฟลเดอร์ public/images เมื่อมีการอัปเดตภาพใหม่หรือลบหนังสือทิ้ง เพื่อเป็นการประหยัดพื้นที่เซิร์ฟเวอร์

```
nd > controllers > JS bookController.js > listBooks > listBooks
const fs = require('fs');
const path = require('path');
const { Op } = require('sequelize');
const db = require('../models');

const defaultImages = new Set(['default_book.png']);

function removeOldImage(imageName) {
  if (!imageName || defaultImages.has(imageName)) return;
  const filePath = path.join(__dirname, '../public/images', imageName);
  if (fs.existsSync(filePath)) {
    fs.unlinkSync(filePath);
  }
}
```

ภาพที่ 4-21 ภาพส่วนตั้งค่าและการจัดการไฟล์ภาพ

- ฟังก์ชัน getBookById การแสดงรายละเอียดหนังสือ 1 เล่ม

ฟังก์ชันนี้ทำหน้าที่ดึงข้อมูลหนังสือแบบเจาะจงโดยใช้คำสั่ง findByPk (Find by Primary Key) ตาม ID ที่ส่งมาใน URL หากไม่พบข้อมูลในฐานข้อมูล ระบบจะดักจับข้อผิดพลาดและส่งรหัสสถานะ 404 Not Found กลับไปให้ผู้ใช้งาน

```
exports.getBookById = async (req, res) => {
  try {
    const book = await db.books.findByPk(req.params.id);
    if (!book) {
      return res.status(404).json({ success: false, message: 'Book not found' });
    }

    return res.json({ success: true, book });
  } catch (error) {
    console.error('getBookById error:', error);
    return res.status(500).json({ success: false, message: 'Failed to get book' });
  }
};
```

ภาพที่ 4-22 ภาพไฟล์ฟังก์ชัน getBookById

- ฟังก์ชัน listBooks การแสดงผลและค้นหาหนังสือทั้งหมด

ฟังก์ชันนี้ใช้สำหรับดึงข้อมูลหนังสือทั้งหมดเพื่อไปแสดงผลที่หน้าบ้าน โดยมีจุดเด่นคือ มีการแบ่งข้อมูลออกเป็นหน้าๆ มีระบบรองรับการค้นหาจากชื่อหนังสือ การกรองตามหมวดหมู่ และการกรองตามสถานะใช้คำสั่ง findAndCountAll ซึ่งจะดึงทั้งข้อมูลหนังสือ และจำนวนรวมทั้งหมดออกมา พร้อมทั้งเชื่อมตาราง ดึงชื่อหมวดหมู่ มาแสดงผลด้วย

```
exports.listBooks = async (req, res) => {
  try {
    const limit = Number(req.query.limit || 12);
    const page = Number(req.query.page || 1);
    const offset = (page - 1) * limit;

    const where = {};

    if (req.query.search) {
      where.title = { [Op.like]: `%${req.query.search}%` };
    }

    if (req.query.category && req.query.category !== 'all') {
      where.category_id = req.query.category;
    }

    if (req.query.status) {
      where.book_status = req.query.status;
    }

    if (req.query.onlyAvailable === '1') {
      where.book_status = 'Available';
    }

    const { count, rows } = await db.books.findAndCountAll({
      where,
      include: [{ model: db.category, attributes: ['category_id', 'category_name'] }],
      limit,
      offset,
      order: [['book_id', 'DESC']]
    });

    res.json({
      success: true,
      books: rows,
      totalItems: count,
      totalPages: Math.max(1, Math.ceil(count / limit)),
      currentPage: page,
      limit
    });
  } catch (error) {
    console.error('listBooks error:', error);
    res.status(500).json({ success: false, message: 'Failed to list books' });
  }
};
```

ภาพที่ 4-23 ภาพไฟล์ฟังก์ชัน listBooks

- ฟังก์ชัน createBook การเพิ่มหนังสือใหม่

ฟังก์ชันนี้มีการตรวจสอบชื่อหนังสือป้องกันข้อมูลซ้ำ หากมีจะปฏิเสธการสร้างใหม่ (400 Bad Request) นำจำนวนสต็อกมาประเมิน หากค่ามากกว่า 0 จะตั้งสถานะเป็น 'Available' แต่ถ้าเป็น 0 จะตั้งเป็น 'Out of Stock' บันทึกชื่อไฟล์ภาพที่รับมาจาก Multer หากแอตมินไม่ได้อัปโหลดรูปมา ระบบจะใช้ภาพ default_book.png


```

exports.createBook = async (req, res) => {
  try {
    const { title, author, price, stock_quantity, category_id } = req.body;

    const existing = await db.books.findOne({ where: { title } });
    if (existing) {
      return res.status(400).json({ success: false, message: 'Book title already exists' });
    }

    const stock = Number(stock_quantity || 0);
    const calculatedStatus = stock > 0 ? 'Available' : 'Out of Stock';

    const created = await db.books.create({
      title,
      author,
      price,
      stock_quantity: stock,
      book_status: calculatedStatus,
      category_id,
      book_img: req.file ? req.file.filename : 'default_book.png'
    });

    return res.status(201).json({ success: true, book: created });
  } catch (error) {
    console.error('createBook error:', error);
    return res.status(500).json({ success: false, message: 'Failed to create book' });
  }
};

```

ภาพที่ 4-24 ภาพไฟล์ฟังก์ชัน createBook

- ฟังก์ชัน updateBook การแก้ไขข้อมูลหนังสือ

มีตรวจสอบว่ามีหนังสือเล่มนี้อยู่จริงหรือไม่ก่อนทำการแก้ไข คำนวณสถานะสต็อกแบบอัตโนมัติเช่นเดียวกับตอนสร้างใหม่ หากมีการอัปโหลดรูปภาพใหม่เข้ามา ระบบจะนำชื่อไฟล์ใหม่ไปอัปเดตในฐานข้อมูล และเรียกใช้ฟังก์ชัน removeOldImage เพื่อลบไฟล์รูปภาพเก่าทิ้งทันที ป้องกันปัญหาไฟล์ขยะค้างในเซิร์ฟเวอร์

```

exports.updateBook = async (req, res) => {
  try {
    const current = await db.books.findByPk(req.params.id);
    if (!current) {
      return res.status(404).json({ success: false, message: 'Book not found' });
    }

    const { title, author, price, stock_quantity, category_id } = req.body;
    const stock = Number(stock_quantity || 0);
    const calculatedStatus = stock > 0 ? 'Available' : 'Out of Stock';

    const updateData = {
      title,
      author,
      price,
      stock_quantity: stock,
      book_status: calculatedStatus,
      category_id
    };

    if (req.file) {
      updateData.book_img = req.file.filename;
      removeOldImage(current.book_img);
    }

    await db.books.update(updateData, { where: { book_id: req.params.id } });
    const updated = await db.books.findByPk(req.params.id);

    return res.json({ success: true, book: updated });
  } catch (error) {
    console.error('updateBook error:', error);
    return res.status(500).json({ success: false, message: 'Failed to update book' });
  }
};

```

ภาพที่ 4-25 ภาพไฟล์ฟังก์ชัน updateBook

- ฟังก์ชัน deleteBook การลบหนังสือ

มีตรวจสอบและค้นหาหนังสือที่ต้องการลบ เรียกใช้ฟังก์ชัน removeOldImage เพื่อลบไฟล์รูปภาพของหนังสือเล่มนั้นออกจากเครื่องเซิร์ฟเวอร์อย่างถาวร ใช้คำสั่ง destroy ลบ Record ข้อมูลออกจากฐานข้อมูล แล้วจึงส่งสถานะความสำเร็จกลับไป

```
exports.deleteBook = async (req, res) => {
  try {
    const target = await db.books.findByPk(req.params.id);
    if (!target) {
      return res.status(404).json({ success: false, message: 'Book not found' });
    }

    removeOldImage(target.book_img);
    await db.books.destroy({ where: { book_id: req.params.id } });

    return res.json({ success: true, message: 'Book deleted successfully' });
  } catch (error) {
    console.error('deleteBook error:', error);
    return res.status(500).json({ success: false, message: 'Failed to delete book' });
  }
};
```

ภาพที่ 4-26 ภาพไฟล์ฟังก์ชัน deleteBook

4.3.4.2 ส่วนจัดการข้อมูลหมวดหมู่หนังสือ

ไฟล์ categoryController.js ทำหน้าที่เป็น Controller กับหมวดหมู่หนังสือทั้งหมด โดยทำงานเชื่อมต่อกับโมเดลฐานข้อมูลมีรายละเอียดการทำงานในแต่ละฟังก์ชันดังนี้

- ฟังก์ชัน createCategory การเพิ่มหมวดหมู่ใหม่

ฟังก์ชันสำหรับสร้างหมวดหมู่ใหม่ มีการใช้คำสั่ง .trim() เพื่อตัดช่องว่างหน้า-หลังของชื่อหมวดหมู่ที่ผู้ใช้อาจเผลอพิมพ์เข้ามา ป้องกันข้อมูลซ้ำซ้อนเพื่อตรวจสอบว่าชื่อหมวดหมู่นี้มีอยู่แล้วหรือไม่ หากมีซ้ำ ระบบจะปฏิเสธการสร้างและส่งรหัส 400 Bad Request กลับไป เพื่อป้องกันไม่ให้เกิดหมวดหมู่ชื่อซ้ำกันในระบบ

```
exports.createCategory = async (req, res) => {
  try {
    const category_name = (req.body.category_name || '').trim();

    const existing = await db.category.findOne({
      where: { category_name: { [Op.like]: category_name } }
    });

    if (existing) {
      return res.status(400).json({ success: false, message: 'Category name already exists' });
    }

    const created = await db.category.create({ category_name });
    return res.status(201).json({ success: true, category: created });
  } catch (error) {
    console.error('createCategory error:', error);
    return res.status(500).json({ success: false, message: 'Failed to create category' });
  }
};
```

ภาพที่ 4-27 ฟังก์ชัน createCategory

- ฟังก์ชัน listCategories การแสดงผลและค้นหาหมวดหมู่

ฟังก์ชันนี้ทำหน้าที่ดึงรายการหมวดหมู่ทั้งหมดไปแสดงผล โดยมีกระบวนการดังนี้ ระบบมีการแบ่งหน้า และสามารถค้นหาหมวดหมู่จากชื่อได้โดยใช้ตัวดำเนินการ %LIKE% ของ Sequelize มีการคำนวณจำนวนหนังสือต่อหมวดหมู่ โดยระบบจะใช้ Promise.all ร่วมกับการวนลูป map เพื่อนับจำนวนหนังสือที่อยู่ในแต่ละหมวดหมู่ การส่งข้อมูลสรุปยอดรวม เช่น จำนวนหมวดหมู่ทั้งหมด และจำนวนหนังสือทั้งหมด ในคลั่งกลับไปให้ฝั่งหน้าบ้านใช้แสดงผลบนแดชบอร์ดด้วย

```
const { Op } = require('sequelize');
const db = require('../models');

exports.listCategories = async (req, res) => {
  try {
    const search = (req.query.search || '').trim();
    const limit = Number(req.query.limit || 50);
    const page = Number(req.query.page || 1);
    const offset = (page - 1) * limit;

    const where = search
      ? { category_name: { [Op.like]: `%${search}%` } }
      : {};

    const { count, rows } = await db.category.findAndCountAll({
      where,
      limit,
      offset,
      order: [['category_id', 'DESC']]
    });

    const categories = await Promise.all(
      rows.map(async (cat) => {
        const book_count = await db.books.count({ where: { category_id: cat.category_id } });
        return { ...cat.toJSON(), book_count };
      })
    );

    res.json({
      success: true,
      categories,
      totalItems: count,
      totalPages: Math.max(1, Math.ceil(count / limit)),
      currentPage: page,
      limit,
      stats: {
        totalCategories: count,
        totalBooks: await db.books.count()
      }
    });
  } catch (error) {
    console.error('listCategories error:', error);
    res.status(500).json({ success: false, message: 'Failed to list categories' });
  }
};
```

ภาพที่ 4-28 ฟังก์ชัน listCategories

- ฟังก์ชัน getCategoryById การแสดงรายละเอียด 1 หมวดหมู่

ทำหน้าที่ค้นหาหมวดหมู่ที่ต้องการตามรหัส ID (findByPk) หากค้นหาไม่พบ ระบบจะจัดการข้อผิดพลาดโดยส่งรหัสสถานะ 404 Not Found เพื่อแจ้งเตือนผู้ใช้งาน

```

exports.getCategoryById = async (req, res) => {
  try {
    const category = await db.category.findByPk(req.params.id);
    if (!category) {
      return res.status(404).json({ success: false, message: 'Category not found' });
    }

    return res.json({ success: true, category });
  } catch (error) {
    console.error('getCategoryById error:', error);
    return res.status(500).json({ success: false, message: 'Failed to get category' });
  }
};

```

ภาพที่ 4-29 ฟังก์ชัน getCategoryById

- ฟังก์ชัน updateCategory การแก้ไขข้อมูลหมวดหมู่

ทำหน้าที่อัปเดตชื่อหมวดหมู่เดิม โดยนำระบบตรวจสอบชื่อเข้ามาใช้เช่นเดียวกับการสร้างใหม่ ระบบจะตรวจสอบหาชื่อซ้ำในฐานข้อมูล โดยละเว้น รหัส ID ของหมวดหมู่ที่กำลังแก้ไขอยู่ เพื่อให้ผู้ใช้สามารถกดบันทึกชื่อเดิมของตัวเองได้โดยไม่ติดแจ้งเตือนข้อผิดพลาด

```

exports.updateCategory = async (req, res) => {
  try {
    const categoryId = req.params.id;
    const category_name = (req.body.category_name || '').trim();

    const duplicate = await db.category.findOne({
      where: {
        category_name: { [Op.like]: category_name },
        category_id: { [Op.ne]: categoryId }
      }
    });

    if (duplicate) {
      return res.status(400).json({ success: false, message: 'Category name already exists' });
    }

    await db.category.update({ category_name }, { where: { category_id: categoryId } });
    const category = await db.category.findByPk(categoryId);

    return res.json({ success: true, category });
  } catch (error) {
    console.error('updateCategory error:', error);
    return res.status(500).json({ success: false, message: 'Failed to update category' });
  }
};

```

ภาพที่ 4-30 ฟังก์ชัน updateCategory

- ฟังก์ชัน deleteCategory การลบหมวดหมู่

ฟังก์ชันนี้มีการทำงานในด้านการรักษาความสมบูรณ์ของข้อมูลดังนี้ ป้องกันการเกิดข้อมูลกำพว้า ก่อนที่จะทำการลบหมวดหมู่ ระบบจะใช้โมเดล db.books เพื่อนับว่ายังมีหนังสือเล่มใดผูกติดอยู่กับหมวดหมู่นี้หรือไม่ หากพบว่ามีหนังสืออยู่ในหมวดหมู่นั้น (bookCount > 0) ระบบจะ ปฏิเสธการลบทันที (400 Bad Request) และแจ้งเตือนแอดมินว่าไม่สามารถลบได้ เพื่อป้องกันไม่ให้ข้อมูลหนังสือเกิดข้อผิดพลาดในการแสดงผลหมวดหมู่นั้นเอง

```

exports.deleteCategory = async (req, res) => {
  try {
    const categoryId = req.params.id;
    const bookCount = await db.books.count({ where: { category_id: categoryId } });

    if (bookCount > 0) {
      return res.status(400).json({ success: false, message: 'Cannot delete category with books' });
    }

    await db.category.destroy({ where: { category_id: categoryId } });
    return res.json({ success: true, message: 'Category deleted successfully' });
  } catch (error) {
    console.error('deleteCategory error:', error);
    return res.status(500).json({ success: false, message: 'Failed to delete category' });
  }
};

```

ภาพที่ 4-31 ฟังก์ชัน deleteCategory

4.3.4.3 ส่วนจัดการข้อมูลผู้ใช้งาน

ไฟล์ userController.js นี้ทำหน้าที่เป็น Controller หลักในการจัดการระบบสมาชิก (Authentication) และการบริหารจัดการข้อมูลผู้ใช้งาน (User Management) โดยมีการทำงานดังนี้

- ฟังก์ชัน register การสมัครสมาชิกใหม่

มีการป้องกันอีเมลซ้ำ หากซ้ำจะแจ้งเตือน 400 Bad Request เมื่อสร้างผู้ใช้ใหม่ ระบบจะบังคับตั้งค่าฟิลด์ user_role ให้เป็น 'member' และกำหนดสถานะ user_status เป็น 'active' เสมอ เพื่อป้องกันไม่ให้ผู้ใช้ทั่วไปแอบกำหนดสิทธิ์ตัวเองเป็นแอดมินผ่านช่องโหว่ของ API

```

exports.register = async (req, res) => {
  try {
    const { username, email, password } = req.body;

    const existing = await db.users.findOne({ where: { user_email: email } });
    if (existing) {
      return res.status(400).json({ success: false, message: 'Email already in use' });
    }

    const user = await db.users.create({
      user_name: username,
      user_email: email,
      password,
      user_role: 'member',
      user_status: 'active'
    });

    return res.status(201).json({
      success: true,
      user: {
        user_id: user.user_id,
        user_name: user.user_name,
        user_email: user.user_email,
        user_role: user.user_role
      }
    });
  } catch (error) {
    console.error('register error:', error);
    return res.status(500).json({ success: false, message: 'Registration failed' });
  }
};

```

ภาพที่ 4-32 ฟังก์ชัน register

- ฟังก์ชัน login การเข้าสู่ระบบ

ฟังก์ชันนี้ทำหน้าที่ตรวจสอบสิทธิ์การเข้าใช้งานคือ ตรวจสอบอีเมล ว่ามีอีเมลนี้ในระบบหรือไม่ หากไม่มีจะส่งรหัส 404 Not Found กลับไป ตรวจสอบรหัสผ่าน เรียกใช้เมธอด validPassword เพื่อเปรียบเทียบรหัสผ่านที่ผู้ใช้พิมพ์มากับค่า Hash ในฐานข้อมูล ตรวจสอบสถานะบัญชี หากบัญชีถูกระงับ (ไม่ได้มีค่าเป็น 'active') ระบบจะปฏิเสธการเข้าสู่ระบบ (403 Forbidden) ระบบจะส่งข้อมูลผู้ใช้กลับไปให้หน้าบ้าน (Frontend) โดย คัดกรองฟิลด์รหัสผ่านออกไป เพื่อความปลอดภัยขั้นสูงสุด

```
const db = require('../models');
const { Op } = require('sequelize');

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await db.users.findOne({ where: { user_email: email } });

    if (!user) {
      return res.status(404).json({ success: false, message: 'Email not found' });
    }

    const isValid = await user.validPassword(password);
    if (!isValid) {
      return res.status(401).json({ success: false, message: 'Invalid password' });
    }

    if (user.user_status !== 'active') {
      return res.status(403).json({ success: false, message: 'User is suspended' });
    }

    return res.json({
      success: true,
      user: {
        user_id: user.user_id,
        user_name: user.user_name,
        user_email: user.user_email,
        user_role: user.user_role,
        user_status: user.user_status
      }
    });
  } catch (error) {
    console.error('login error:', error);
    return res.status(500).json({ success: false, message: 'Login failed' });
  }
};
```

ภาพที่ 4-33 ฟังก์ชัน login

- ฟังก์ชัน changeUserRole การเปลี่ยนระดับสิทธิ์ผู้ใช้งาน

ทำหน้าที่อัปเดตฟิลด์ user_role (เช่น การเลื่อนชั้นจาก member เป็น admin) มีการเขียนเงื่อนไขตรวจสอบว่า ID ของคนที่กำลังกดยืนยัน (req.authUser.userId) ตรงกับ ID ของคนที่กำลังถูกเปลี่ยนสิทธิ์ (userId) หรือไม่ หากตรงกัน ระบบจะบล็อกการทำงานทันที เพื่อป้องกันความผิดพลาดที่แอดมินอาจเผลอลดสิทธิ์ของตัวเองจนทำให้ระบบไม่มีผู้ดูแล

```

exports.changeUserRole = async (req, res) => {
  try {
    const userId = req.params.id;
    const { role } = req.body;

    if (String(req.authUser.userId) === String(userId)) {
      return res.status(403).json({ success: false, message: 'Cannot change your own role' });
    }

    await db.users.update({ user_role: role }, { where: { user_id: userId } });
    return res.json({ success: true, message: 'Role updated successfully' });
  } catch (error) {
    console.error('changeUserRole error:', error);
    return res.status(500).json({ success: false, message: 'Failed to update role' });
  }
};

```

ภาพที่ 4-34 ฟังก์ชัน changeUserRole

- ฟังก์ชัน deleteUser การลบบัญชีผู้ใช้งาน

ใช้คำสั่ง destroy ในการลบข้อมูลผู้ใช้ออกจากฐานข้อมูล มีการป้องกัน ป้องกันตัวเอง เช่นเดียวกับฟังก์ชันเปลี่ยนสิทธิ์ คือ แอดมินไม่สามารถกดลบบัญชีของตัวเองได้ เพื่อป้องกันอุบัติเหตุและการเกิดบั๊กในระบบ

```

exports.deleteUser = async (req, res) => {
  try {
    const userId = req.params.id;

    if (String(req.authUser.userId) === String(userId)) {
      return res.status(403).json({ success: false, message: 'Cannot delete yourself' });
    }

    await db.users.destroy({ where: { user_id: userId } });
    return res.json({ success: true, message: 'User deleted successfully' });
  } catch (error) {
    console.error('deleteUser error:', error);
    return res.status(500).json({ success: false, message: 'Failed to delete user' });
  }
};

```

ภาพที่ 4-35 ฟังก์ชัน deleteUser

- ฟังก์ชัน listUsers การแสดงผลและค้นหารายชื่อผู้ใช้งาน

ฟังก์ชันนี้มีระบบการทำงานดังนี้ มีระบบรองรับการค้นหาผู้ใช้งาน "ชื่อ" หรือ "อีเมล" และสามารถกรองพร้อมกันด้วย "ระดับสิทธิ์" (roleFilter) ได้ในเวลาเดียวกัน ระบบมีการแบ่งหน้า ใช้คำสั่ง findAndCountAll เพื่อแบ่งข้อมูลเป็นหน้าๆ (กำหนดค่าเริ่มต้นไว้ที่ 6 รายการต่อหน้า)

ระบบมีการสรุปสถิติ (User Statistics) นอกจากการส่งรายชื่อผู้ใช้แล้ว ระบบยังควรี ข้อมูลเพิ่มเติมนับจำนวนผู้ใช้ทั้งหมด แยกเป็นจำนวน Admin และจำนวน Member เพื่อส่งกลับไปแสดงผลเป็นกราฟหรือตัวเลขสรุปบนหน้าเว็บได้ทันที

```

exports.listUsers = async (req, res) => {
  try {
    const searchQuery = req.query.search || '';
    const roleFilter = req.query.role || 'member';
    const page = Number(req.query.page || 1);
    const limit = Number(req.query.limit || 6);

    let where = {};

    if (roleFilter) {
      where.user_role = roleFilter;
    }

    if (searchQuery) {
      where = {
        [Op.and]: [
          roleFilter ? { user_role: roleFilter } : {},
          {
            [Op.or]: [
              { user_name: { [Op.like]: `%${searchQuery}%` } },
              { user_email: { [Op.like]: `%${searchQuery}%` } }
            ]
          }
        ]
      };
    }

    const { count, rows } = await db.users.findAndCountAll({
      where,
      limit,
      offset: (page - 1) * limit,
      order: [['user_id', 'DESC']]
    });

    return res.json({
      success: true,
      users: rows,
      totalItems: count,
      totalPages: Math.max(1, Math.ceil(count / limit)),
      currentPage: page,
      limit,
      totalUsersCount: await db.users.count(),
      totalAdminCount: await db.users.count({ where: { user_role: 'admin' } }),
      totalMemberCount: await db.users.count({ where: { user_role: 'member' } })
    });
  } catch (error) {
    console.error('listUsers error:', error);
    return res.status(500).json({ success: false, message: 'Failed to list users' });
  }
};

```

ภาพที่ 4-36 ฟังก์ชัน listUsers

4.3.4.4 ส่วนจัดการข้อมูลคำสั่งซื้อ

ไฟล์ orderController.js นี้ทำหน้าที่เป็น Controller สำหรับการจัดการคำสั่งซื้อ และการตัดสต็อกสินค้า โดยทำงานเชื่อมต่อกับตารางหลายตารางพร้อมกัน มีรายละเอียดการทำงานดังนี้

- ฟังก์ชัน listOrders การแสดงรายการคำสั่งซื้อ

ฟังก์ชันนี้ออกแบบมาเพื่อดึงข้อมูลคำสั่งซื้อทั้งหมดไปแสดง โดยมีระบบการทำงานดังนี้ ระบบค้นหาโดยระบบจะตรวจสอบค่าที่รับมาจะค้นหาจากรหัสคำสั่งซื้อ หรือหากเป็นตัวอักษร จะสลับไป

ค้นหาจากชื่อผู้สั่งซื้อ ผ่านการเชื่อมตาราง หลังจากดึงข้อมูลจากฐานข้อมูล ระบบจะใช้คำสั่ง `.reduce()` วนลูป คำนวณหายอดเงินรวม และจำนวนชิ้นของแต่ละคำสั่งซื้อ มีการแบ่งหน้าข้อมูลผ่าน `limit` และ `offset` พร้อมส่งคืนค่าสถิติหน้าทั้งหมด เพื่อให้หน้าบ้านนำไปสร้างปุ่มเปลี่ยนหน้าได้ทันที

```
const db = require('../models');
const { Op } = require('sequelize');

exports.listOrders = async (req, res) => {
  try {
    const page = Number(req.query.page || 1);
    const limit = Number(req.query.limit || 10);
    const offset = (page - 1) * limit;

    const searchQuery = req.query.search || '';
    const statusFilter = req.query.status || '';

    const where = {};
    if (statusFilter) where.status = statusFilter;
    if (searchQuery && !Number.isNaN(Number(searchQuery))) where.order_id = Number(searchQuery);

    const { count, rows } = await db.order.findAndCountAll({
      where,
      include: [
        {
          model: db.users,
          attributes: ['user_name', 'user_email'],
          where: searchQuery && Number.isNaN(Number(searchQuery))
            ? { user_name: { [Op.like]: `%${searchQuery}%` } }
            : undefined
        },
        { model: db.orderDetail, attributes: ['quantity', 'unit_price'] }
      ],
      order: [['order_date', 'DESC']],
      limit,
      offset,
      distinct: true
    });

    const orders = rows.map((o) => {
      const total_amount = o.orderDetails.reduce((sum, item) => sum + item.quantity * item.unit_price, 0);
      const total_items = o.orderDetails.reduce((sum, item) => sum + item.quantity, 0);

      return {
        order_id: o.order_id,
        order_date: o.order_date,
        status: o.status,
        user_name: o.user ? o.user.user_name : 'Unknown User',
        total_amount,
        total_items
      };
    });

    return res.json({
      success: true,
      orders,
      totalItems: count,
      totalPages: Math.max(1, Math.ceil(count / limit)),
      currentPage: page,
      limit
    });
  } catch (error) {
    console.error('listOrders error:', error);
    return res.status(500).json({ success: false, message: 'Failed to list orders' });
  }
};
```

ภาพที่ 4-37 ฟังก์ชัน listOrders

- ฟังก์ชัน getOrderById การดูรายละเอียดใบเสร็จ

ฟังก์ชันนี้ทำหน้าที่ดึงข้อมูลคำสั่งซื้อแบบเจาะจง 1 รายการ จากการดึงข้อมูลจากหลายตาราง เริ่มจากข้อมูลผู้ซื้อ รายละเอียดสินค้าแต่ละรายการ และดึงชื่อและรูปภาพหนังสือ ของสินค้านั้นๆ

```
exports.getOrderById = async (req, res) => {
  try {
    const order = await db.order.findPk(req.params.id, {
      include: [
        { model: db.users, attributes: ['user_name', 'user_email'] },
        { model: db.orderDetail, include: [{ model: db.books, attributes: ['title', 'book_img'] }] }
      ]
    });

    if (!order) {
      return res.status(404).json({ success: false, message: 'Order not found' });
    }

    const itemsTotal = order.orderDetails.reduce((sum, item) => sum + item.quantity * item.unit_price, 0);

    return res.json({
      success: true,
      order,
      itemsTotal,
      grandTotal: itemsTotal
    });
  } catch (error) {
    console.error('getOrderById error:', error);
    return res.status(500).json({ success: false, message: 'Failed to get order' });
  }
};
```

ภาพที่ 4-38 ฟังก์ชัน getOrderById

- ฟังก์ชัน updateOrderStatus การเปลี่ยนสถานะการจัดส่ง

เป็นฟังก์ชันที่ทำหน้าที่รับค่าสถานะใหม่ เช่นจาก pending เป็น deviled มาอัปเดตลงในตาราง order ตามรหัส ID ที่ระบุ

```
exports.updateOrderStatus = async (req, res) => {
  try {
    await db.order.update({ status: req.body.status }, { where: { order_id: req.params.id } });
    return res.json({ success: true, message: 'Status updated successfully' });
  } catch (error) {
    console.error('updateOrderStatus error:', error);
    return res.status(500).json({ success: false, message: 'Failed to update status' });
  }
};
```

ภาพที่ 4-39 ฟังก์ชัน updateOrderStatus

- ฟังก์ชัน checkout กระบวนการยืนยันคำสั่งซื้อและตัดสต็อก

มีขั้นตอนการทำงานประกอบดังนี้ ระบบจะสร้างเซสชันการทำงานแบบรวม หากเกิดข้อผิดพลาดขึ้นในขั้นตอนใดขั้นตอนหนึ่ง ข้อมูลทั้งหมดจะถูกยกเลิก (Rollback) กลับไปจุดเริ่มต้นทันที เพื่อป้องกันปัญหา จะสร้างใบคำสั่งซื้อหลักลงในตาราง order พร้อมประทับเวลาปัจจุบัน ระบบจะวนลูปอ่านสินค้าในตะกร้าทีละชิ้นเพื่อบันทึกข้อมูลลงตารางรายละเอียด ลดจำนวนสต็อกหนังสือ หากพบว่าสินค้าเหลือ 0 จะทำการอัปเดตสถานะหนังสือ เมื่อทุกขั้นตอนผ่าน ระบบจะสั่ง transaction.commit() เพื่อบันทึกข้อมูลทั้งหมดลงฐานข้อมูลอย่างถาวร

```

exports.checkout = async (req, res) => {
  const transaction = await db.sequelize.transaction();

  try {
    const userId = req.authUser.userId;
    const cart = Array.isArray(req.body.cart) ? req.body.cart : [];

    if (!cart.length) {
      await transaction.rollback();
      return res.status(400).json({ success: false, message: 'Cart is empty' });
    }

    const newOrder = await db.order.create(
      { user_id: userId, status: 'delivered', order_date: new Date() },
      { transaction }
    );

    for (const item of cart) {
      await db.orderDetail.create(
        {
          order_id: newOrder.order_id,
          book_id: Number(item.bookId),
          quantity: Number(item.quantity),
          unit_price: Number(item.price)
        },
        { transaction }
      );

      await db.books.decrement('stock_quantity', {
        by: Number(item.quantity),
        where: { book_id: Number(item.bookId) },
        transaction
      });

      const updatedBook = await db.books.findByPk(item.bookId, { transaction });
      if (updatedBook && updatedBook.stock_quantity <= 0) {
        await updatedBook.update({ book_status: 'Out of Stock' }, { transaction });
      }
    }

    await transaction.commit();
    return res.status(201).json({ success: true, orderId: newOrder.order_id });
  } catch (error) {
    await transaction.rollback();
    console.error('checkout error:', error);
    return res.status(500).json({ success: false, message: 'Failed to process checkout' });
  }
};

```

ภาพที่ 4-40 ฟังก์ชัน checkout

4.3.4.5 ส่วนจัดการข้อมูลตะกร้าสินค้า

ไฟล์ cartController.js นี้ทำหน้าที่เป็น Controller ของตะกร้าสินค้า โดยใช้รูปแบบการทำงานแบบ Stateless ซึ่งช่วยลดภาระการเก็บข้อมูลชั่วคราวบนฐานข้อมูล มีรายละเอียดการทำงานดังนี้

- ฟังก์ชัน summarize(cart) การคำนวณยอดรวม

ทำหน้าที่รับค่าอาร์เรย์ของสินค้าในตะกร้ามาวนลูปคำนวณด้วยเมธอด reduce() ของ JavaScript เพื่อหาผลรวมจำนวนชิ้น และราคาสุทธิ อย่างรวดเร็ว ก่อนส่งค่ากลับไปให้ฟังก์ชันหลักใช้งาน

```

const db = require('../models');

function summarize(cart) {
  const totalItems = cart.reduce((sum, item) => sum + item.quantity, 0);
  const totalPrice = cart.reduce((sum, item) => sum + item.quantity * item.price, 0);
  return { totalItems, totalPrice };
}

```

ภาพที่ 4-41 ฟังก์ชัน summarize(cart)

- ฟังก์ชัน getSummary การดึงข้อมูลสรุปตะกร้าสินค้า

ทำหน้าที่รับข้อมูลตะกร้าสินค้าที่ฝั่งหน้าบ้านส่งมา แล้วเรียกใช้ฟังก์ชัน summarize เพื่อคำนวณยอดรวมล่าสุด แล้วส่งข้อมูลตะกร้าพร้อมยอดสรุปกลับไปแสดงผลที่หน้าจอผู้ใช้

```
exports.getSummary = async (req, res) => {
  const cart = Array.isArray(req.body.cart) ? req.body.cart : [];
  const summary = summarize(cart);
  return res.json({ success: true, cart, ...summary });
};
```

ภาพที่ 4-42 ฟังก์ชัน getSummary

- ฟังก์ชัน addToCart การเพิ่มสินค้าลงตะกร้า

เป็นฟังก์ชันที่มีการตรวจสอบเงื่อนไขคือ นำรหัสหนังสือไปค้นหาในตารางหนังสือ ถ้ากรณีที่ใช้เคยหยิบหนังสือไว้ในตะกร้าแล้ว ระบบจะทำการบวกจำนวนขึ้น ถ้าเป็นหนังสือใหม่ที่ไม่เคยใส่ไว้ในตะกร้า ระบบจะทำการสร้างออบเจกต์ใหม่ และเพิ่มลงในอาร์เรย์ตะกร้าสินค้า จากนั้นส่งข้อมูลตะกร้าที่อัปเดตแล้วพร้อมยอดรวมกลับไปให้ผู้ใช้งาน

```
exports.addToCart = async (req, res) => {
  try {
    const { bookId } = req.body;
    const cart = Array.isArray(req.body.cart) ? req.body.cart : [];

    const book = await db.books.findByPk(bookId);
    if (!book) {
      return res.status(404).json({ success: false, message: 'Book not found' });
    }

    const index = cart.findIndex((item) => String(item.bookId) === String(bookId));

    if (index > -1) {
      if (cart[index].quantity >= book.stock_quantity) {
        return res.json({ success: false, message: 'Not enough stock', isMax: true, cart, ...summarize(cart) });
      }
      cart[index].quantity += 1;
    } else {
      if (book.stock_quantity <= 0) {
        return res.json({ success: false, message: 'Out of stock', cart, ...summarize(cart) });
      }

      cart.push({
        bookId: String(book.book_id),
        title: book.title,
        price: Number(book.price),
        img: book.book_img || 'default_book.png',
        quantity: 1,
        maxStock: book.stock_quantity
      });
    }

    return res.json({ success: true, cart, ...summarize(cart) });
  } catch (error) {
    console.error('addToCart error:', error);
    return res.status(500).json({ success: false, message: 'Failed to add to cart' });
  }
};
```

ภาพที่ 4-43 ฟังก์ชัน addToCart

- ฟังก์ชัน updateCart การปรับปรุงและแก้ไขตะกร้าสินค้า

ทำหน้าที่จัดการคำสั่ง (Action) ต่างๆ ที่ผู้ใช้กระทำกับตะกร้าสินค้าของตนเอง โดยค้นหาสินค้าเป้าหมายจาก bookId ในตะกร้า ถ้ามีการเพิ่มจำนวน ระบบจะบวกจำนวนสินค้าขึ้น 1 ชิ้น แต่จะถูกควบคุมไม่ให้เกินสต็อกสูงสุด กรณีลดจำนวน ระบบจะลบจำนวนสินค้าลง 1 ชิ้น แต่มีการดักเงื่อนไขป้องกัน

ไม่ให้จำนวนติดลบกรณีลบสินค้า หากผู้ใช้ต้องการนำสินค้าออกจากตะกร้า ระบบจะใช้เมธอด splice() ในการลบข้อมูลสินค้านั้นออกจากอาร์เรย์อย่างสมบูรณ์

```
exports.updateCart = async (req, res) => {
  try {
    const { bookId, action } = req.body;
    const cart = Array.isArray(req.body.cart) ? req.body.cart : [];

    const index = cart.findIndex((item) => String(item.bookId) === String(bookId));

    if (index > -1) {
      if (action === 'increase' && cart[index].quantity < cart[index].maxStock) {
        cart[index].quantity += 1;
      } else if (action === 'decrease' && cart[index].quantity > 1) {
        cart[index].quantity -= 1;
      } else if (action === 'remove') {
        cart.splice(index, 1);
      }
    }

    return res.json({ success: true, cart, ...summarize(cart) });
  } catch (error) {
    console.error('updateCart error:', error);
    return res.status(500).json({ success: false, message: 'Failed to update cart' });
  }
};
```

ภาพที่ 4-44 ฟังก์ชัน updateCart

4.3.4.6 ส่วนจัดการข้อมูลรายงาน

ไฟล์ reportController.js นี้ทำหน้าที่เป็น Controller ในการดึงข้อมูลหลายตารางมา รวมกัน เพื่อสร้างเป็นข้อมูลสรุป สำหรับแสดงผลหน้ารายงานของแอดมิน มีรายละเอียดการทำงานดังนี้

- ฟังก์ชัน dashboard การแสดงผลสรุปหน้าแรกของแอดมิน

ทำหน้าที่รวบรวมข้อมูลสถิติที่สำคัญแบบภาพรวม เพื่อนับจำนวนรวมของหนังสือทั้งหมด และคำสั่งซื้อทั้งหมด ดึงข้อมูลรายละเอียดคำสั่งซื้อ และจำนวนหนังสือที่ขายไปแล้วทั้งหมด

```
const db = require('../models');
const { Op } = require('sequelize');

exports.dashboard = async (req, res) => {
  try {
    const totalBooks = await db.books.count();
    const totalUsers = await db.users.count({ where: { user_role: 'member' } });
    const totalOrders = await db.order.count();
    const orderDetails = await db.orderDetail.findAll();

    const totalSales = orderDetails.reduce((sum, item) => sum + item.quantity * item.unit_price, 0);
    const totalBooksSold = orderDetails.reduce((sum, item) => sum + item.quantity, 0);

    const recentOrdersRaw = await db.order.findAll({
      limit: 5,
      order: [['order_date', 'DESC']],
      include: [
        { model: db.users, attributes: ['user_name'] },
        { model: db.orderDetail, attributes: ['quantity', 'unit_price'] }
      ],
      distinct: true
    });

    const recentOrders = recentOrdersRaw.map((o) => ({
      order_id: o.order_id,
      order_date: o.order_date,
      status: o.status,
      user_name: o.user ? o.user.user_name : 'Unknown',
      total_items: o.orderDetails.reduce((sum, item) => sum + item.quantity, 0),
      total_amount: o.orderDetails.reduce((sum, item) => sum + item.quantity * item.unit_price, 0)
    })));

    return res.json({
      success: true,
      totalBooks,
      totalUsers,
      totalOrders,
      totalSales,
      totalBooksSold,
      recentOrders
    });
  } catch (error) {
    console.error('dashboard report error:', error);
    return res.status(500).json({ success: false, message: 'Failed to load dashboard' });
  }
};
```

ภาพที่ 4-45 ฟังก์ชัน dashboard

- ฟังก์ชัน salesByCategory รายงานยอดขายแยกตามหมวดหมู่ เป็นฟังก์ชันที่ใช้ดึงรายละเอียดคำสั่งซื้อทั้งหมด เชื่อมโยงข้อมูลผ่าน 3 ตารางเพื่อหาว่าสินค้าที่ขายได้อยู่ในหมวดหมู่ใด ระบบจะทำการเรียงลำดับจากหมวดหมู่ที่ขายดีที่สุดไปน้อยที่สุด

```
exports.salesByCategory = async (req, res) => {
  try {
    const orderDetails = await db.orderDetail.findAll({
      include: [
        { model: db.order, where: { status: { [Op.ne]: 'cancelled' } }, attributes: ['status'] },
        { model: db.books, include: [{ model: db.category, attributes: ['category_id', 'category_name'] }] }
      ]
    });

    const categorySales = {};

    for (const item of orderDetails) {
      const cat = item.book && item.book.category ? item.book.category : null;
      const categoryId = cat ? cat.category_id : 'uncategorized';
      const categoryName = cat ? cat.category_name : 'Uncategorized';

      if (!categorySales[categoryId]) {
        categorySales[categoryId] = { id: categoryId, name: categoryName, totalQty: 0, totalRevenue: 0 };
      }

      categorySales[categoryId].totalQty += item.quantity;
      categorySales[categoryId].totalRevenue += item.quantity * Number(item.unit_price);
    }

    const reportData = Object.values(categorySales).sort((a, b) => b.totalQty - a.totalQty);
    const grandTotalQty = reportData.reduce((sum, item) => sum + item.totalQty, 0);
    const grandTotalRevenue = reportData.reduce((sum, item) => sum + item.totalRevenue, 0);

    return res.json({ success: true, reportData, grandTotalQty, grandTotalRevenue });
  } catch (error) {
    console.error('salesByCategory error:', error);
    return res.status(500).json({ success: false, message: 'Failed to load sales report' });
  }
};
```

ภาพที่ 4-46 ฟังก์ชัน salesByCategory

- ฟังก์ชัน salesByCategoryDetail รายงานยอดขายเจาะลึกแต่ละหมวดหมู่ ฟังก์ชันนี้ทำหน้าที่ดูข้อมูลหนังสือแต่ละเล่มภายในหมวดหมู่ที่เลือก และเรียงลำดับจากหนังสือที่สร้างรายได้มากที่สุดไปน้อยที่สุด

```
exports.salesByCategoryDetail = async (req, res) => {
  try {
    const categoryId = req.params.id;
    let categoryName = 'Uncategorized';

    if (categoryId !== 'uncategorized') {
      const category = await db.category.findPk(categoryId);
      if (category) categoryName = category.category_name;
    }

    const orderDetails = await db.orderDetail.findAll({
      include: [
        { model: db.order, where: { status: { [Op.ne]: 'cancelled' } }, attributes: ['status'] },
        {
          model: db.books,
          where: categoryId !== 'uncategorized' ? { category_id: categoryId } : { category_id: null },
          attributes: ['book_id', 'title']
        }
      ]
    });

    const bookSales = {};

    for (const item of orderDetails) {
      const bookId = item.book.book_id;
      if (!bookSales[bookId]) {
        bookSales[bookId] = { id: bookId, title: item.book.title, qty: 0, revenue: 0 };
      }

      bookSales[bookId].qty += item.quantity;
      bookSales[bookId].revenue += item.quantity * Number(item.unit_price);
    }

    const reportData = Object.values(bookSales).sort((a, b) => b.revenue - a.revenue);
    const grandTotalQty = reportData.reduce((sum, item) => sum + item.qty, 0);
    const grandTotalRevenue = reportData.reduce((sum, item) => sum + item.revenue, 0);

    return res.json({ success: true, categoryName, reportData, grandTotalQty, grandTotalRevenue });
  } catch (error) {
    console.error('salesByCategoryDetail error:', error);
    return res.status(500).json({ success: false, message: 'Failed to load detail report' });
  }
};
```

ภาพที่ 4-47 ฟังก์ชัน salesByCategoryDetail

- ฟังก์ชัน inventoryStatus รายงานสถานะสินค้าคงคลัง

ทำหน้าที่ดึงข้อมูลหนังสือทุกเล่มในระบบ พร้อมชื่อหมวดหมู่ โดยเรียงลำดับจากสินค้าที่เหลือน้อยที่สุดขึ้นมาก่อนระบบคำนวณสถานะอัตโนมัติ วนลูปผ่านข้อมูลหนังสือแต่ละเล่มเพื่อกำหนดป้ายกำกับสถานะตามเงื่อนไขที่กำหนด

```
exports.inventoryStatus = async (req, res) => {
  try {
    const categories = await db.category.findAll({ order: [['category_name', 'ASC']] });
    const books = await db.books.findAll({
      include: [{ model: db.category, attributes: ['category_name'] }],
      order: [['stock_quantity', 'ASC']]
    });

    const reportData = books.map((book) => {
      let statusName = 'In Stock';
      let statusCode = 'in-stock';

      if (book.stock_quantity === 0) {
        statusName = 'Out of Stock';
        statusCode = 'out-of-stock';
      } else if (book.stock_quantity <= 10) {
        statusName = 'Low Stock';
        statusCode = 'low-stock';
      }

      return {
        id: book.book_id,
        title: book.title,
        category: book.category ? book.category.category_name : 'Uncategorized',
        categoryId: book.category_id,
        stock: book.stock_quantity,
        statusName,
        statusCode
      };
    });

    return res.json({ success: true, categories, reportData, totalBooks: reportData.length });
  } catch (error) {
    console.error('inventoryStatus error:', error);
    return res.status(500).json({ success: false, message: 'Failed to load inventory report' });
  }
};
```

ภาพที่ 4-48 ฟังก์ชัน inventoryStatus

4.3.5 โครงสร้างไฟล์แพ็คเกจฝั่ง Backend

ไฟล์ package.json สามารถแบ่งส่วนประกอบหลักออกเป็น 3 ส่วนคือ ส่วนไลบรารีหลักของระบบ (Dependencies) ส่วนไลบรารีสำหรับการพัฒนา (devDependencies) และส่วนตั้งค่าพื้นฐานและสคริปต์คำสั่ง (Scripts)

```
{
  "name": "c-book-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "seed": "node Database/seed.js"
  },
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.5",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "multer": "^2.0.2",
    "sequelize": "^6.37.7",
    "sqlite3": "^5.1.7"
  },
  "devDependencies": {
    "nodemon": "^3.1.11"
  }
}
```

ภาพที่ 4-49 โครงสร้างไฟล์แพ็คเกจฝั่ง Backend

4.4 ส่วนฝั่งหน้าบ้าน (Frontend - UI Web Server)

ใน frontend ไฟล์ app.js นี้ทำหน้าที่เป็น Web Server (ตัวกลาง) ที่คอยรับคำสั่งจากบราวเซอร์ของผู้ใช้งาน นำไปร้องขอข้อมูลจาก Backend API (ผ่านไลบรารี Axios) และนำข้อมูลที่ได้มาประกอบร่างกับไฟล์เทมเพลต (EJS) เพื่อแสดงผลเป็นหน้าเว็บไซต์ที่สวยงาม

4.4.1 ส่วนการตั้งค่าเริ่มต้น

โค้ดส่วนนี้ทำหน้าที่เตรียมการนำเข้าโมดูลที่จำเป็น การตั้งค่าตัวแปรสภาพแวดล้อม การตั้งค่าตัวกลางการเชื่อมต่อ (Axios & Multer) และสร้างฟังก์ชันสำหรับจัดเตรียม HTTP Headers เพื่อใช้ในการยืนยันตัวตนแบบ Stateless ระหว่าง Frontend และ Backend

```
require('dotenv').config();

const express = require('express');
const path = require('path');
const fs = require('fs');
const session = require('express-session');
const axios = require('axios');
const multer = require('multer');
const FormData = require('form-data');

const app = express();

const BACKEND_URL = process.env.BACKEND_URL || 'http://localhost:3000';
const INTERNAL_API_KEY = process.env.INTERNAL_API_KEY || 'cbook_internal_key';
const PORT = Number(process.env.PORT || 5000);

const api = axios.create({
  baseURL: BACKEND_URL,
  timeout: 15000
});

const upload = multer({ dest: path.join(__dirname, 'tmp') });

function apiHeaders(req) {
  const headers = {
    'x-internal-api-key': INTERNAL_API_KEY
  };

  if (req.session?.userId) headers['x-user-id'] = String(req.session.userId);
  if (req.session?.role) headers['x-user-role'] = req.session.role;

  return headers;
}
```

ภาพที่ 4-50 ส่วนการตั้งค่าเริ่มต้นฝั่ง frontend

โค้ดสองฟังก์ชันนี้คือ การคัดกรองสิทธิ์ผู้ใช้งานฝั่งหน้าบ้านคอยตรวจสอบเช็คกว่าผู้ใช้มีสิทธิ์เปิดดูหน้าเว็บหน้านั้นๆ หรือไม่ ก่อนที่จะปล่อยให้ระบบโหลดข้อมูลมาแสดงผล

```
function isLoggedIn(req, res, next) {
  if (req.session?.userId) return next();
  return res.redirect('/login');
}

function isAdmin(req, res, next) {
  if (req.session?.userId && req.session?.role === 'admin') return next();
  return res.redirect('/login');
}
```

ภาพที่ 4-51 ฟังก์ชันคัดกรองสิทธิ์ผู้ใช้งาน

โค้ดส่วนนี้คือส่วนการตั้งค่าระบบและมิดเดิลแวร์พื้นฐานของแอปพลิเคชันครับทำหน้าที่เตรียมความพร้อมให้เซิร์ฟเวอร์ฝั่งหน้าบ้านสามารถแสดงผลหน้าเว็บ (View Engine & Static Files) , การจัดการข้อมูลขาเข้า (Body Parsing Middlewares), การตั้งค่าระบบจดจำสถานะผู้ใช้งาน (Session Management), ตัวแปรส่วนกลางและการจัดการรูปภาพ (Global Variables & Image Proxy)

```
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'ejs');

app.use(express.static(path.join(__dirname, 'public')));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

app.use(
  session({
    secret: process.env.SESSION_SECRET || 'cbook_frontend_secret',
    resave: false,
    saveUninitialized: true,
    cookie: { maxAge: 1000 * 60 * 60 * 24 }
  })
);

app.use((req, res, next) => {
  res.locals.backendBaseUrl = BACKEND_URL;
  next();
});

app.use('/images', (req, res) => {
  res.redirect(`${BACKEND_URL}/images${req.url}`);
});
```

ภาพที่ 4-52 ฟังก์ชันพื้นฐานฝั่งหน้าบ้าน

4.4.2 กำหนดเส้นทาง (Route)

ตารางที่ 4-7 ตารางเส้นทางส่วนของผู้ใช้

การทำงาน	Method	Route Pattern	การประมวลผล / การทำงาน
แสดงหน้าหลักและรายการหนังสือ	GET	/	ดึงข้อมูลหนังสือและหมวดหมู่มาแสดงผล
แสดงหน้าติดต่อเรา	GET	/contact	แสดงข้อมูลช่องทางการติดต่อและหมวดหมู่หนังสือ
เข้าสู่ระบบ	POST	/login	ตรวจสอบบัญชีและสร้าง Session (แยก Admin/Member)
สมัครสมาชิกใหม่	POST	/register	ส่งข้อมูลลงทะเบียนไปยังระบบ Backend
ออกจากระบบ	GET	/logout	ทำลาย Session ทั้งหมดและกลับหน้าหลัก

ตารางที่ 4-8 ตารางเส้นทางส่วนจัดการตะกร้าสินค้า

การทำงาน	Method	Route Pattern	การประมวลผล / การทำงาน
เพิ่มหนังสือลงตะกร้า	POST	/cart/add	บันทึกข้อมูลหนังสือลง Session และซิงค์กับ Backend
ดึงข้อมูลสรุปตะกร้าสินค้า	GET	/cart/get	ดึงรายการหนังสือและยอดรวมในตะกร้า
อัปเดตจำนวน/ลบสินค้าในตะกร้า	POST	/cart/update	แก้ไขจำนวนหนังสือใน Session ตะกร้าสินค้า

ตารางที่ 4-9 ตารางเส้นทางส่วนการชำระเงิน

การทำงาน	Method	Route Pattern	การประมวลผล / การทำงาน
แสดงหน้าสรุปคำสั่งซื้อ	GET	/checkout	คำนวณยอดรวมสุทธิและตรวจสอบสินค้าก่อนชำระเงิน
ยืนยันการสั่งซื้อสินค้า	POST	/checkout/process	บันทึกออเดอร์ลงฐานข้อมูลและล้างตะกร้าสินค้า

ตารางที่ 4-10 ตารางเส้นทางส่วนผู้ดูแลระบบ

การทำงาน	Method	Route Pattern	การประมวลผล / การทำงาน
แสดงแดชบอร์ดสรุปผล	GET	/management	แสดงสถิติภาพรวมจากรายงาน
รายการจัดการหนังสือ	GET	/management/books	แสดงตารางชื่อหนังสือ ค้นหา
เพิ่มหนังสือใหม่	POST	/management/books/create	อัปโหลดรูปและบันทึกข้อมูลหนังสือใหม่
อัปเดตข้อมูลหนังสือ	PUT/POST	/management/books/edit/:id	แก้ไขข้อมูลหนังสือและจัดการไฟล์ภาพ
ลบข้อมูลหนังสือ	DELETE	/management/books/delete/:id	ลบข้อมูลหนังสือผ่านระบบ
รายการจัดการหมวดหมู่	GET	/management/categories	แสดงตารางหมวดหมู่
เพิ่มหมวดหมู่	POST	/management/category/create	จัดการเพิ่มข้อมูลหมวดหมู่หนังสือ
แก้ไขหมวดหมู่	PUT	/management/category/edit/:id	จัดการแก้ไขข้อมูลหมวดหมู่หนังสือ
ลบหมวดหมู่	DELETE	/management/category/delete/:id	จัดการลบข้อมูลหมวดหมู่หนังสือ
รายการจัดการผู้ใช้งาน	GET	/management/users	แสดงรายชื่อสมาชิก
เปลี่ยนสิทธิ์ผู้ใช้งาน	PUT	/management/users/:id/role	อัปเดตสิทธิ์ผู้ใช้ (Admin/Member)
รายการจัดการคำสั่งซื้อ	GET	/management/orders	ตรวจสอบรายการสั่งซื้อทั้งหมดของลูกค้า
อัปเดตสถานะออเดอร์	PUT	/management/orders/:id/status	เปลี่ยนสถานะ

ตารางที่ 4-11 ตารางเส้นทางส่วนของหน้ารายงาน

การทำงาน	Method	Route Pattern	การประมวลผล / การทำงาน
รายงานยอดขายตามหมวดหมู่	GET	/management/reports/sales_category	สรุปยอดขายและรายได้แยกตามประเภทหนังสือ
รายงานยอดขายเจาะลึก	GET	/management/reports/sales-by-category/:id	ดูสถิติการขายรายเล่มภายในหมวดหมู่ที่เลือก
รายงานสถานะสินค้าคงคลัง	GET	/management/reports/inventory-status	ตรวจสอบจำนวนสต็อกคงเหลือของหนังสือทั้งหมด

4.4.3 ส่วนติดต่อกับหน้าผู้ใช้

4.4.3.1 ฟังก์ชันหน้าแรกของเว็บไซต์ (Home Page Route)

ทำหน้าที่เป็นจุดเริ่มต้นของเว็บไซต์ โดยรับหน้าที่ดึงข้อมูลหนังสือและหมวดหมู่จาก Backend API มาแสดงผลให้ผู้ใช้งานเห็น ซึ่งมีขั้นตอนการประมวลผลดังนี้

- มีการรับค่าพารามิเตอร์ ระบบจะทำการดักจับค่าจาก URL Query String นำค่าที่ได้มาแปลงและกำหนดค่าเริ่มต้น (Default Values) ไว้ล่วงหน้าเช่น ดึงข้อมูลทั้งหมด
- การดึงข้อมูลแบบคู่ขนาน (Concurrent Data Fetching) ระบบได้เลือกใช้คำสั่ง Promise.all() ทำให้เซิร์ฟเวอร์หน้าบ้านสามารถส่ง Request ไปขอข้อมูล จาก Backend ได้พร้อมกันในเวลา มีการแนบเงื่อนไขเพื่อให้แสดงเฉพาะหนังสือที่มีสต็อกพร้อมขายเท่านั้น
- การส่งต่อสถานะผู้ใช้งานระบบจะดึงข้อมูลจาก Session ของผู้ใช้งานคนนั้นๆ เช่น ตรวจสอบว่าล็อกอินหรือยัง เพื่อให้เทมเพลต EJS นำไปแสดงบนแถบเมนูนำทาง (Navbar) ได้อย่างถูกต้อง
- มีการจัดการข้อผิดพลาด (Error Handling) ที่ครอบคลุมการทำงานทั้งหมด หาก API หลังบ้านล่ม หรือเกิดข้อผิดพลาดในการดึงข้อมูล ระบบจะพิมพ์ข้อความแสดงข้อผิดพลาดลงในหน้าต่างคอนโซล และส่งสถานะ HTTP 500 Internal Server Error แจ้งให้ผู้ใช้ทราบ เพื่อป้องกันไม่ให้ระบบหน้าบ้านหยุดทำงาน

```

app.get('/', async (req, res) => {
  try {
    const page = Number(req.query.page || 1);
    const search = req.query.search || '';
    const category = req.query.category || 'all';

    const [booksResp, categoriesResp] = await Promise.all([
      api.get('/api/books', {
        params: { page, search, category, onlyAvailable: 1, limit: 12 },
        headers: apiHeaders(req)
      }),
      api.get('/api/categories', {
        params: { limit: 100 },
        headers: apiHeaders(req)
      })
    ]);

    const booksData = booksResp.data;
    const categoriesData = categoriesResp.data;

    if (req.xhr || (req.headers.accept && req.headers.accept.includes('json'))) {
      return res.json({
        books: booksData.books,
        currentPage: booksData.currentPage,
        totalPages: booksData.totalPages,
        userLoggedIn: !!req.session?.userId
      });
    }

    return res.render('home', {
      activePage: 'home',
      books: booksData.books,
      categories: categoriesData.categories,
      currentCategory: category,
      currentPage: booksData.currentPage,
      totalPages: booksData.totalPages,
      searchQuery: search,
      userLoggedIn: !!req.session?.userId,
      cartCount: req.session.cartCount || 0,
      userName: req.session.userName || null,
      backendBaseUrl: BACKEND_URL
    });
  } catch (error) {
    console.error('home error:', error.message);
    return res.status(500).send('Error loading home page');
  }
});

```

ภาพที่ 4-53 ฟังก์ชันหน้าแรกของเว็บไซต์ (Home Page Route)

4.4.3.2 ฟังก์ชัน Contact Page Route

ทำหน้าที่ประมวลผลและแสดงผลหน้า contact.ejs โดยมีการเตรียมข้อมูลที่จำเป็นสำหรับส่วนประกอบต่างๆ บนหน้าเว็บ ดังนี้

- การดึงข้อมูลหมวดหมู่สำหรับเมนูนำทาง
- การกำหนดสถานะเมนู มีการส่งตัวแปร activePage ไปยังเทมเพลต EJS
- การรักษาโครงสร้างของช่องค้นหา เพื่อป้องกันไม่ให้โค้ดส่วนของช่องค้นหา Search Bar ที่ฝังอยู่ในแถบนำทางเกิดข้อผิดพลาด

Bar ที่ฝังอยู่ในแถบนำทางเกิดข้อผิดพลาด

- ระบบยังคงดึงข้อมูลจาก Session เช่น สถานะการเข้าสู่ระบบ เพื่อให้แถบเมนูด้านบนยังคงแสดงชื่อลูกค้าและตัวเลขแจ้งเตือนในตะกร้าสินค้าได้อย่างถูกต้อง

```
app.get('/contact', async (req, res) => {
  try {
    const categoriesResp = await api.get('/api/categories', {
      params: { limit: 100 },
      headers: apiHeaders(req)
    });

    return res.render('contact', {
      categories: categoriesResp.data.categories || [],
      currentCategory: 'all',
      searchQuery: '',
      activePage: 'contact',
      userLoggedIn: !!req.session?.userId,
      cartCount: req.session.cartCount || 0,
      userName: req.session.userName || null,
      backendBaseUrl: BACKEND_URL
    });
  } catch (error) {
    console.error('contact error:', error.message);
    return res.status(500).send('Error loading contact page');
  }
});
```

ภาพที่ 4-54 ฟังก์ชัน Contact Page Route

4.4.3.3 ฟังก์ชันการจัดการหน้าจอลงชื่อเข้าสู่ระบบและสมัครสมาชิก

ระบบใช้คำสั่ง res.render() เพื่อสั่งให้ Express ไปค้นหาไฟล์เทมเพลต EJS ในโฟลเดอร์ views ทำการประมวลผลเป็นหน้าเว็บ HTML ที่สมบูรณ์ แล้วส่งกลับไปแสดงผลที่เบราว์เซอร์ของผู้ใช้งาน

```
app.get('/login', (req, res) => res.render('login', { error: null }));
app.get('/register', (req, res) => res.render('register', { error: null }));
```

ภาพที่ 4-55 ฟังก์ชันการจัดการหน้าจอลงชื่อเข้าสู่ระบบและสมัครสมาชิก

4.4.3.4 ฟังก์ชันการเข้าสู่ระบบ

เมื่อผู้ใช้งานกรอกอีเมลและรหัสผ่านแล้วกดปุ่ม เข้าสู่ระบบ ระบบจะส่งข้อมูลผ่าน HTTP Method แบบ POST ไปตรวจสอบความถูกต้องกับ Backend API จากนั้นการสร้างเซสชันเพื่อจดจำผู้ใช้

ระบบมีการแยกผู้ใช้งานที่แตกต่างกันตามระดับสิทธิ์ของแต่ละบัญชี หากมีสิทธิ์เป็นแอดมิน ระบบจะสั่ง `res.redirect('/management')` แต่หากเป็นสมาชิก ระบบจะพาไปที่หน้าแรก (/)

```
app.post('/login', async (req, res) => {
  try {
    const response = await api.post('/api/users/login', req.body, { headers: apiHeaders(req) });
    const user = response.data.user;

    req.session.userId = user.user_id;
    req.session.userName = user.user_name;
    req.session.userEmail = user.user_email;
    req.session.role = user.user_role;

    if (user.user_role === 'admin') return res.redirect('/management');
    return res.redirect('/');
  } catch (error) {
    const message = error.response?.data?.message || 'Login failed';
    return res.status(400).render('login', { error: message });
  }
});
```

ภาพที่ 4-56 ฟังก์ชันการเข้าสู่ระบบ

4.4.3.5 ฟังก์ชันการสมัครสมาชิก

ระบบจะทำหน้าที่เป็นตัวกลาง โดยใช้คำสั่ง `api.post()` เพื่อส่งมอบข้อมูลทั้งหมดที่รับมาจากฟอร์มไปให้ Backend API เป็นผู้จัดการต่อ หาก Backend API ตอบกลับมว่าสร้างบัญชีผู้ใช้ใหม่สำเร็จ (ไม่มีข้อผิดพลาด) ระบบหน้าบ้านจะข้ามมาทำงานที่คำสั่ง `res.redirect('/login')` หากมีข้อผิดพลาด จะทำการเรนเดอร์หน้า `register` ใหม่อีกครั้ง

```
app.post('/register', async (req, res) => {
  try {
    await api.post('/api/users/register', req.body, { headers: apiHeaders(req) });
    return res.redirect('/login');
  } catch (error) {
    const message = error.response?.data?.message || 'Registration failed';
    return res.status(400).render('register', { error: message });
  }
});
```

ภาพที่ 4-57 ฟังก์ชันการสมัครสมาชิก

4.4.3.6 ฟังก์ชันการออกจากระบบ

ระบบเรียกใช้ฟังก์ชัน `req.session.destroy()` ซึ่งทำหน้าที่ล้างข้อมูลทั้งหมด ที่ผูกติดอยู่กับเซสชันของผู้ใช้งานคนนั้นๆ ทั้งอย่างถาวรเมื่อเซสชันถูกทำลายเรียบร้อยแล้ว ระบบจะสั่ง `res.redirect('/')` เพื่อพาผู้ใช้งานกลับไปยังหน้าแรก (Home Page) ทันที

```
app.get('/logout', (req, res) => {
  req.session.destroy(() => res.redirect('/'));
});
```

ภาพที่ 4-58 ฟังก์ชันการออกจากระบบ

4.4.3.7 ฟังก์ชันการเพิ่มสินค้าลงตะกร้า

ทำหน้าที่รับคำสั่งเมื่อผู้ใช้งานกดปุ่มเพิ่มลงตะกร้า บนหน้าเว็บไซต์ มีการตรวจสอบว่าผู้ใช้งานจะต้องเข้าสู่ระบบแล้วเท่านั้น การส่งข้อมูลไปตรวจสอบสต็อกไปให้ Backend API ช่วยประมวลผล ระบบหน้าบ้านจะทำการ บันทึกข้อมูลตะกร้าชุดใหม่ทับลงไปในเซสชัน พร้อมกับอัปเดตจำนวนชิ้นสินค้าโดยรวม เพื่อให้หน้าเว็บจดจำสถานะล่าสุดของลูกค้าได้

```
app.post('/cart/add', isLoggedIn, async (req, res) => {
  try {
    const response = await api.post(
      '/api/cart/add',
      { bookId: req.body.bookId, cart: req.session.cart || [] },
      { headers: apiHeaders(req) }
    );

    req.session.cart = response.data.cart;
    req.session.cartCount = response.data.totalItems;

    return res.json({
      success: response.data.success,
      totalItems: response.data.totalItems,
      isMax: response.data.isMax || false
    });
  } catch (error) {
    return res.status(500).json({ success: false, message: 'Server error' });
  }
});
```

ภาพที่ 4-59 ฟังก์ชันการเพิ่มสินค้าลงตะกร้า

4.4.3.8 ฟังก์ชันดึงข้อมูลสรุปตะกร้าสินค้า

ทำหน้าที่ดึงข้อมูลสถานะล่าสุดของตะกร้าสินค้า โดยมีขั้นตอนการทำงานคือ ตรวจสอบสิทธิ์ การสื่อสารกับ Backend เพื่อคำนวณยอดสรุป การส่งคืนข้อมูลแบบ JSON กลับไปให้บราวเซอร์ของผู้ใช้งาน

```
app.get('/cart/get', isLoggedIn, async (req, res) => {
  try {
    const response = await api.post(
      '/api/cart/summary',
      { cart: req.session.cart || [] },
      { headers: apiHeaders(req) }
    );

    return res.json(response.data);
  } catch (error) {
    return res.status(500).json({ success: false, message: 'Server error' });
  }
});
```

ภาพที่ 4-60 ฟังก์ชันดึงข้อมูลสรุปตะกร้าสินค้า

4.4.3.9 ฟังก์ชันการปรับปรุงตะกร้าสินค้า

ทำหน้าที่รับคำสั่งการเปลี่ยนแปลงจำนวนสินค้าจากผู้ใช้งานมีขั้นตอนการทำงานคือ รวบรวมข้อมูลและส่งต่อให้ Backend เมื่อประมวลผลและคำนวณยอดตะกร้าใหม่เสร็จสิ้น ระบบฝั่งหน้าบ้าน จะทำการเขียนทับข้อมูลตะกร้าเดิมในเซสชัน และอัปเดตตัวเลขจำนวนสินค้าคงเหลือรวม

```

app.post('/cart/update', isLoggedIn, async (req, res) => {
  try {
    const response = await api.post(
      '/api/cart/update',
      { ...req.body, cart: req.session.cart || [] },
      { headers: apiHeaders(req) }
    );

    req.session.cart = response.data.cart;
    req.session.cartCount = response.data.totalItems;

    return res.json(response.data);
  } catch (error) {
    return res.status(500).json({ success: false, message: 'Server error' });
  }
});

```

ภาพที่ 4-61 ฟังก์ชันการปรับปรุงตะกร้าสินค้า

4.4.3.10 ฟังก์ชันเตรียมข้อมูลสำหรับหน้ายืนยันคำสั่งซื้อ

ทำหน้าที่รวบรวมข้อมูลสรุปยอดทั้งหมดในตะกร้าสินค้า เพื่อนำไปแสดงผลบนหน้ายืนยันการสั่งซื้อ (Checkout) โดยจะตรวจสอบเพื่อให้แน่ใจว่ามีเฉพาะสมาชิกที่เข้าสู่ระบบแล้วเท่านั้น ระบบจะทำการดึงข้อมูลตะกร้าสินค้าจากเซสชันระบบหน้าบ้านจะทำหน้าที่คำนวณตัวเลขสรุปยอดด้วยตนเองผ่านเมธอด .reduce() จากนั้นระบบส่งข้อมูลทั้งหมด ทั้งรายการสินค้า, ข้อมูลการเงิน (ค่าสินค้า, ค่าส่ง, ยอดสุทธิ), และข้อมูลส่วนตัวของลูกค้า (ชื่อ, อีเมลที่ดึงมาจากเซสชัน) ไปให้กับเทมเพลต checkout.ejs

```

app.get('/checkout', isLoggedIn, (req, res) => {
  const cart = req.session.cart || [];
  if (!cart.length) return res.redirect('/');

  const totalItems = cart.reduce((sum, item) => sum + item.quantity, 0);
  const totalPrice = cart.reduce((sum, item) => sum + item.quantity * item.price, 0);
  const shippingFee = 50;

  return res.render('checkout', {
    cart,
    totalItems,
    totalPrice,
    shippingFee,
    grandTotal: totalPrice + shippingFee,
    userLoggedIn: true,
    userName: req.session.userName,
    userEmail: req.session.userEmail,
    cartCount: req.session.cartCount || 0
  });
});

```

ภาพที่ 4-62 ฟังก์ชันเตรียมข้อมูลสำหรับหน้ายืนยันคำสั่งซื้อ

4.4.3.11 ฟังก์ชันหน้ายืนยันการสั่งซื้อ

ทำหน้าที่รวบรวมข้อมูลแล้วส่งผ่าน api.post() ไปให้ Backend เป็นผู้รับช่วงต่อในการทำ Database Transaction ถ้าสั่งซื้อสำเร็จและตัดสต็อกเรียบร้อยแล้ว รีเซ็ตหน่วยความจำเซสชันของลูกค้าคนนั้นให้กลับมาว่างเปล่า เพื่อป้องกันไม่ให้เกิด Bug จากนั้นแล้วทำการแสดงหน้าต่างแจ้งเตือนว่าการสั่งซื้อสำเร็จแล้ว

```

app.post('/checkout/process', isLoggedIn, async (req, res) => {
  try {
    await api.post(
      '/api/orders/checkout/process',
      { ...req.body, cart: req.session.cart || [] },
      { headers: apiHeaders(req) }
    );

    req.session.cart = [];
    req.session.cartCount = 0;

    return res.redirect('/?success=true');
  } catch (error) {
    return res.status(500).send('Error processing checkout');
  }
});

```

ภาพที่ 4-63 ฟังก์ชันหน้ายืนยันการสั่งซื้อ

4.4.4 ส่วนติดต่อกับหน้าผู้ดูแลระบบ

4.4.4.1 ฟังก์ชันหน้าแรกสำหรับผู้ดูแลระบบ

มีการตรวจสอบสิทธิ์ว่าเป็นแอดมิน จากนั้นส่งคำร้องขอไปยังเส้นทาง ของฝั่ง Backend เมื่อ Backend คำนวณเสร็จ และส่งกลับมาเก็บไว้ในตัวแปร data ระบบหน้าบ้านจะทำการส่งต่อข้อมูลไปให้ เทมเพลต management.ejs

```

app.get('/management', isAdmin, async (req, res) => {
  try {
    const response = await api.get('/api/reports/dashboard', { headers: apiHeaders(req) });
    const data = response.data;

    return res.render('management', {
      activePage: 'home',
      ...data,
      userName: req.session.userName,
      userEmail: req.session.userEmail,
      backendBaseUrl: BACKEND_URL
    });
  } catch (error) {
    return res.status(500).send('Error loading dashboard');
  }
});

```

ภาพที่ 4-64 ฟังก์ชันหน้าแรกสำหรับผู้ดูแลระบบ

4.4.4.2 ฟังก์ชันจัดการรายการหนังสือ

ทำหน้าที่แสดงตารางรายการหนังสือทั้งหมดในระบบให้แอดมินโดยมีกรทำงานดังนี้ ตรวจสอบสิทธิ์ระดับแอดมิน ระบบใช้วิธีการส่งคำร้องขอหา Backend API แบบ ทำงานพร้อมกัน 2 เส้นทาง

คือ การดึงรายชื่อหนังสือ และ การดึงหมวดหมู่ทั้งหมด และส่งการค้นหาลงไปให้หลังบ้านเมื่อมีการค้นหา จากนั้นระบบทำการแพ็กข้อมูลทั้งหมดที่จำเป็นส่งไปให้ไฟล์ `list_books.ejs` เพื่อสร้างหน้า UI รวมถึงข้อมูลที่ส่งไปประกอบด้วย อาร์เรย์ของหนังสือ, รายชื่อหมวดหมู่เพื่อให้หน้าเว็บสามารถจำค่าที่แอดมินพิมพ์ค้นหาค้างไว้ได้ แม้จะเปลี่ยนหน้า

```
app.get('/management/books', isAdmin, async (req, res) => {
  try {
    const [booksResp, categoriesResp] = await Promise.all([
      api.get('/api/books', {
        params: {
          page: req.query.page || 1,
          search: req.query.search || '',
          category: req.query.category || '',
          status: req.query.status || '',
          limit: 6
        },
        headers: apiHeaders(req)
      }),
      api.get('/api/categories', { params: { limit: 100 }, headers: apiHeaders(req) })
    ]);

    const data = booksResp.data;
    const categories = categoriesResp.data.categories || [];

    if (req.xhr || (req.headers.accept && req.headers.accept.includes('json'))) {
      return res.json(data);
    }

    return res.render('management/book/list_books', {
      books: data.books || [],
      categories,
      totalPages: data.totalPages || 1,
      currentPage: data.currentPage || 1,
      totalItems: data.totalItems || 0,
      limit: data.limit || 6,
      searchQuery: req.query.search || '',
      filterCategory: req.query.category || '',
      filterStatus: req.query.status || '',
      totalBooksCount: data.totalItems || 0,
      activePage: 'books',
      userName: req.session.userName,
      userEmail: req.session.userEmail,
      backendBaseUrl: BACKEND_URL
    });
  } catch (error) {
    return res.status(500).send('Error loading books');
  }
});
```

ภาพที่ 4-65 ฟังก์ชันจัดการรายการหนังสือ

4.4.4.3 ฟังก์ชันการเพิ่มข้อมูลหนังสือใหม่

ระบบนี้แบ่งการทำงานออกเป็น 2 ส่วนหลัก คือ การเตรียมหน้าฟอร์ม (GET) และ การรับข้อมูลฟอร์มพร้อมไฟล์รูปภาพเพื่อส่งต่อให้หลังบ้าน (POST) โดยมีกระบวนการทำงานดังนี้ เมื่อแอดมิน

เพิ่มหนังสือใหม่" ระบบจะเรียกใช้งานเส้นทาง GET ระบบจะทำการใช้ api.get() ไปดึงรายชื่อหมวดหมู่ทั้งหมดจาก Backend API เมื่อแอดมินกรอกข้อมูลและกดบันทึก ข้อมูลจะถูกส่งไปที่เส้นทาง POST แล้วส่งต่อข้อมูลใช้ api.post() ส่ง FormData ไปที่ Backend เพื่อบันทึกข้อมูล

```
app.get('/management/books/create', isAdmin, async (req, res) => {
  try {
    const response = await api.get('/api/categories', { params: { limit: 100 }, headers: apiHeaders(req) });
    return res.render('management/book/create_book', {
      categories: response.data.categories || [],
      activePage: 'books',
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(500).send('Error loading create form');
  }
});

app.post('/management/books/create', isAdmin, upload.single('book_img'), async (req, res) => {
  try {
    const form = new FormData();
    form.append('title', req.body.title || '');
    form.append('author', req.body.author || '');
    form.append('price', req.body.price || 0);
    form.append('stock_quantity', req.body.stock_quantity || 0);
    form.append('category_id', req.body.category_id || '');

    if (req.file) {
      form.append('book_img', fs.createReadStream(req.file.path), req.file.originalname);
    }

    await api.post('/api/books', form, {
      headers: { ...apiHeaders(req), ...form.getHeaders() }
    });

    if (req.file && fs.existsSync(req.file.path)) {
      fs.unlinkSync(req.file.path);
    }
    return res.redirect('/management/books');
  } catch (error) {
    if (req.file && fs.existsSync(req.file.path)) {
      fs.unlinkSync(req.file.path);
    }
    return res.status(400).send(error.response?.data?.message || 'Create failed');
  }
});
```

ภาพที่ 4-66 ฟังก์ชันการเพิ่มข้อมูลหนังสือใหม่

4.4.4.4 ฟังก์ชันเตรียมหน้าฟอร์มแก้ไขหนังสือ

ทำหน้าที่เตรียมความพร้อมก่อนที่แอดมินจะทำการแก้ไขข้อมูลหนังสือ โดยจะทำการดึงข้อมูลหนังสือเล่มที่ต้องการแก้ไข และข้อมูลที่จำเป็นอื่นๆ มาแสดงผลบนฟอร์ม

```

app.get('/management/books/edit/:id', isAdmin, async (req, res) => {
  try {
    const [bookResp, categoriesResp] = await Promise.all([
      api.get(`/api/books/${req.params.id}`, { headers: apiHeaders(req) }),
      api.get('/api/categories', { params: { limit: 100 }, headers: apiHeaders(req) })
    ]);

    return res.render('management/book/edit_book', {
      book: bookResp.data.book,
      categories: categoriesResp.data.categories || [],
      activePage: 'books',
      currentPage: req.query.page || 1,
      searchQuery: req.query.search || '',
      filterCategory: req.query.category || '',
      filterStatus: req.query.status || '',
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(500).send('Error loading edit form');
  }
});

```

ภาพที่ 4-67 ฟังก์ชันเตรียมหน้าฟอร์มแก้ไขหนังสือ

4.4.4.5 ฟังก์ชันการบันทึกข้อมูลแก้ไขหนังสือ

เมื่อแอดมินทำการแก้ไขข้อมูลในหน้าฟอร์มและบันทึกข้อมูล ระบบจะส่งข้อมูลมาที่เส้นทางนี้เพื่อประมวลผล โดยมีขั้นตอนการทำงานคือ ตรวจสอบการอัปโหลดรูปภาพหากไม่มีการเลือกไฟล์รูปใหม่ ระบบก็จะไม่นำไฟล์มารวมในฟอร์มก่อนส่งไปยัง Backend AP ทำให้ Backend รู้ว่าต้องคงไฟล์รูปภาพเดิมไว้ในฐานข้อมูล และระบบมีการใช้คำสั่ง `fs.unlinkSync(req.file.path)` เพื่อลบไฟล์รูปภาพที่อัปโหลดไว้ในโฟลเดอร์ tmp ที่ทันทีหลังจากส่ง API ให้ฝั่งหลังบ้านเสร็จสิ้น

```

app.post('/management/books/edit/:id', isAdmin, upload.single('book_img'), async (req, res) => {
  try {
    const form = new FormData();
    form.append('title', req.body.title || '');
    form.append('author', req.body.author || '');
    form.append('price', req.body.price || 0);
    form.append('stock_quantity', req.body.stock_quantity || 0);
    form.append('category_id', req.body.category_id || '');

    if (req.file) {
      form.append('book_img', fs.createReadStream(req.file.path), req.file.originalname);
    }

    await api.put(`/api/books/${req.params.id}`, form, {
      headers: { ...apiHeaders(req), ...form.getHeaders() }
    });

    if (req.file && fs.existsSync(req.file.path)) {
      fs.unlinkSync(req.file.path);
    }

    const { page, search, category, status } = req.body;
    return res.redirect(
      `/management/books?page=${page || 1}&search=${encodeURIComponent(search || '')}&category=${encodeURIComponent(
        category || ''
      )}&status=${encodeURIComponent(status || '')}`
    );
  } catch (error) {
    if (req.file && fs.existsSync(req.file.path)) {
      fs.unlinkSync(req.file.path);
    }
    return res.status(400).send(error.response?.data?.message || 'Update failed');
  }
});

```

ภาพที่ 4-68 ฟังก์ชันการบันทึกข้อมูลแก้ไขหนังสือ

4.4.4.6 ฟังก์ชันการลบข้อมูลหนังสือ

ทำหน้าที่รับคำสั่งเมื่อแอดมินกดยืนยันการลบหนังสือออกจากระบบจากหน้าต่างแจ้งเตือน โดยระบบฝั่งหน้าบ้านจะทำหน้าที่เป็นตัวกลาง (Proxy) ส่งคำสั่งต่อไปยัง Backend API หากการส่งคำสั่งไปหลังบ้านล้มเหลว จะส่งสถานะ HTTP 500 พร้อมออบเจกต์แจ้ง เพื่อให้ JavaScript นำไปแสดงกล่องข้อความ Error ให้แอดมินทราบ

```
app.delete('/management/books/delete/:id', isAdmin, async (req, res) => {
  try {
    const response = await api.delete(`/api/books/${req.params.id}`, { headers: apiHeaders(req) });
    return res.json(response.data);
  } catch (error) {
    return res.status(500).json({ success: false, message: 'Delete failed' });
  }
});
```

ภาพที่ 4-69 ฟังก์ชันการลบข้อมูลหนังสือ

4.4.4.7 ฟังก์ชันแสดงผลหน้ารายการหมวดหมู่สินค้า

ทำหน้าที่ดึงข้อมูลหมวดหมู่ทั้งหมดจากฐานข้อมูลมาแสดงผลในรูปแบบตาราง เพื่อให้ผู้ดูแลระบบ (Admin) สามารถค้นหา ดูสถิติ และบริหารจัดการหมวดหมู่ได้อย่างสะดวก

```
app.get('/management/categories/', isAdmin, async (req, res) => {
  try {
    const response = await api.get('/api/categories', {
      params: { page: req.query.page || 1, search: req.query.search || '', limit: 6 },
      headers: apiHeaders(req)
    });
    const data = response.data;

    if (req.xhr || (req.headers.accept && req.headers.accept.includes('json'))) {
      return res.json(data);
    }

    return res.render('management/category/list_cate', {
      categories: data.categories || [],
      activePage: 'categories',
      currentPage: data.currentPage || 1,
      totalPages: data.totalPages || 1,
      totalItems: data.totalItems || 0,
      limit: data.limit || 6,
      searchQuery: req.query.search || '',
      stats: data.stats || { totalCategories: 0, totalBooks: 0 },
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(500).send('Error loading categories');
  }
});
```

ภาพที่ 4-70 ฟังก์ชันแสดงผลหน้ารายการหมวดหมู่สินค้า

4.4.4.8 ฟังก์ชันสร้างหมวดหมู่หนังสือใหม่

การทำงานออกเป็น 2 ส่วน คือ การแสดงหน้าฟอร์ม (GET) และการรับข้อมูลไปบันทึก (POST) โดยทำงานประสานกันเพื่อสร้างหมวดหมู่ใหม่เข้าสู่ระบบ ดังนี้ เมื่อแอดมินเพิ่มหมวดหมู่ระบบจะทำงานแบบ GET จากนั้นจะทำการส่งเรนเดอร์เทมเพลต create_cate.ejs และเมื่อกดบันทึก ข้อมูลจะถูกส่งมาแบบ POST ผ่านตัวแปร req.body แล้วส่งข้อมูลต่อไปยัง Backend API

```

app.get('/management/category/create', isAdmin, (req, res) => {
  res.render('management/category/create_cate', {
    activePage: 'categories',
    userName: req.session.userName,
    userEmail: req.session.userEmail
  });
});

app.post('/management/category/create', isAdmin, async (req, res) => {
  try {
    await api.post('/api/categories', req.body, { headers: apiHeaders(req) });
    return res.redirect('/management/categories/');
  } catch (error) {
    return res.status(400).render('management/category/create_cate', {
      errorMessage: error.response?.data?.message || 'Create category failed',
      oldValue: req.body.category_name,
      activePage: 'categories',
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  }
});

```

ภาพที่ 4-71 ฟังก์ชันสร้างหมวดหมู่หนังสือใหม่

4.4.4.9 ฟังก์ชันการแก้ไขข้อมูลหมวดหมู่หนังสือ

ระบบการแก้ไขหมวดหมู่โดยแบ่งขั้นตอนการทำงานออกเป็น 2 ส่วนดังนี้ เมื่อแอดมินทำการแก้ไขระบบจะไปดึงรายละเอียดของหมวดหมู่นั้นจาก Backend API แล้วเมื่อมีการกดยืนยันการเปลี่ยนแปลง ระบบหน้าบ้านจะใช้คำสั่ง `api.put()` ส่งข้อมูลชื่อหมวดหมู่ใหม่ไปที่ Backend เพื่อทำการอัปเดตข้อมูลในฐานข้อมูล

```

app.get('/management/category/edit/:id', isAdmin, async (req, res) => {
  try {
    const response = await api.get(`/api/categories/${req.params.id}`, { headers: apiHeaders(req) });
    return res.render('management/category/edit_cate', {
      cat: response.data.category,
      activePage: 'categories',
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(404).redirect('/management/categories/');
  }
});

app.post('/management/category/edit/:id', isAdmin, async (req, res) => {
  try {
    await api.put(`/api/categories/${req.params.id}`, req.body, { headers: apiHeaders(req) });
    return res.redirect('/management/categories/');
  } catch (error) {
    return res.status(400).render('management/category/edit_cate', {
      cat: { category_id: req.params.id, category_name: req.body.category_name },
      errorMessage: error.response?.data?.message || 'Update category failed',
      activePage: 'categories',
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  }
});

```

ภาพที่ 4-72 ฟังก์ชันการแก้ไขข้อมูลหมวดหมู่หนังสือ

4.4.4.10 ฟังก์ชันการลบหมวดหมู่หนังสือ

เมื่อต้องการนำหมวดหมู่ที่ไม่ใช้งานแล้วออกจากระบบ โดยมีกลไกการทำงานที่สำคัญดังนี้ เพื่อตรวจสอบว่าผู้ที่ส่งคำสั่งลบเข้ามามีสิทธิ์ระดับผู้ดูแลระบบจริงหรือไม่ เพื่อป้องกันการลบข้อมูลโดยไม่ได้รับอนุญาตระบบเลือกใช้ Method DELETE ในการส่งคำสั่งไปยัง Backend API โดยจะส่งผลลัพธ์กลับเป็นข้อมูล JSON แทนการรีเฟรชหน้าเว็บ ทำให้หน้าจอฝั่งแอดมินสามารถลบแถวข้อมูลออกได้ทันทีเมื่อได้รับสถานะความสำเร็จ

```
app.delete('/management/category/delete/:id', isAdmin, async (req, res) => {
  try {
    const response = await api.delete(`/api/categories/${req.params.id}`, { headers: apiHeaders(req) });
    return res.json(response.data);
  } catch (error) {
    return res.status(400).json(error.response?.data || { success: false, message: 'Delete category failed' });
  }
});
```

ภาพที่ 4-73 ฟังก์ชันการลบหมวดหมู่หนังสือ

4.4.4.11 ฟังก์ชันการข้อมูลผู้ใช้งาน

ฟังก์ชันนี้ทำหน้าที่ดึงรายชื่อผู้ใช้งานทั้งหมดจากฐานข้อมูลมาแสดงผลในรูปแบบตาราง เพื่อให้แอดมินสามารถตรวจสอบสถานะ ค้นหาชื่อ หรือคัดกรองตามระดับสิทธิ์ มีขั้นตอนการทำงานดังนี้ ระบบหน้าบ้านทำการส่งคำร้องขอไปยัง Backend API (/api/users) พร้อมแนบพารามิเตอร์ที่ได้รับจาก URL (req.query) เช่น: สำหรับการแบ่งหน้า สำหรับค้นหาชื่อหรืออีเมลสมาชิก สำหรับกรองเฉพาะ 'member' หรือ 'admin'

```
app.get('/management/users/', isAdmin, async (req, res) => {
  try {
    const response = await api.get('/api/users', {
      params: {
        page: req.query.page || 1,
        search: req.query.search || '',
        role: req.query.role || 'member',
        limit: 6
      },
      headers: apiHeaders(req)
    });

    const data = response.data;

    if (req.xhr || (req.headers.accept && req.headers.accept.includes('json'))) {
      return res.json(data);
    }

    return res.render('management/user/list_users', {
      users: data.users || [],
      totalPages: data.totalPages || 1,
      currentPage: data.currentPage || 1,
      totalItems: data.totalItems || 0,
      limit: data.limit || 6,
      searchQuery: req.query.search || '',
      filterRole: req.query.role || 'member',
      totalUsersCount: data.totalUsersCount || 0,
      totalAdminCount: data.totalAdminCount || 0,
      totalMemberCount: data.totalMemberCount || 0,
      activePage: 'members',
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(500).send('Error loading users');
  }
});
```

ภาพที่ 4-74 ฟังก์ชันการข้อมูลผู้ใช้งาน

4.4.4.12 ฟังก์ชันการแก้ไขสิทธิ์ผู้ใช้งาน

ทางฝั่งเซิร์ฟเวอร์หน้าบ้าน จะทำหน้าที่เป็นตัวกลางส่งต่อคำสั่งไปยัง Backend API (/api/users/\${id}/role) พร้อมแนบข้อมูลยืนยันตัวตนของแอดมิน เพื่อป้องกันไม่ให้นักพัฒนาไปพยายามส่งคำสั่งมาอัปเดตสิทธิ์ของตนเอง ซึ่งระบบใช้ PUT เพื่อระบุว่าเป็นการแก้ไขข้อมูลที่มีอยู่แล้ว โดยส่งรหัสผู้ใช้ผ่านทาง URL Parameter เมื่อ Backend อัปเดตฐานข้อมูลสำเร็จ ระบบหน้าบ้านจะส่งสถานะความสำเร็จกลับไปเป็นรูปแบบ JSON ทันที หากการอัปเดตล้มเหลวระบบจะนำข้อความแจ้งเตือนส่งกลับไปให้หน้าบ้านเพื่อให้ JavaScript แสดงกล่องข้อความ Error

```
app.put('/management/users/:id/role', isAdmin, async (req, res) => {
  try {
    const response = await api.put(`/api/users/${req.params.id}/role`, req.body, { headers: apiHeaders(req) });
    return res.json(response.data);
  } catch (error) {
    return res.status(400).json(error.response?.data || { success: false, message: 'Role update failed' });
  }
});
```

ภาพที่ 4-75 ฟังก์ชันการแก้ไขสิทธิ์ผู้ใช้งาน

4.4.4.13 ฟังก์ชันการลบข้อมูลผู้ใช้งาน

ทำหน้าที่จัดการคำสั่งเมื่อผู้ดูแลระบบต้องการลบบัญชีผู้ใช้งานรายใดรายหนึ่งออกจากฐานข้อมูลถาวร โดยมีกระบวนการทำงานดังนี้ เซิร์ฟเวอร์หน้าบ้านจะรับหน้าที่เป็นตัวกลาง โดยใช้คำสั่ง api.delete() ส่งต่อไปยัง Backend API โดยล็อกสิทธิ์ให้เฉพาะแอดมินเท่านั้นที่สามารถเข้าถึงคำสั่งลบได้ ซึ่งในการลบผู้ใช้งาน ผู้ดูแลระบบจะไม่สามารถลบบัญชีของตนเองได้เพื่อเป็นการป้องกันข้อผิดพลาด หาก Backend ตรวจพบเงื่อนไขเหล่านี้และปฏิเสธการลบ ระบบหน้าบ้านจะดักจับในบล็อก catch และนำข้อความแจ้งเตือนที่เจาะจง (error.response?.data) ส่งกลับไปให้หน้าบ้านทันที

```
app.delete('/management/users/:id', isAdmin, async (req, res) => {
  try {
    const response = await api.delete(`/api/users/${req.params.id}`, { headers: apiHeaders(req) });
    return res.json(response.data);
  } catch (error) {
    return res.status(400).json(error.response?.data || { success: false, message: 'Delete failed' });
  }
});
```

ภาพที่ 4-76 ฟังก์ชันการลบข้อมูลผู้ใช้งาน

4.4.4.14 ฟังก์ชันจัดการรายการสั่งซื้อ

ทำหน้าที่ดึงข้อมูลการสั่งซื้อทั้งหมดจากฐานข้อมูลมาแสดงผลในรูปแบบตาราง มีขั้นตอนการทำงานดังนี้ โดยระบบทำการส่งคำร้องขอไปยัง Backend API โดยแนบพารามิเตอร์คือ สำหรับระบบแบ่งหน้า สำหรับค้นหาออเดอร์ด้วยรหัสคำสั่งซื้อหรือชื่อลูกค้า สำหรับกรองออเดอร์ตามสถานะ เพื่อรองรับการงานต่าง ๆ เมื่อได้รับข้อมูลมาก็จะทำการส่งไปให้เทมเพลต EJS หากเกิดข้อผิดพลาดในการดึงข้อมูลจาก Backend จะตอบด้วยสถานะ 500 Error loading orders เพื่อป้องกันการแสดงผลข้อมูลที่ผิดพลาด

```

app.get('/management/orders', isAdmin, async (req, res) => {
  try {
    const response = await api.get('/api/orders', {
      params: {
        page: req.query.page || 1,
        search: req.query.search || '',
        status: req.query.status || '',
        limit: 8
      },
      headers: apiHeaders(req)
    });

    const data = response.data;

    if (req.xhr || (req.headers.accept && req.headers.accept.includes('json'))) {
      return res.json(data);
    }

    return res.render('management/orders/list_orders', {
      orders: data.orders || [],
      currentPage: data.currentPage || 1,
      totalPages: data.totalPages || 1,
      totalItems: data.totalItems || 0,
      limit: data.limit || 10,
      searchQuery: req.query.search || '',
      filterStatus: req.query.status || '',
      activePage: 'orders',
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(500).send('Error loading orders');
  }
});

```

ภาพที่ 4-77 ฟังก์ชันจัดการรายการสั่งซื้อ

4.4.4.15 ฟังก์ชันแสดงรายละเอียดคำสั่งซื้อ

เมื่อแอดมินคลิกดูรายละเอียด ระบบจะขอข้อมูลจาก Backend ซึ่งรวมถึงข้อมูลลูกค้า, รายการหนังสือที่สั่งซื้อ, และราคาสุทธิ ข้อมูลที่ได้รับจะประกอบไปด้วย ข้อมูลทั่วไปของคำสั่งซื้อและที่อยู่จัดส่ง ยอดรวมราคาสินค้าทั้งหมดในออเดอร์นั้น

```

app.put('/management/orders/:id/status', isAdmin, async (req, res) => {
  try {
    const response = await api.put('/api/orders/${req.params.id}/status', req.body, { headers: apiHeaders(req) });
    return res.json(response.data);
  } catch (error) {
    return res.status(400).json(error.response?.data || { success: false, message: 'Status update failed' });
  }
});

app.get('/management/orders/:id', isAdmin, async (req, res) => {
  try {
    const response = await api.get('/api/orders/${req.params.id}', { headers: apiHeaders(req) });
    const data = response.data;

    return res.render('management/orders/order_detail', {
      order: data.order,
      itemsTotal: data.itemsTotal,
      grandTotal: data.grandTotal,
      activePage: 'orders',
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(404).send('Order not found');
  }
});

```

ภาพที่ 4-78 ฟังก์ชันแสดงรายละเอียดคำสั่งซื้อ

4.4.4.16 ฟังก์ชันรายงานสรุยอดขายแยกตามหมวดหมู่

ระบบหน้าบ้านจะทำการร้องขอข้อมูลไปยัง Backend API ข้อมูลที่ได้รับจาก Backend จะไม่ใช่ข้อมูลออเดอร์ดิบ แต่เป็นข้อมูลที่ผ่านการประมวลผลสรุยอด Group by Category มาให้เรียบร้อยแล้ว เพื่อลดภาระการคำนวณของฝั่งหน้าบ้าน เมื่อได้ข้อมูลแล้วระบบส่งตัวแปรที่จำเป็นไปให้เทมเพลต sales_category.ejs เพื่อทำการแสดงผลต่อไป

```
app.get('/management/reports/sales_category', isAdmin, async (req, res) => {
  try {
    const response = await api.get('/api/reports/sales_category', { headers: apiHeaders(req) });
    const data = response.data;

    return res.render('management/reports/sales_category', {
      activePage: 'reports1',
      reportData: data.reportData || [],
      grandTotalQty: data.grandTotalQty || 0,
      grandTotalRevenue: data.grandTotalRevenue || 0,
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(500).send('Error loading report');
  }
});
```

ภาพที่ 4-79 ฟังก์ชันรายงานสรุยอดขายแยกตามหมวดหมู่

4.4.4.17 ฟังก์ชันรายงานยอดขายตามหมวดหมู่ที่เลือก

โดยจะแสดงรายชื่อหนังสือทุกเล่มที่อยู่ภายใต้หมวดหมู่นั้นพร้อมยอดขายรายตัว มีขั้นตอนการทำงานดังนี้ ระบบหน้าบ้านส่งคำขอไปยัง Backend API เพื่อนำข้อมูลมาแสดงที่เทมเพลต sales_category_detail.ejs เพื่อจัดรูปแบบเป็นตารางสรุปผลที่ดูง่าย ข้อมูลที่ได้รับจะมีความละเอียดสูงขึ้น โดยประกอบด้วย ชื่อหมวดหมู่ที่กำลังเรียกดู รายการหนังสือทั้งหมดในหมวดหมู่นั้น พร้อมข้อมูลยอดขายแยกตามรายเล่ม

```
app.get('/management/reports/sales-by-category/:id', isAdmin, async (req, res) => {
  try {
    const response = await api.get(`/api/reports/sales-by-category/${req.params.id}`, { headers: apiHeaders(req) });
    const data = response.data;

    return res.render('management/reports/sales_category_detail', {
      activePage: 'reports1',
      categoryName: data.categoryName,
      reportData: data.reportData || [],
      grandTotalQty: data.grandTotalQty || 0,
      grandTotalRevenue: data.grandTotalRevenue || 0,
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(500).send('Error loading detail report');
  }
});
```

ภาพที่ 4-80 ฟังก์ชันรายงานยอดขายตามหมวดหมู่ที่เลือก

4.4.4.18 ฟังก์ชันรายงานสถานะสินค้าคงคลัง

ทำหน้าที่ดึงข้อมูลสรุปด้านจำนวนสินค้าคงเหลือของหนังสือทั้งหมดในระบบเพื่อให้ผู้ดูแลระบบสามารถตรวจสอบความเคลื่อนไหวของสต็อกได้ มีขั้นตอนการทำงานดังนี้ ระบบหน้าบ้านส่งคำขอไปยัง Backend API เมื่อได้รับข้อมูลระบบจะส่งตัวแปรที่จำเป็นไปให้เทมเพลต `inventory_status.ejs` เพื่อสร้างหน้าสรุปผล

```
app.get('/management/reports/inventory-status', isAdmin, async (req, res) => {
  try {
    const response = await api.get('/api/reports/inventory-status', { headers: apiHeaders(req) });
    const data = response.data;

    return res.render('management/reports/inventory_status', {
      activePage: 'reports2',
      reportData: data.reportData || [],
      categories: data.categories || [],
      totalBooks: data.totalBooks || 0,
      userName: req.session.userName,
      userEmail: req.session.userEmail
    });
  } catch (error) {
    return res.status(500).send('Error loading inventory report');
  }
});
```

ภาพที่ 4-81 ฟังก์ชันรายงานสถานะสินค้าคงคลัง

บทที่ 5

ปัญหา อุปสรรค และแนวทางการแก้ไข

5.1 สรุปผลการดำเนินงาน

โครงการนี้ประสบความสำเร็จในการพัฒนาระบบร้านหนังสือออนไลน์ C-BOOK ด้วยสถาปัตยกรรมแบบแยกส่วน (Decoupled Architecture) ที่เชื่อมต่อระหว่างหน้าบ้านและหลังบ้านผ่าน RESTful API อย่างมีประสิทธิภาพ โดยครอบคลุมทั้งระบบซื้อขายสินค้าสำหรับลูกค้าและระบบบริหารจัดการข้อมูลแบบ CRUD สำหรับผู้ดูแลระบบที่ใช้งานง่ายและรวดเร็ว พร้อมทั้งมีระบบรายงานสถิติยอดขายและสถานะสินค้าคงคลังแบบ Real-time เพื่อช่วยในการวิเคราะห์ข้อมูลทางธุรกิจและวางแผนสต็อกสินค้าได้อย่างแม่นยำ

5.2 ปัญหาและอุปสรรค

5.2.1 ข้อจำกัดด้านการรับส่งข้อมูลไฟล์ภาพระหว่างเซิร์ฟเวอร์ เนื่องจากโครงสร้างระบบเป็นแบบแยกส่วน การส่งไฟล์รูปภาพหนังสือจากส่วน Frontend ไปยังส่วน Backend มีความซับซ้อนกว่าสูง ทำให้ต้องมีการจัดการลำดับการไหลของข้อมูลที่รัดกุม

5.2.2 การจัดการไฟล์ขยะในระบบระหว่างกระบวนการอัปโหลดหรือแก้ไขข้อมูล หากเกิดข้อผิดพลาดในขั้นตอนการส่งข้อมูลข้ามเซิร์ฟเวอร์ อาจทำให้มีไฟล์รูปภาพตกค้างอยู่ในโฟลเดอร์ชั่วคราวของเซิร์ฟเวอร์

5.3 แนวทางการแก้ไข

5.3.1 การใช้ FormData สามารถแก้ปัญหาการส่งรูปภาพโดยใช้ไลบรารี Multer ร่วมกับ FormData เพื่อแปลงไฟล์เป็นสตริมข้อมูล ทำให้สามารถส่งไฟล์ข้ามเซิร์ฟเวอร์ได้อย่างมีประสิทธิภาพ

5.3.2 ระบบใช้คำสั่ง fs.unlinkSync เพื่อลบไฟล์ชั่วคราวทิ้งทันทีหลังจากจบกระบวนการทำงาน ไม่ว่าจะการส่งข้อมูลจะสำเร็จหรือล้มเหลว เพื่อรักษาพื้นที่ของทรัพยากรเซิร์ฟเวอร์